

Explore More

Subscription : Premium CDAC NOTES & MATERIAL @99



Contact to Join
Premium Group



Click to Join
Telegram Group

<CODEWITHARRAY'S/>

For More E-Notes

Join Our Community to stay Updated

TAP ON THE ICONS TO JOIN!

	codewitharrays.in freelance project available to buy contact on 8007592194	
SR.NO	Project NAME	Technology
1	Online E-Learning Platform Hub	React+Springboot+MySql
2	PG Mates / RoomSharing / Flat Mates	React+Springboot+MySql
3	Tour and Travel management System	React+Springboot+MySql
4	Election commition of India (online Voting System)	React+Springboot+MySql
5	HomeRental Booking System	React+Springboot+MySql
6	Event Management System	React+Springboot+MySql
7	Hotel Management System	React+Springboot+MySql
8	Agriculture web Project	React+Springboot+MySql
9	AirLine Reservation System / Flight booking System	React+Springboot+MySql
10	E-commerce web Project	React+Springboot+MySql
11	Hospital Management System	React+Springboot+MySql
12	E-RTO Driving licence portal	React+Springboot+MySql
13	Transpotation Services portal	React+Springboot+MySql
14	Courier Services Portal / Courier Management System	React+Springboot+MySql
15	Online Food Delivery Portal	React+Springboot+MySql
16	Muncipal Corporation Management	React+Springboot+MySql
17	Gym Management System	React+Springboot+MySql
18	Bike/Car ental System Portal	React+Springboot+MySql
19	CharityDonation web project	React+Springboot+MySql
20	Movie Booking System	React+Springboot+MySql

freelance_Project available to buy contact on 8007592194		
21	Job Portal web project	React+Springboot+MySql
22	LIC Insurance Portal	React+Springboot+MySql
23	Employee Management System	React+Springboot+MySql
24	Payroll Management System	React+Springboot+MySql
25	RealEstate Property Project	React+Springboot+MySql
26	Marriage Hall Booking Project	React+Springboot+MySql
27	Online Student Management portal	React+Springboot+MySql
28	Resturant management System	React+Springboot+MySql
29	Solar Management Project	React+Springboot+MySql
30	OneStepService LinkLabourContractor	React+Springboot+MySql
31	Vehical Service Center Portal	React+Springboot+MySql
32	E-wallet Banking Project	React+Springboot+MySql
33	Blogg Application Project	React+Springboot+MySql
34	Car Parking booking Project	React+Springboot+MySql
35	OLA Cab Booking Portal	React+NextJs+Springboot+MySql
36	Society management Portal	React+Springboot+MySql
37	E-College Portal	React+Springboot+MySql
38	FoodWaste Management Donate System	React+Springboot+MySql
39	Sports Ground Booking	React+Springboot+MySql
40	BloodBank mangement System	React+Springboot+MySql

41	Bus Tickit Booking Project	React+Springboot+MySql
42	Fruite Delivery Project	React+Springboot+MySql
43	Woodworks Bed Shop	React+Springboot+MySql
44	Online Dairy Product sell Project	React+Springboot+MySql
45	Online E-Pharma medicine sell Project	React+Springboot+MySql
46	FarmerMarketplace Web Project	React+Springboot+MySql
47	Online Cloth Store Project	React+Springboot+MySql
48	Train Ticket Booking Project	React+Springboot+MySql
49	Quizz Application Project	JSP+Springboot+MySql
50	Hotel Room Booking Project	React+Springboot+MySql
51	Online Crime Reporting Portal Project	React+Springboot+MySql
52	Online Child Adoption Portal Project	React+Springboot+MySql
53	online Pizza Delivery System Project	React+Springboot+MySql
54	Online Social Complaint Portal Project	React+Springboot+MySql
55	Electric Vehical management system Project	React+Springboot+MySql
56	Online mess / Tiffin management System Project	React+Springboot+MySql
57		React+Springboot+MySql
58		React+Springboot+MySql
59		React+Springboot+MySql
60		React+Springboot+MySql

Spring Boot + React JS + MySQL Project List

Sr.No	Project Name	YouTube Link
1	Online E-Learning Hub Platform Project	https://youtu.be/KMjyBaWmgzg?si=YckHuNzs7eC84-IW
2	PG Mate / Room sharing/Flat sharing	https://youtu.be/4P9clHg3wvk?si=4uEsi0962CG6Xodp
3	Tour and Travel System Project Version 1.0	https://youtu.be/-UHOBywHaP8?si=KHHfE_A0uv725f12
4	Marriage Hall Booking	https://youtu.be/VXz0kZQi5to?si=ILOS-QG3TpAFP5k7
5	Ecommerce Shopping project	https://youtu.be/vJ_C6LkhrZ0?si=YhcBylSErvdn7paq
6	Bike Rental System Project	https://youtu.be/FlzsAmIBCbk?si=7ujQTJqEgkQ8ju2H
7	Multi-Restaurant management system	https://youtu.be/pvV-pM2Jf3s?si=PgvnT-yFc8ktrDxB
8	Hospital management system Project	https://youtu.be/lynlouBZvY4?si=CXzQs3BsRkjKhZCw
9	Municipal Corporation system Project	https://youtu.be/cVMx9NVyl4I?si=qX0oQt-GT-LR_5jF
10	Tour and Travel System Project version 2.0	https://youtu.be/_4u0mB9mHXE?si=gDiAhKBowi2gNUKZ

Sr.No	Project Name	YouTube Link
11	Tour and Travel System Project version 3.0	https://youtu.be/Dm7nOdpasWg?si=P_Lh2gcOFhlyudug
12	Gym Management system Project	https://youtu.be/J8_7Zrkg7ag?si=LcxV51ynfUB7OptX
13	Online Driving License system Project	https://youtu.be/3yRzsMs8TLE?si=JRI_z4FDx4Gmt7fn
14	Online Flight Booking system Project	https://youtu.be/m755rOwdk8U?si=HURvAY2VnizlyJlh
15	Employee management system project	https://youtu.be/ID1iE3W_GRw?si=Y_jv1xV_BljhrD0H
16	Online student school or college portal	https://youtu.be/4A25aEKfei0?si=RoVgZtxMk9TPdQvD
17	Online movie booking system project	https://youtu.be/Lfjv_U74SC4?si=fiDvrhhrjb4KSIsm
18	Online Pizza Delivery system project	https://youtu.be/Tp3izreZ458?si=8eWAOzA8SVdNwlyM
19	Online Crime Reporting system Project	https://youtu.be/0UlzReSk9tQ?si=6vN0e70TVY1GOwPO
20	Online Children Adoption Project	https://youtu.be/3T5HC2HKyT4?si=bntP78niYH802I7N

Q #2) What are the features of JAVA?

Answer: Features of Java are as follows:

- **OOP concepts**
 - Object-oriented
 - Inheritance
 - Encapsulation
 - Polymorphism
 - Abstraction
- **Platform independent:** A single program works on different platforms without any modification.
- **High Performance:** JIT (Just In Time compiler) enables high performance in Java. JIT converts the bytecode into machine language and then JVM starts the execution.
- **Multi-threaded:** A flow of execution is known as a Thread. JVM creates a thread which is called the main thread. The user can create multiple threads by extending the thread class or by implementing the Runnable interface.

Q #3) How does Java enable high performance?

Answer: Java uses Just In Time compiler to enable high performance. It is used to convert the instructions into bytecodes.

Q #4) Name the Java IDE's?

Answer: Eclipse and NetBeans are the IDE's of JAVA.

Q #5) What do you mean by Constructor?

Answer: Constructor can be explained in detail with enlisted points:

- When a new object is created in a program a constructor gets invoked corresponding to the class.
- The constructor is a method which has the same name as the class name.
- If a user doesn't create a constructor implicitly a default constructor will be created.
- The constructor can be overloaded.
- If the user created a constructor with a parameter then he should create another constructor explicitly without a parameter.

Q #6) What is meant by the Local variable and the Instance variable?

Answer:

Local variables are defined in the method and scope of the variables that exist inside the method itself.

Instance variable is defined inside the class and outside the method and the scope of the variables exists throughout the class.

Q #7) What is a Class?

Answer: All Java codes are defined in a Class. It has variables and methods.

Variables are attributes which define the state of a class.

Methods are the place where the exact business logic has to be done. It contains a set of statements (or) instructions to satisfy the particular requirement.

Example:

```
public class Addition{ //Class name declaration
int a = 5; //Variable declaration
int b= 5;
public void add(){ //Method declaration
int c = a+b;
}
}
```

Q #8) What is an Object?

Answer: An instance of a class is called an object. The object has state and behavior.

Whenever the JVM reads the “new()” keyword then it will create an instance of that class.

Example:

```
public class Addition{
public static void main(String[] args){
Addion add = new Addition();//Object creation
}
}
```

The above code creates the object for the Addition class.

Q #9)What are the OOPs concepts?

Answer: OOPs concepts include:

- Inheritance
- Encapsulation
- Polymorphism
- Abstraction
- Interface

Suggested Read =>> [Top OOPs Interview Questions](#)

Q #10) What is Inheritance?

Answer: Inheritance means one class can extend to another class. So that the codes can be reused from one class to another class. The existing class is known as the Super class whereas the derived class is known as a sub class.

Example:

```
Super class:
public class Manipulation() {
}
Sub class:
public class Addition extends Manipulation() {
}
```

Inheritance is only applicable to the public and protected members only. Private members can't be inherited.

Q #11) What is Encapsulation?

Answer: Purpose of Encapsulation:

- Protects the code from others.
- Code maintainability.

Example:

We are declaring 'a' as an integer variable and it should not be negative.

```
public class Addition() {
int a=5;
}
```

If someone changes the exact variable as "~~a~~ = -5" then it is bad.

In order to overcome the problem we need to follow the steps below:

- We can make the variable private or protected.
- Use public accessor methods such as set<property> and get<property>.

So that the above code can be modified as:

```
public class Addition() {
private int a = 5; //Here the variable is marked as private
}
```

The code below shows the getter and setter.

Conditions can be provided while setting the variable.

```
get A() {
}
set A(int a) {
if(a>0){// Here condition is applied
.....
}
}
```

For encapsulation, we need to make all the instance variables private and create setter and getter for those variables. Which in turn will force others to call the setters rather than access the data directly.

Q #12) What is Polymorphism?

Answer: Polymorphism means many forms.

A single object can refer to the super-class or sub-class depending on the reference type which is called polymorphism.

Example:

```
Public class Manipulation(){ //Super class
public void add() {
}
}
public class Addition extends Manipulation(){ // Sub class
public void add() {
}
public static void main(String args[]){
Manipulation addition = new Addition();//Manipulation is reference type and
Addition is reference type
addition.add();
}
}
```

Using the Manipulation reference type we can call the Addition class “add()” method. This ability is known as Polymorphism. Polymorphism is applicable for **overriding** and not for **overloading**.

Q #13) What is meant by Method Overriding?

Answer: Method overriding happens if the sub-class method satisfies the below conditions with the Super-class method:

- Method name should be the same
- The argument should be the same
- Return type should also be the same

The key benefit of overriding is that the Sub-class can provide some specific information about that sub-class type than the super-class.

Example:

```
public class Manipulation{ //Super class
public void add() {
.....
}
}

Public class Addition extends Manipulation() {
Public void add() {
.....
}
```

```

}
Public static void main(String args[]){
Manipulation addition = new Addition(); //Polimorphism is applied
addition.add(); // It calls the Sub class add() method
}
}

```

addition.add() method calls the add() method in the Sub-class and not the parent class. So it overrides the Super-class method and is known as Method Overriding.

Q #14) What is meant by Overloading?

Answer: Method overloading happens for different classes or within the same class.

For method overloading, sub-class method should satisfy the below conditions with the Super-class method (or) methods in the same class itself:

- Same method name
- Different argument types
- There may be different return types

Example:

```

public class Manipulation{ //Super class
public void add(String name){ //String parameter
.....
}
}

Public class Addition extends Manipulation(){
Public void add(){//No Parameter
.....
}
Public void add(int a){ //integer parameter

}
Public static void main(String args[]){
Addition addition = new Addition();
addition.add();
}
}

```

Here the add() method has different parameters in the Addition class is overloaded in the same class as with the super-class.

Note: Polymorphism is not applicable for method overloading.

Q #15) What is meant by Interface?

Answer: Multiple inheritances cannot be achieved in java. To overcome this problem the Interface concept is introduced.

An interface is a template which has only method declarations and not the method implementation.

Example:

```
Public abstract interface IManipulation{ //Interface declaration
Public abstract void add();//method declaration
public abstract void subtract();
}
```

- All the methods in the interface are internally **public abstract void**.
- All the variables in the interface are internally **public static final** that is constants.
- Classes can implement the interface and not extends.
- The class which implements the interface should provide an implementation for all the methods declared in the interface.

```
public class Manipulation implements IManipulation{ //Manipulation class
uses the interface
Public void add(){
.....
}
Public void subtract(){
.....
}
}
```

Q #16) What is meant by Abstract class?

Answer: We can create the Abstract class by using the “Abstract” keyword before the class name. An abstract class can have both “Abstract” methods and “Non-abstract” methods that are a concrete class.

Abstract method:

The method which has only the declaration and not the implementation is called the abstract method and it has the keyword called “abstract”. Declarations ends with a semicolon.

Example:

```
public abstract class Manipulation{
public abstract void add();//Abstract method declaration
Public void subtract(){
}
}
```

- An abstract class may have a non- abstract method also.
- The concrete Subclass which extends the Abstract class should provide the implementation for abstract methods.

Q #17) Difference between Array and Array List.

Answer: The Difference between Array and Array List can be understood from the table below:

Array

Size should be given at the time of array declaration.

```
String[] name = new String[2]
```

To put an object into array we need to specify the index.

```
name[1] = "book"
```

Array is not type parameterized

Array List

Size may not be required. It changes the size dynamically.

```
ArrayList name = new ArrayList
```

No index required.

```
name.add("book")
```

ArrayList in java 5.0 are parameterized.

Eg: This angle bracket is a type parameter which means a list of String.

Q #18) Difference between String, String Builder, and String Buffer.

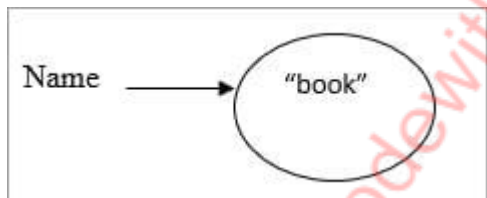
Answer:

String: String variables are stored in a “constant string pool”. Once the string reference changes the old value that exists in the “constant string pool”, it cannot be erased.

Example:

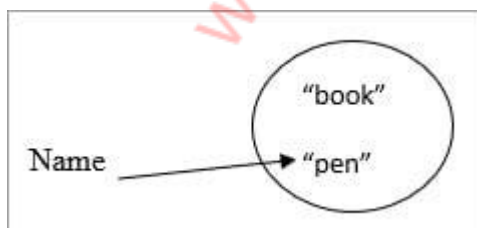
```
String name = "book";
```

Constant string pool



If the name-value has changed from “book” to “pen”.

Constant string pool



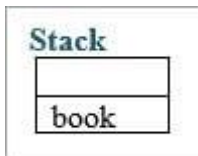
Then the older value remains in the constant string pool.

String Buffer:

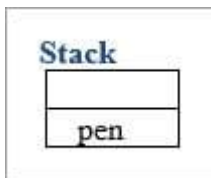
- Here string values are stored in a stack. If the values are changed then the new value replaces the older value.
- The string buffer is synchronized which is thread-safe.
- Performance is slower than the String Builder.

Example:

String Buffer name ="book";



Once the name value has been changed to “pen” then the “book” is erased in the stack.



String Builder:

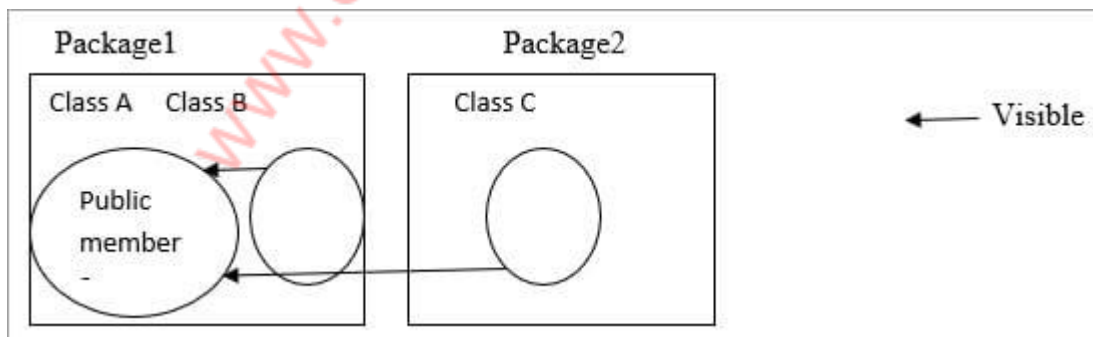
This is the same as String Buffer except for the String Builder which is not threaded safely that is not synchronized. So obviously the performance is fast.

Q #19) Explain about Public and Private access specifiers.

Answer: Methods and instance variables are known as members.

Public:

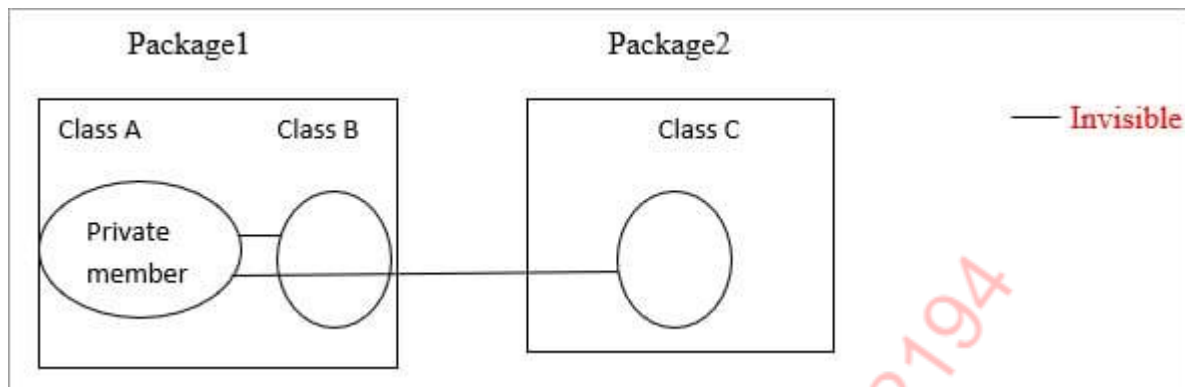
Public members are visible in the same package as well as the outside package that is for other packages.



Public members of Class A are visible to Class B (same package) as well as Class C (different packages).

Private:

Private members are visible in the same class only and not for the other classes in the same package as well as classes in the outside packages.

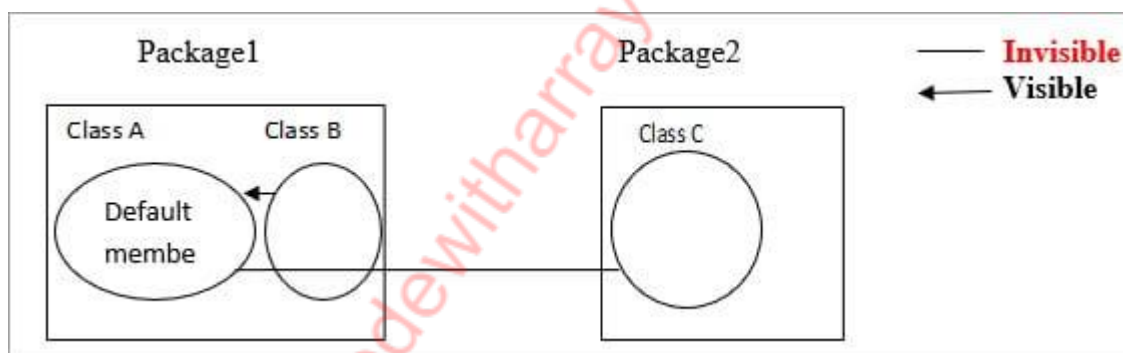


Private members in class A are visible only in that class. It is invisible for class B as well as class C.

Q #20) Difference between Default and Protected access specifiers.

Answer:

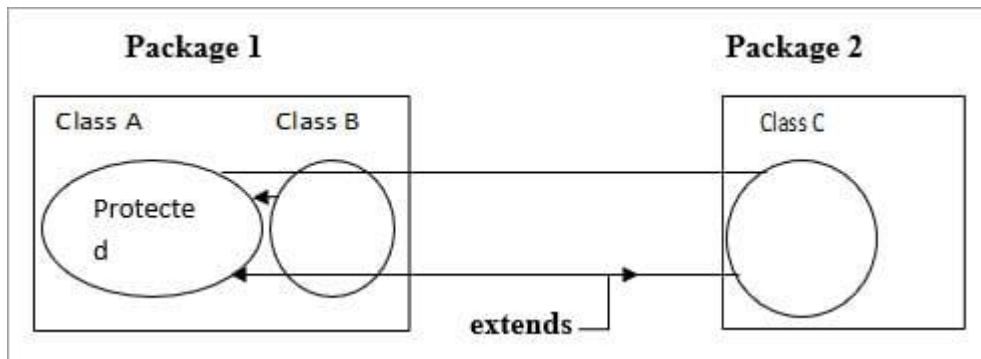
Default: Methods and variables declared in a class without any access specifiers are called default.



Default members in Class A are visible to the other classes which are inside the package and invisible to the classes which are outside the package.

So Class A members are visible to Class B and invisible to Class C.

Protected:



Protected is the same as Default but if a class extends then it is visible even if it is outside the package.

Class A members are visible to Class B because it is inside the package. For Class C it is invisible but if Class C extends Class A then the members are visible to Class C even if it is outside the package.

Q #21) Difference between HashMap and Hashtable.

Answer: The difference between HashMap and Hashtable can be seen below:

HashMap	Hashtable
Methods are not synchronized	Key methods are synchronized
Not thread safety	Thread safety
Iterator is used to iterate the values	Enumerator is used to iterate the values
Allows one null key and multiple null values	Doesn't allow anything that is null
Performance is high than Hashtable	Performance is slow

Q #22) Difference between HashSet and TreeSet.

Answer: The difference between HashSet and TreeSet can be seen below:

HashSet	TreeSet
Inserted elements are in random order	Maintains the elements in the sorted order
Can able to store null objects	Couldn't store null objects
Performance is fast	Performance is slow

Q #23) Difference between Abstract class and Interface.

Answer: The differences between Abstract Class and Interface are as follows:

Abstract Class:

- Abstract classes have a default constructor and it is called whenever the concrete subclass is instantiated.
- It contains Abstract methods as well as Non-Abstract methods.

- The class which extends the Abstract class shouldn't require the implementation of all the methods, only Abstract methods need to be implemented in the concrete sub-class.
- Abstract class contains instance variables.

Interface:

- It doesn't have any constructor and couldn't be instantiated.
- The abstract method alone should be declared.
- Classes that implement the interface should provide the implementation for all the methods.
- The interface contains only constants.

Q #24) What is the meaning of Collections in Java?

Answer: Collection is a framework that is designed to store the objects and manipulate the design to store the objects.

Collections are used to perform the following operations:

- Searching
- Sorting
- Manipulation
- Insertion
- Deletion

A group of objects is known as collections. All the classes and interfaces for collecting are available in Java util package.

Q #25) What are all the Classes and Interfaces that are available in the collections?

Answer: Given below are the Classes and Interfaces that are available in Collections:

Interfaces:

- Collection
- List
- Set
- Map
- Sorted Set
- Sorted Map
- Queue

Classes:

- Lists:
- Array List
- Vector
- Linked List

Sets:

- Hash set
- Linked Hash Set
- Tree Set

Maps:

- Hash Map
- Hash Table
- TreeMap
- Linked Hashed Map

Queue:

- Priority Queue

Q #26) What is meant by Ordered and Sorted in collections?

Answer:

Ordered: It means the values that are stored in a collection is based on the values that are added to the collection. So we can iterate the values from the collection in a specific order.

Sorted: Sorting mechanisms can be applied internally or externally so that the group of objects sorted in a particular collection is based on the properties of the objects.

Q #27) Explain the different lists available in the collection.

Answer: Values added to the list are based on the index position and it is ordered by index position. Duplicates are allowed.

The types of Lists are:

a) Array List:

- Fast iteration and fast Random Access.
- It is an ordered collection (by index) and not sorted.
- It implements the Random Access Interface.

Example:

```
public class Fruits{
public static void main (String [ ] args){
ArrayList <String> names=new ArrayList <String> ();
names.add ("apple");
names.add ("cherry");
names.add ("kiwi");
names.add ("banana");
names.add ("cherry");
System.out.println (names);
}
}
```

Output:

[Apple, cherry, kiwi, banana, cherry]

From the output, Array List maintains the insertion order and it accepts the duplicates. But it's not sorted.

b) Vector:

It is the same as Array List.

- Vector methods are synchronized.
- Thread safety.
- It also implements Random Access.
- Thread safety usually causes a performance hit.

Example:

```
public class Fruit {  
    public static void main (String [ ] args){  
        Vector <String> names = new Vector <String> ( );  
        names.add ("cherry");  
        names.add ("apple");  
        names.add ("banana");  
        names.add ("kiwi");  
        names.add ("apple");  
        System.out.println ("names");  
    }  
}
```

Output:

[cherry,apple,banana,kiwi,apple]

Vector also maintains the insertion order and accepts the duplicates.

c) Linked List:

- Elements are doubly linked to one another.
- Performance is slower than the Array list.
- Good choice for insertion and deletion.
- In Java 5.0 it supports common queue methods peek(), Pool (), Offer () etc.

Example:

```
public class Fruit {  
    public static void main (String [ ] args){  
        LinkedList <String> names = new linkedlist <String> ( ) ;  
        names.add("banana");  
        names.add("cherry");  
        names.add("apple");  
        names.add("kiwi");  
        names.add("banana");  
    }  
}
```

```
System.out.println (names);  
}  
}
```

Output:

[banana,cherry,apple,kiwi,banana]

Maintains the insertion order and accepts the duplicates.

Q #28) Explain about Set and their types in a collection.

Answer: Set cares about uniqueness. It doesn't allow duplications. Here "equals ()" method is used to determine whether two objects are identical or not.

a) Hash Set:

- Unordered and unsorted.
- Uses the hash code of the object to insert the values.
- Use this when the requirement is "no duplicates and don't care about the order".

Example:

```
public class Fruit {  
    public static void main (String[] args){  
        HashSet<String> names = new HashSet <String>( ) ;  
        names.add("banana");  
        names.add("cherry");  
        names.add("apple");  
        names.add("kiwi");  
        names.add("banana");  
        System.out.println (names);  
    }  
}
```

Output:

[banana, cherry, kiwi, apple]

It doesn't follow any insertion order. Duplicates are not allowed.

b) Linked Hash set:

- An ordered version of the hash set is known as Linked Hash Set.
- Maintains a doubly-Linked list of all the elements.
- Use this when an iteration order is required.

Example:

```
public class Fruit {  
    public static void main (String[] args){  
        LinkedHashSet<String> names = new LinkedHashSet <String>( ) ;  
        names.add("banana");  
    }  
}
```

```

names.add("cherry");
names.add("apple");
names.add("kiwi");
names.add("banana");
System.out.println (names);
}
}

```

Output:

[banana, cherry, apple, kiwi]

It maintains the insertion order in which they have been added to the Set. Duplicates are not allowed.

c) Tree Set:

- It is one of the two sorted collections.
- Uses the “Read-Black” tree structure and guarantees that the elements will be in ascending order.
- We can construct a tree set with the constructor by using a comparable (or) comparator.

Example:

```

public class Fruits{
public static void main (String[] args) {
Treeset<String> names= new TreeSet<String>( ) ;
names.add("cherry");
names.add("banana");
names.add("apple");
names.add("kiwi");
names.add("cherry");
System.out.println(names);
}
}

```

Output:

[apple, banana, cherry, kiwi]

TreeSet sorts the elements in ascending order. And duplicates are not allowed.

Q #29) Explain about Map and its types.

Answer: Map cares about the unique identifier. We can map a unique key to a specific value. It is a key/value pair. We can search a value, based on the key. Like the set, the map also uses the “equals ()” method to determine whether two keys are the same or different.

Map is of following types:

a) Hash Map:

- Unordered and unsorted map.
- Hashmap is a good choice when we don't care about the order.
- It allows one null key and multiple null values.

Example:

```
Public class Fruit{
Public static void main(String[ ] args){
HashMap<String,String> names =new HashMap<String,String>( );
names.put ("key1","cherry");
names.put ("key2","banana");
names.put ("key3","apple");
names.put ("key4","kiwi");
names.put ("key1","cherry");
System.out.println(names);
}
}
```

Output:

{key2 =banana, key1=cherry, key4 =kiwi, key3= apple}

Duplicate keys are not allowed in Map.

It doesn't maintain any insertion order and is unsorted.

b) Hash Table:

- Like the vector key, methods of the class are synchronized.
- Thread safety and therefore slows the performance.
- It doesn't allow anything that is null.

Example:

```
public class Fruit{
public static void main(String[ ] args){
Hashtable<String,String> names =new Hashtable<String,String>( );
names.put ("key1","cherry");
names.put ("key2","apple");
names.put ("key3","banana");
names.put ("key4","kiwi");
names.put ("key2","orange");
System.out.println(names);
}
}
```

Output:

{key2=apple, key1=cherry, key4=kiwi, key3=banana}

Duplicate keys are not allowed.

c) Linked Hash Map:

- Maintains insertion order.
- Slower than Hash map.
- I can expect a faster iteration.

Example:

```
public class Fruit{
public static void main(String[ ] args){
LinkedHashMap<String,String> names =new
LinkedHashMap<String,String>( );
names.put("key1","cherry");
names.put("key2","apple");
names.put("key3","banana");
names.put("key4","kiwi");
names.put("key2","orange");
System.out.println(names);
}
}
```

Output:

{key2=apple, key1=cherry, key4=kiwi, key3=banana}

Duplicate keys are not allowed.

d) TreeMap:

- Sorted Map.
- Like Tree set, we can construct a sort order with the constructor.

Example:

```
public class Fruit{
public static void main(String[ ] args){
TreeMap<String,String> names =new TreeMap<String,String>( );
names.put("key1","cherry");
names.put("key2","banana");
names.put("key3","apple");
names.put("key4","kiwi");
names.put("key2","orange");
System.out.println(names);
}
}
```

Output:

{key1=cherry, key2=banana, key3 =apple, key4=kiwi}

It is sorted in ascending order based on the key. Duplicate keys are not allowed.

Q #30) Explain the Priority Queue.

Answer: Queue Interface

Priority Queue: Linked list class has been enhanced to implement the queue interface. Queues can be handled with a linked list. The purpose of a queue is “Priority-in, Priority-out”.

Hence elements are ordered either naturally or according to the comparator. The elements ordering represents their relative priority.

Q #31) What is meant by Exception?

Answer: An Exception is a problem that can occur during the normal flow of execution. A method can throw an exception when something goes wrong at runtime. If that exception couldn't be handled, then the execution gets terminated before it completes the task.

If we handled the exception, then the normal flow gets continued. Exceptions are a subclass of java.lang.Exception.

Example for handling Exception:

```
try{
//Risky codes are surrounded by this block
}catch(Exception e){
//Exceptions are caught in catch block
}
```

Q #32) What are the types of Exceptions?

Answer: There are two types of Exceptions. They are explained below in detail.

a) Checked Exception:

These exceptions are checked by the compiler at the time of compilation. Classes that extend Throwable class except RuntimeException and Error are called checked Exception.

Checked Exceptions must either declare the exception using throws keyword (or) surrounded by appropriate try/catch.

For Example, ClassNotFoundException

b) Unchecked Exception:

These exceptions are not checked during the compile time by the compiler. The compiler doesn't force to handle these exceptions. **It includes:**

- Arithmetic Exception
- ArrayIndexOutOfBoundsException

Q #33) What are the different ways to handle exceptions?

Answer: Two different ways to handle exceptions are explained below:

a) Using try/catch:

The risky code is surrounded by try block. If an exception occurs, then it is caught by the catch block which is followed by the try block.

Example:

```
class Manipulation{
public static void main(String[] args){
add();
}
Public void add(){
try{
addition();
}catch(Exception e){
e.printStackTrace();
}
}
}
```

b) By declaring throws keyword:

At the end of the method, we can declare the exception using throws keyword.

Example:

```
class Manipulation{
public static void main(String[] args){
add();
}
public void add() throws Exception{
addition();
}
}
```

Q #34) What are the advantages of Exception handling?

Answer: The advantages are as follows:

- The normal flow of the execution won't be terminated if an exception gets handled
- We can identify the problem by using catch declaration

Q #35) What are the Exception handling keywords in Java?

Answer: Enlisted below are the two Exception Handling Keywords:

a) try:

When a risky code is surrounded by a try block. An exception occurring in the try block is caught by a catch block. Try can be followed either by catch (or) finally (or) both. But any one of the blocks is mandatory.

b) catch:

This is followed by a try block. Exceptions are caught here.

c) finally:

This is followed either by try block (or) catch block. This block gets executed regardless of an exception. So generally clean up codes are provided here.

Q #36) Explain about Exception Propagation.

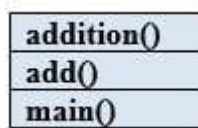
Answer: Exception is first thrown from the method which is at the top of the stack. If it doesn't catch, then it pops up the method and moves to the previous method and so on until they are got.

This is called Exception propagation.

Example:

```
public class Manipulation{
public static void main(String[] args){
add();
}
public void add(){
addition();
}
}
```

From the above example, the stack looks like as shown below:



addition()
add()
main()

If an exception occurs in the **addition()** method is not caught, then it moves to the method **add()**. Then it is moved to the **main()** method and then it will stop the flow of execution. It is called Exception Propagation.

Q #37) What is the final keyword in Java?

Answer:

Final variable: Once a variable is declared as final, then the value of the variable could not be changed. It is like a constant.

Example:

```
final int = 12;
```

Final method: A final keyword in a method, couldn't be overridden. If a method is marked as a final, then it can't be overridden by the subclass.

Final class: If a class is declared as final, then the class couldn't be subclassed. No class can extend the final class.

Q #38) What is a Thread?

Answer: In Java, the flow of execution is called Thread. Every java program has at least one thread called the main thread, the main thread is created by JVM. The user can define their own threads by extending the Thread class (or) by implementing the Runnable interface. Threads are executed concurrently.

Example:

```
public static void main(String[] args){//main thread starts here  
}
```

Q #39) How do you make a thread in Java?

Answer: There are two ways available to make a thread.

a) Extend Thread class: Extending a Thread class and override the run method. The thread is available in java.lang.thread.

Example:

```
Public class Addition extends Thread {  
public void run () {  
}  
}
```

The disadvantage of using a thread class is that we cannot extend any other classes because we have already extended the thread class. We can overload the run () method in our class.

b) Implement Runnable interface: Another way is by implementing the runnable interface. For that, we should provide the implementation for the run () method which is defined in the interface.

Example:

```
Public class Addition implements Runnable {  
public void run () {  
}  
}
```

Q #40) Explain about join () method.

Answer: Join () method is used to join one thread with the end of the currently running thread.

Example:

```
public static void main (String[] args){  
Thread t = new Thread ();  
t.start ();  
t.join ();  
}
```

Based on the above code, the main thread has started the execution. When it reaches the code **t.start()** then 'thread t' starts the own stack for the execution. JVM switches between the main thread and 'thread t'.

Once it reaches the code **t.join()** then 'thread t' alone is executed and completes its task, then only the main thread starts the execution.

It is a non-static method. The Join () method has an overloaded version. So we can mention the time duration in join () method also ".s".

Q #41) What does the yield method of the Thread class do?

Answer: A yield () method moves the currently running thread to a runnable state and allows the other threads for execution. So that equal priority threads have a chance to run. It is a static method. It doesn't release any lock.

Yield () method moves the thread back to the Runnable state only, and not the thread to sleep (), wait () (or) block.

Example:

```
public static void main (String[] args){
    Thread t = new Thread ();
    t.start ();
}
public void run() {
    Thread.yield();
}
}
```

Q #42) Explain about wait () method.

Answer: **wait ()** method is used to make the thread to wait in the waiting pool. When the wait () method is executed during a thread execution then immediately the thread gives up the lock on the object and goes to the waiting pool. Wait () method tells the thread to wait for a given amount of time.

Then the thread will wake up after notify () (or) notify all () method is called.

Wait() and the other above-mentioned methods do not give the lock on the object immediately until the currently executing thread completes the synchronized code. It is mostly used in synchronization.

Example:

```
public static void main (String[] args){
    Thread t = new Thread ();
    t.start ();
    Synchronized (t) {
        Wait();
    }
}
```

Q #43) Difference between notify() method and notifyAll() method in Java.

Answer: The differences between notify() method and notifyAll() method are enlisted below:

notify()	notifyAll()
This method is used to send a signal to wake up a single thread in the waiting pool.	This method sends the signal to wake up all the threads in a waiting pool.

Q #44) How to stop a thread in java? Explain about sleep () method in a thread?

Answer: We can stop a thread by using the following thread methods:

- Sleeping
- Waiting
- Blocked

Sleep: Sleep () method is used to sleep the currently executing thread for the given amount of time. Once the thread is wake up it can move to the runnable state. So sleep () method is used to delay the execution for some period.

It is a static method.

Example:

Thread. Sleep (2000)

So it delays the thread to sleep 2 milliseconds. Sleep () method throws an interrupted exception, hence we need to surround the block with try/catch.

```
public class ExampleThread implements Runnable{
    public static void main (String[] args){
        Thread t = new Thread ();
        t.start ();
    }
    public void run() {
        try{
            Thread.sleep(2000);
        }catch (InterruptedException e){
        }
    }
}
```

Q #45) When to use the Runnable interface Vs Thread class in Java?

Answer: If we need our class to extend some other classes other than the thread then we can go with the runnable interface because in java we can extend only one class.

If we are not going to extend any class then we can extend the thread class.

Q #46) Difference between start() and run() method of thread class.

Answer: Start() method creates a new thread and the code inside the run () method is executed in the new thread. If we directly called the run() method then a new thread is not created and the currently executing thread will continue to execute the run() method.

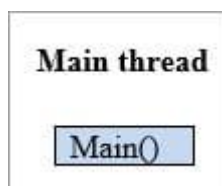
Q #47) What is Multi-threading?

Answer: Multiple threads are executed simultaneously. Each thread starts its own stack based on the flow (or) priority of the threads.

Example Program:

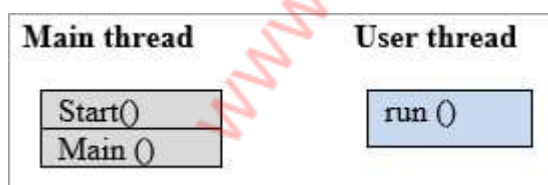
```
public class MultipleThreads implements Runnable
{
    public static void main (String[] args){//Main thread starts here
    Runnable r = new runnable ();
    Thread t=new thread ();
    t.start ();//User thread starts here
    Addition add=new addition ();
    }
    public void run(){
    go();
    }//User thread ends here
}
```

On the 1st line execution, JVM calls the main method and the main thread stack looks as shown below.



Once the execution reaches, **t.start ()** line then a new thread is created and the new stack for the thread is also created. Now JVM switches to the new thread and the main thread are back to the runnable state.

The two stacks look as shown below.



Now, the user thread executed the code inside the run() method.



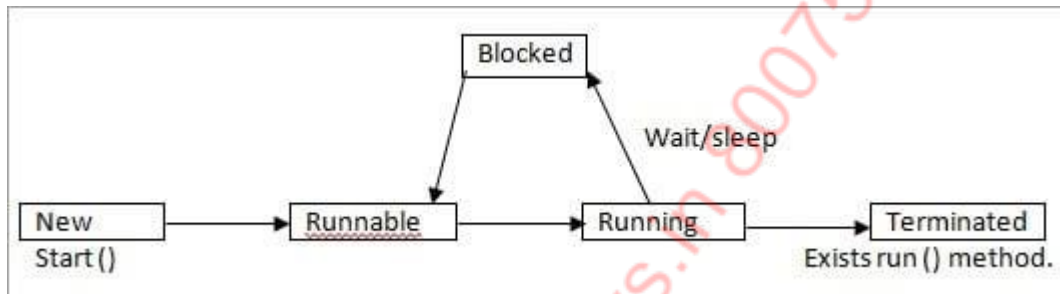
Once the run() method has completed, then JVM switches back to the main thread and the user thread has completed the task and the stack was disappeared.

JVM switches between each thread until both the threads are completed. This is called Multi-threading.

Q #48) Explain the thread life cycle in Java.

Answer: Thread has the following states:

- New
- Runnable
- Running
- Non-runnable (Blocked)
- Terminated



- **New:** In New state, a Thread instance has been created but start () method is not yet invoked. Now the thread is not considered alive.
- **Runnable:** The Thread is in the runnable state after the invocation of the start () method, but before the run () method is invoked. But a thread can also return to the runnable state from waiting/sleeping. In this state, the thread is considered alive.
- **Running:** The thread is in a running state after it calls the run () method. Now the thread begins the execution.
- **Non-Runnable(Blocked):** The thread is alive but it is not eligible to run. It is not in the runnable state but also, it will return to the runnable state after some time.
Example: wait, sleep, block.
- **Terminated:** Once the run method is completed then it is terminated. Now the thread is not alive.

Q #49) What is Synchronization?

Answer: Synchronization makes only one thread to access a block of code at a time. If multiple threads accesses the block of code, then there is a chance for inaccurate results at the end. To avoid this issue, we can provide synchronization for the sensitive block of codes.

The synchronized keyword means that a thread needs a key in order to access the synchronized code.

Locks are per objects. Every Java object has a lock. A lock has only one key. A thread can access a synchronized method only if the thread can get the key to the objects to lock.

For this, we use the “Synchronized” keyword.

Example:

```
public class ExampleThread implements Runnable{
    public static void main (String[] args){
        Thread t = new Thread ();
        t.start ();
    }
    public void run(){
        synchronized(object){
            {
            }
        }
    }
}
```

Q #50) What is the disadvantage of Synchronization?

Ans: Synchronization is not recommended to implement all the methods. Because if one thread accesses the synchronized code then the next thread should have to wait. So it makes a slow performance on the other end.

Q #51) What is meant by Serialization?

Answer: Converting a file into a byte stream is known as Serialization. The objects in the file are converted to bytes for security purposes. For this, we need to implement a java.io.Serializable interface. It has no method to define.

Variables that are marked as transient will not be a part of the serialization. So we can skip the serialization for the variables in the file by using a transient keyword.

Learn More =>> [Serializable and Cloneable](#)

Q #52) What is the purpose of a transient variable?

Answer: Transient variables are not part of the serialization process. During deserialization, the values of the transient variables are set to the default value. It is not used with static variables.

Example:

```
transient int numbers;
```

Q #53) Which methods are used during the Serialization and Deserialization process?

Answer: ObjectOutputStream and ObjectInputStream classes are higher level java.io. package. We will use them with lower level classes FileOutputStream and FileInputStream.

ObjectOutputStream.writeObject —>Serialize the object and write the serialized object to a file.

ObjectInputStream.readObject —> Reads the file and deserializes the object.

To be serialized, an object must implement the serializable interface. If superclass implements Serializable, then the subclass will automatically be serializable.

Q #54) What is the purpose of a Volatile Variable?

Answer: Volatile variable values are always read from the main memory and not from thread's cache memory. This is used mainly during synchronization. It is applicable only for variables.

Example:

volatile int number;

Q #55) Difference between Serialization and Deserialization in Java.

Answer: These are the differences between serialization and deserialization in java:

Serialization	Deserialization
Serialization is the process which is used to convert the objects into byte stream	Deserialization is the opposite process of serialization where we can get the objects back from the byte stream.
An object is serialized by writing it an ObjectOutputStream.	An object is deserialized by reading it from an ObjectInputStream.

Q #56) What is SerialVersionUID?

Answer: Whenever an object is Serialized, the object is stamped with a version ID number for the object class. This ID is called the SerialVersionUID. This is used during deserialization to verify that the sender and receiver that are compatible with the Serialization.

Conclusion

These are some of the core JAVA interview questions that cover both the basic and advanced Java concepts for programming as well as developer interview, and these are ones which have been answered by our JAVA experts.

I hope that this tutorial will give you a great insight into JAVA core coding concepts in detail. The explanations given above will really enrich your knowledge and increase your understanding of JAVA programming.

Get ready to crack a JAVA interview confidently.

→REFERENCE:-EDUREKA

Basic Java Interview Questions for Freshers

Q57. Explain JDK, JRE and JVM?

JDK vs JRE vs JVM		
JDK	JRE	JVM
It stands for Java Development Kit.	It stands for Java Runtime Environment.	It stands for Java Virtual Machine.
It is the tool necessary to compile, document and package Java programs.	JRE refers to a runtime environment in which Java bytecode can be executed.	It is an abstract machine. It is a specification that provides a runtime environment in which Java bytecode can be executed.
It contains JRE + development tools.	It's an implementation of the JVM which physically exists.	JVM follows three notations: Specification, Implementation , and Runtime Instance .

Q58. Explain public static void main(String args[]) in Java.

main() in Java is the entry point for any Java program. It is always written as **public static void main(String[] args)**.

- **public:** Public is an access modifier, which is used to specify who can access this method. Public means that this Method will be accessible by any Class.
- **static:** It is a keyword in java which identifies it is class-based. main() is made static in Java so that it can be accessed without creating the instance of a Class. In case, main is not made static then the compiler will throw an error as **main()** is called by the JVM before any objects are made and only static methods can be directly invoked via the class.
- **void:** It is the return type of the method. Void defines the method which will not return any value.
- **main:** It is the name of the method which is searched by JVM as a starting point for an application with a particular signature only. It is the method where the main execution occurs.
- **String args[]:** It is the parameter passed to the main method.

Q59. Why Java is platform independent?

Java is called platform independent because of its byte codes which can run on any system irrespective of its underlying operating system.

Q60. Why Java is not 100% Object-oriented?

Java is not 100% Object-oriented because it makes use of eight primitive data types such as boolean, byte, char, int, float, double, long, short which are not objects.

Q61. What are wrapper classes in Java?

Wrapper classes convert the Java primitives into the reference types (objects). Every primitive data type has a class dedicated to it. These are known as wrapper classes because they “wrap” the primitive data type into an object of that class. Refer to the below image which displays different primitive type, wrapper class and constructor argument.

Q62. What are constructors in Java?

In Java, constructor refers to a block of code which is used to initialize an object. It must have the same name as that of the class. Also, it has no return type and it is automatically called when an object is created.

There are two types of constructors:

1. **Default Constructor:** In Java, a default constructor is the one which does not take any inputs. In other words, default constructors are the no argument constructors which will be created by default in case you no other constructor is defined by the user. Its main purpose is to initialize the instance variables with the default values. Also, it is majorly used for object creation.
2. **Parameterized Constructor:** The parameterized constructor in Java, is the constructor which is capable of initializing the instance variables with the provided values. In other words, the constructors which take the arguments are called parameterized constructors.

Q63. What is singleton class in Java and how can we make a class singleton?

Singleton class is a class whose only one instance can be created at any given time, in one JVM. A class can be made singleton by making its constructor private.

Q64. What is the difference between Array list and vector in Java?

ArrayList	Vector
Array List is not synchronized.	Vector is synchronized.
Array List is fast as it's non-synchronized.	Vector is slow as it is thread safe.
If an element is inserted into the Array List, it increases its Array size by 50%.	Vector defaults to doubling size of its array.

Array List does not define the increment size.

Vector defines the increment size.

Array List can only use Iterator for traversing an Array List.

Vector can use both Enumeration and Iterator for traversing.

Q65. What is the difference between equals() and == in Java?

Equals() method is defined in Object class in Java and used for checking equality of two objects defined by business logic.

“==” or equality operator in Java is a binary operator provided by Java programming language and used to compare primitives and objects. *public boolean equals(Object o)* is the method provided by the Object class. The default implementation uses == operator to compare two objects. For example: method can be overridden like String class. equals() method is used to compare the values of two objects.

Q66. When can you use the super keyword?

In Java, the super keyword is a reference variable that refers to an immediate parent class object.

When you create a subclass instance, you’re also creating an instance of the parent class, which is referenced to by the super reference variable.

The uses of the Java super Keyword are-

1. To refer to an immediate parent class instance variable, use super.
2. The keyword super can be used to call the method of an immediate parent class.
3. Super() can be used to call the constructor of the immediate parent class.

Q67. What makes a HashSet different from a TreeSet?

HashSet	TreeSet
It is implemented through a hash table.	TreeSet implements SortedSet Interface that uses trees for storing data.
It permits the null object.	It does not allow the null object.
It is faster than TreeSet especially for search, insert, and delete operations.	It is slower than HashSet for these operations.

It does not maintain elements in an ordered way.	The elements are maintained in a sorted order.
It uses equals() method to compare two objects.	It uses compareTo() method for comparing two objects.
It does not permit a heterogenous object.	It permits a heterogenous object.

Q68. What are the differences between HashMap and Hashtable in Java?

HashMap	Hashtable
It is non synchronized. It cannot be shared between many threads without proper synchronization code.	It is synchronized. It is thread-safe and can be shared with many threads.
It permits one null key and multiple null values.	It does not permit any null key or value.
is a new class introduced in JDK 1.2.	It was present in earlier versions of java as well.
It is faster.	It is slower.
It is traversed through the iterator.	It is traversed through Enumerator and Iterator.
It uses fail fast iterator.	It uses an enumerator which is not fail fast.
It inherits AbstractMap class.	It inherits Dictionary class.

Q69. What is the importance of reflection in Java?

Reflection is a runtime API for inspecting and changing the behavior of methods, classes, and interfaces. Java Reflection is a powerful tool that can be really beneficial. Java Reflection allows you to analyze classes, interfaces, fields, and methods during runtime without knowing what they are called at compile time. Reflection can also be used to create new objects, call methods, and get/set field values. External, user-defined classes can be used by creating instances of extensibility objects with their fully-qualified names. Debuggers can also use reflection to examine private members of classes.

Q70. How to not allow serialization of attributes of a class in Java?

The NonSerialized attribute can be used to prevent member variables from being serialized. You should also make an object that potentially contains security-sensitive data nonserializable if possible. Apply the NonSerialized attribute to certain fields that store sensitive data if the object must be serialized. If you don't exclude these fields from

serialisation, the data they store will be visible to any programmes with serialization permission.

Q71. Can you call a constructor of a class inside another constructor?

Yes, we can call a constructor of a class inside another constructor. This is also called as constructor chaining. Constructor chaining can be done in 2 ways-

1. **Within the same class:** For constructors in the same class, the `this()` keyword can be used.
2. **From the base class:** The `super()` keyword is used to call the constructor from the base class. The constructor chaining follows the process of inheritance. The constructor of the sub class first calls the constructor of the super class. Due to this, the creation of sub class's object starts with the initialization of the data members of the super class. The constructor chaining works similarly with any number of classes. Every constructor keeps calling the chain till the top of the chain.

Q72. Contiguous memory locations are usually used for storing actual values in an array but not in ArrayList. Explain.

An array generally contains elements of the primitive data types such as int, float, etc. In such cases, the array directly stores these elements at contiguous memory locations. While an ArrayList does not contain primitive data types. An ArrayList contains the reference of the objects at different memory locations instead of the object itself. That is why the objects are not stored at contiguous memory locations.

Q73. How is the creation of a String using `new()` different from that of a literal?

When we create a string using `new()`, a new object is created. Whereas, if we create a string using the string literal syntax, it may return an already existing object with the same name.

Q74. Why is synchronization necessary? Explain with the help of a relevant example.

Java allows multiple threads to execute. They may be accessing the same variable or object. Synchronization helps to execute threads one after another.

It is important as it helps to execute all concurrent threads while being in sync. It prevents memory consistency errors due to access to shared memory. An example of synchronization code is-

Programming & Frameworks Training

```
1 public synchronized void increment ()
2 {
3     a++;
```

As we have synchronized this function, this thread can only use the object after the previous thread has used it.

Q75. Explain the term “Double Brace Initialization” in Java?

Double Brace Initialization is a Java term that refers to the combination of two independent processes. There are two braces used in this. The first brace creates an anonymous inner class. The second brace is an initialization block. When these both are used together, it is known as Double Brace Initialization. The inner class has a reference to the enclosing outer class, generally using the ‘this’ pointer. It is used to do both creation and initialization in a single statement. It is generally used to initialize collections. It reduces the code and also makes it more readable.

Q76. Why is it said that the length() method of String class doesn’t return accurate results?

The length() method of String class doesn’t return accurate results because it simply takes into account the number of characters within in the String. In other words, code points outside of the BMP (Basic Multilingual Plane), that is, code points having a value of U+10000 or above, will be ignored.

The reason for this is historical. One of Java’s original goals was to consider all text as Unicode; yet, Unicode did not define code points outside of the BMP at the time. It was too late to modify char by the time Unicode specified such code points.

Q77. What are the differences between Heap and Stack Memory in Java?

The major difference between Heap and Stack memory are:

Features	Stack	Heap
Memory	Stack memory is used only by one thread of execution.	Heap memory is used by all the parts of the application.
Access	Stack memory can’t be accessed by other threads.	Objects stored in the heap are globally accessible.
Memory Management	Follows LIFO manner to free memory.	Memory management is based on the generation associated with each object.

Lifetime	Exists until the end of execution of the thread.	Heap memory lives from the start till the end of application execution.
Usage	Stack memory only contains local primitive and reference variables to objects in heap space.	Whenever an object is created, it's always stored in the Heap space.

Q78. What is a package in Java? List down various advantages of packages.

Packages in Java, are the collection of related classes and interfaces which are bundled together. By using packages, developers can easily modularize the code and optimize its reuse. Also, the code within the packages can be imported by other classes and reused. Below I have listed down a few of its advantages:

- Packages help in avoiding name clashes
- They provide easier access control on the code
- Packages can also contain hidden classes which are not visible to the outer classes and only used within the package
- Creates a proper hierarchical structure which makes it easier to locate the related classes

Q79. Why pointers are not used in Java?

Java doesn't use pointers because they are unsafe and increases the complexity of the program. Since, Java is known for its simplicity of code, adding the concept of pointers will be contradicting. Moreover, since JVM is responsible for implicit memory allocation, thus in order to avoid direct access to memory by the user, pointers are discouraged in Java.

Q80. What is JIT compiler in Java?

JIT stands for Just-In-Time compiler in Java. It is a program that helps in converting the Java bytecode into instructions that are sent directly to the processor. By default, the JIT compiler is enabled in Java and is activated whenever a Java method is invoked. The JIT compiler then compiles the bytecode of the invoked method into native machine code, compiling it "just in time" to execute. Once the method has been compiled, the JVM summons the compiled code of that method directly rather than interpreting it. This is why it is often responsible for the performance optimization of Java applications at the run time.

Q81. What are access modifiers in Java?

In Java, access modifiers are special keywords which are used to restrict the access of a class, constructor, data member and method in another class. Java supports four types of access modifiers:

1. *Default*
2. *Private*
3. *Protected*
4. *Public*

Modifier	Default	Private	Protected	Public
<i>Same class</i>	YES	YES	YES	YES
<i>Same Package subclass</i>	YES	NO	YES	YES
<i>Same Package non-subclass</i>	YES	NO	YES	YES
<i>Different package subclass</i>	NO	NO	YES	YES
<i>Different package non-subclass</i>	NO	NO	NO	YES

Q82. Define a Java Class.

A class in Java is a blueprint which includes all your data. A class contains fields (variables) and methods to describe the behavior of an object. Let's have a look at the syntax of a class.

```
1class Abc {
2member variables // class body
3methods}
```

Q83. What is an object in Java and how is it created?

An object is a real-world entity that has a state and behavior. An object has three characteristics:

1. State
2. Behavior
3. Identity

An object is created using the 'new' keyword. For example:

```
ClassName obj = new ClassName();
```

Q84. What is Object Oriented Programming?

Object-oriented programming or popularly known as OOPs is a programming model or approach where the programs are organized around objects rather than logic and functions. In other words, OOP mainly focuses on the objects that are required to be manipulated instead of logic. This approach is ideal for the programs large and complex codes and needs to be actively updated or maintained.

Q85. What are the main concepts of OOPs in Java?

Object-Oriented Programming or OOPs is a programming style that is associated with concepts like:

1. *Inheritance*: Inheritance is a process where one class acquires the properties of another.
2. *Encapsulation*: Encapsulation in Java is a mechanism of wrapping up the data and code together as a single unit.
3. *Abstraction*: Abstraction is the methodology of hiding the implementation details from the user and only providing the functionality to the users.
4. *Polymorphism*: Polymorphism is the ability of a variable, function or object to take multiple forms.

Q86. What is the difference between a local variable and an instance variable?

In Java, a **local variable** is typically used inside a method, constructor, or a **block** and has only local scope. Thus, this variable can be used only within the scope of a block. The best benefit of having a local variable is that other methods in the class won't be even aware of that variable.

Example

```
1 if(x > 100)
2 {
3     String test = "Edureka";
4 }
```

Whereas, an **instance variable** in Java, is a variable which is bounded to its object itself. These variables are declared within a **class**, but outside a method. Every object of that class will create its own copy of the variable while using it. Thus, any changes made to the variable won't reflect in any other instances of that class and will be bound to that particular instance only.

```
1 class Test{
2     public String EmpName;
3     public int empAge;
4 }
```

Q87. Differentiate between the constructors and methods in Java?

Methods

1. Used to represent the behavior of an object
2. Must have a return type
3. Needs to be invoked explicitly

Constructors

1. Used to initialize the state of an object
2. Do not have any return type
3. Is invoked implicitly

- | | |
|---|--|
| 4. No default method is provided by the compiler | 4. A default constructor is provided by the compiler if the class has none |
| 5. Method name may or may not be same as class name | 5. Constructor name must always be the same as the class name |

In case you are facing any challenges with these Core Java interview questions, please comment on your problems in the section below.

Q88. What is final keyword in Java?

final is a special keyword in Java that is used as a non-access modifier. A final variable can be used in different contexts such as:

- **final variable**

When the final keyword is used with a variable then its value can't be changed once assigned. In case the no value has been assigned to the final variable then using only the class constructor a value can be assigned to it.

- *final method*

When a method is declared final then it can't be overridden by the inheriting class.

- *final class*

When a class is declared as final in Java, it can't be extended by any subclass class but it can extend other class.

Q89. What is the difference between break and continue statements?

break

1. Can be used in switch and loop (for, while, do while) statements
2. It causes the switch or loop statements to terminate the moment it is executed
3. It terminates the innermost enclosing loop or switch immediately

continue

1. Can be only used with loop statements
2. It doesn't terminate the loop but causes the loop to jump to the next iteration
3. A continue within a loop nested with a switch will cause the next loop iteration to execute

Example break:

```
for (int i = 0; i < 5; i++)</div>
1 <div>
```

```
2<pre>{
3if (i == 3)
4{
5    break;
6}
7System.out.println(i);
8}
9
```

Example continue:

```
1for (int i = 0; i < 5; i++)
2{
3    if(i == 2)
4    {
5        continue;
6    }
7    System.out.println(i);
8}
```

Q90. What is an infinite loop in Java? Explain with an example.

An infinite loop is an instruction sequence in Java that loops endlessly when a functional exit isn't met. This type of loop can be the result of a programming error or may also be a deliberate action based on the application behavior. An infinite loop will terminate automatically once the application exits.

For example:

```
1public class InfiniteForLoopDemo
2{
3    public static void main(String[] arg) {
4        for(;;)
5            System.out.println("Welcome to Edureka!");
6    }
7}
```

Q91. What is the difference between this() and super() in Java?

In Java, super() and this(), both are special keywords that are used to call the constructor.

this()

1. this() represents the current instance of a class
2. Used to call the default constructor of the same class
3. Used to access methods of the current class
4. Used for pointing the current class instance
5. Must be the first line of a block

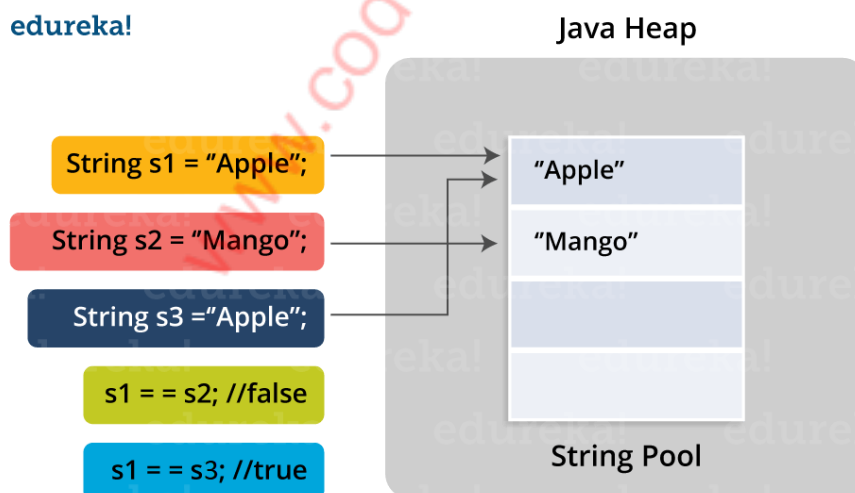
super()

1. super() represents the current instance of a parent/base class
2. Used to call the default constructor of the parent/base class
3. Used to access methods of the base class
4. Used for pointing the superclass instance
5. Must be the first line of a block

Q92. What is Java String Pool?

Java String pool refers to a collection of Strings which are stored in heap memory. In this, whenever a new object is created, String pool first checks whether the object is already present in the pool or not. If it is present, then the same reference is returned to the variable else new object will be created in the String pool and the respective reference will be returned.

edureka!



Q93. Differentiate between static and non-static methods in Java.

Static Method

1. The *static* keyword must be used before the method name
2. It is called using the class (className.methodName)
3. They can't access any non-static instance variables or methods

Non-Static Method

1. No need to use the *static* keyword before the method name
2. It can be called like any general method
3. It can access any static method and any static variable without creating an instance of the class

Q94. Explain the term “Double Brace Initialisation” in Java?

Double Brace Initialization is a Java term that refers to the combination of two independent processes. There are two braces used in this. The first brace creates an anonymous inner class. The second brace is an initialization block. When these both are used together, it is known as Double Brace Initialisation. The inner class has a reference to the enclosing outer class, generally using the ‘this’ pointer. It is used to do both creation and initialization in a single statement. It is generally used to initialize collections. It reduces the code and also makes it more readable.

Q95. What is constructor chaining in Java?

In Java, constructor chaining is the process of calling one constructor from another with respect to the current object. Constructor chaining is possible only through legacy where a subclass constructor is responsible for invoking the superclass' constructor first. There could be any number of classes in the constructor chain. Constructor chaining can be achieved in two ways:

1. Within the same class using this()
2. From base class using super()

Q96. Difference between String, StringBuilder, and StringBuffer.

Factor	String	StringBuilder	StringBuffer
Storage Area	Constant String Pool	Heap Area	Heap Area
Mutability	Immutable	Mutable	Mutable
Thread Safety	Yes	No	Yes
Performance	Fast	More efficient	Less efficient

If you think this article on Java Interview Questions is helpful, you can check out Edureka's [Java Training in Chennai](#) as well.

Q97. What is a classloader in Java?

The **Java ClassLoader** is a subset of JVM (Java Virtual Machine) that is responsible for loading the class files. Whenever a Java program is executed it is first loaded by the classloader. Java provides three built-in classloaders:

1. Bootstrap ClassLoader
2. Extension ClassLoader
3. System/Application ClassLoader

Q98. Why Java Strings are immutable in nature?

In Java, string objects are immutable in nature which simply means once the String object is created its state cannot be modified. Whenever you try to update the value of that object instead of updating the values of that particular object, Java creates a new string object. Java String objects are immutable as String objects are generally cached in the String pool. Since String literals are usually shared between multiple clients, action from one client might affect the rest. It enhances security, caching, synchronization, and performance of the application.

Q99. What is the difference between an array and an array list?

Array	ArrayList
Cannot contain values of different data types	Can contain values of different data types.
Size must be defined at the time of declaration	Size can be dynamically changed
Need to specify the index in order to add data	No need to specify the index
Arrays are not type parameterized	Arraylists are type
Arrays can contain primitive data types as well as objects	Arraylists can contain only objects, no primitive data types are allowed

Q100. What is a Map in Java?

In Java, Map is an interface of Util package which maps unique keys to values. The Map interface is not a subset of the main Collection interface and thus it behaves little different from the other collection types. Below are a few of the characteristics of Map interface:

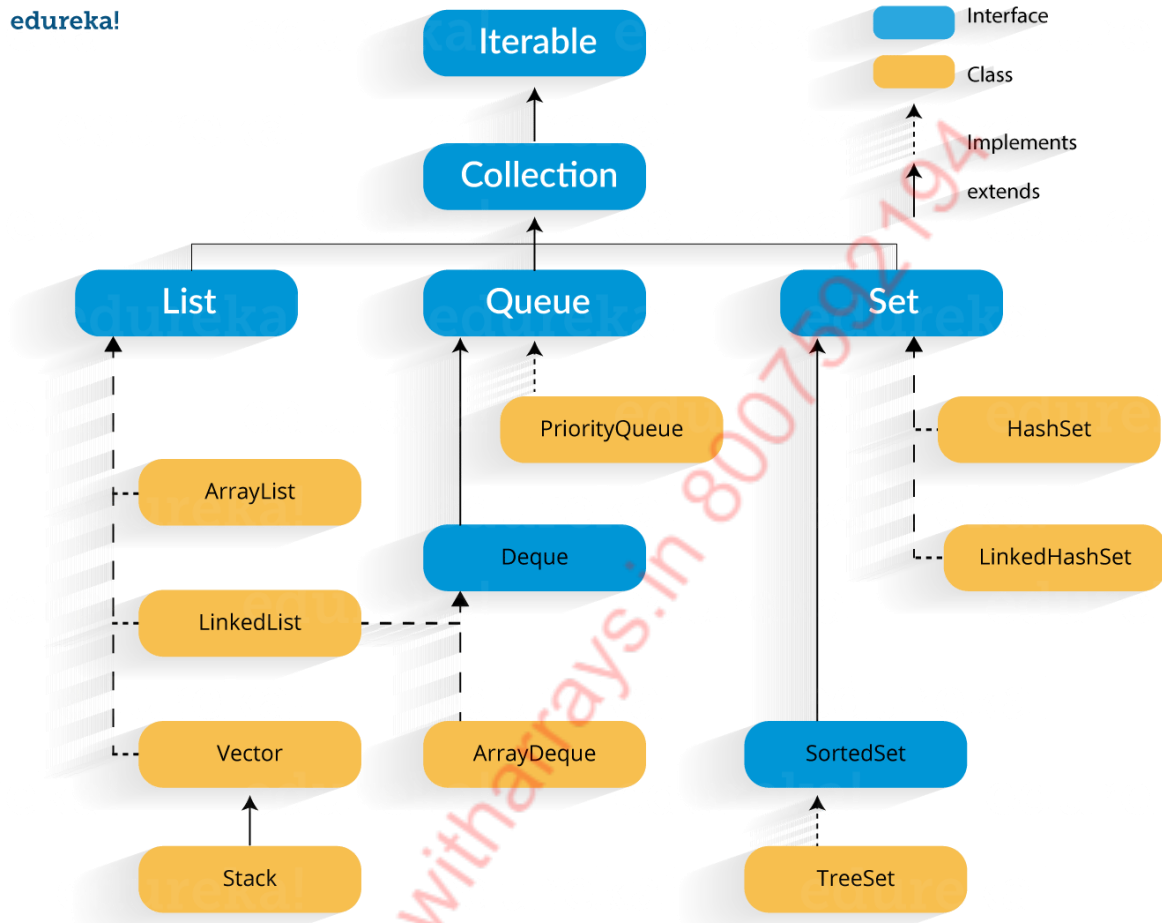
1. Map doesn't contain duplicate keys.
2. Each key can map at max one value.

Q101. What is collection class in Java? List down its methods and interfaces.

In Java, the collection is a framework that acts as an architecture for storing and manipulating a group of objects. Using Collections you can perform various tasks like searching, sorting, insertion, manipulation, deletion, etc. Java collection framework includes the following:

- Interfaces
- Classes
- Methods

The below image shows the complete hierarchy of the Java Collection.

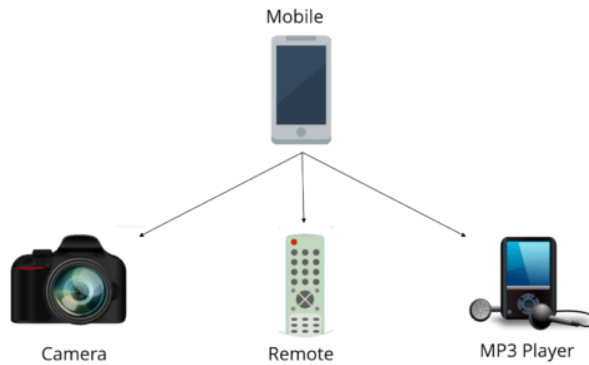


OOPS Java Interview Questions

Q102. What is Polymorphism?

Polymorphism is briefly described as “one interface, many implementations”. Polymorphism is a characteristic of being able to assign a different meaning or usage to something in different contexts – specifically, to allow an entity such as a variable, a function, or an object to have

more than one form. There are two types of polymorphism:



1. Compile time polymorphism
2. Run time polymorphism

Compile time polymorphism is method overloading whereas Runtime time polymorphism is done using inheritance and interface.

Q103. What is runtime polymorphism or dynamic method dispatch?

In Java, runtime polymorphism or dynamic method dispatch is a process in which a call to an overridden method is resolved at runtime rather than at compile-time. In this process, an overridden method is called through the reference variable of a superclass. Let's take a look at the example below to understand it better.

```
1 class Car {
2     void run()
3     {
4         System.out.println("car is running");
5     }
6 }
7 class Audi extends Car {
8     void run()
9     {
10        System.out.println("Audi is running safely with 100km");
11    }
12    public static void main(String args[])
13    {
14        Car b= new Audi();    //upcasting
15        b.run();
16    }
17 }
```

16

17

Q104. What is abstraction in Java?

Abstraction refers to the quality of dealing with ideas rather than events. It basically deals with hiding the details and showing the essential things to the user. Thus you can say that abstraction in Java is the process of hiding the implementation details from the user and revealing only the functionality to them. Abstraction can be achieved in two ways:

1. **Abstract Classes** (0-100% of abstraction can be achieved)
2. **Interfaces** (100% of abstraction can be achieved)

Q105. What do you mean by an interface in Java?

An interface in Java is a blueprint of a class or you can say it is a collection of abstract methods and static constants. In an interface, each method is public and abstract but it does not contain any constructor. Thus, interface basically is a group of related methods with empty bodies. Example:

```
public interface Animal {  
    public void eat();  
    public void sleep();  
    public void run();  
}
```

Q106. What is the difference between abstract classes and interfaces?

Abstract Class	Interfaces
An abstract class can provide complete, default code and/or just the details that have to be overridden	An interface cannot provide any code at all, just the signature
In the case of an abstract class, a class may extend only one abstract class	A Class may implement several interfaces
An abstract class can have non-abstract methods	All methods of an Interface are abstract
An abstract class can have instance variables	An Interface cannot have instance variables
An abstract class can have any visibility: public, private, protected	An Interface visibility must be public (or) none

If we add a new method to an abstract class then we have the option of providing default implementation and therefore all the existing code might work properly	If we add a new method to an Interface then we have to track down all the implementations of the interface and define implementation for the new method
An abstract class can contain constructors	An Interface cannot contain constructors
Abstract classes are fast	Interfaces are slow as it requires extra indirection to find the corresponding method in the actual class

Q107. What is inheritance in Java?

Inheritance in Java is the concept where the properties of one class can be inherited by the other. It helps to reuse the code and establish a relationship between different classes. Inheritance is performed between two types of classes:

1. Parent class (Super or Base class)
2. Child class (Subclass or Derived class)

A class which inherits the properties is known as Child Class whereas a class whose properties are inherited is known as Parent class.

Q108. What are the different types of inheritance in Java?

Java supports four types of inheritance which are:

1. **Single Inheritance:** In single inheritance, one class inherits the properties of another i.e there will be only one parent as well as one child class.
2. **Multilevel Inheritance:** When a class is derived from a class which is also derived from another class, i.e. a class having more than one parent class but at different levels, such type of inheritance is called Multilevel Inheritance.
3. **Hierarchical Inheritance:** When a class has more than one child classes (subclasses) or in other words, more than one child classes have the same parent class, then such kind of inheritance is known as hierarchical.
4. **Hybrid Inheritance:** Hybrid inheritance is a combination of two or more types of inheritance.

Q109. What is method overloading and method overriding?

Method Overloading :

- In Method Overloading, Methods of the same class shares the same name but each method must have a different number of parameters or parameters having different types and order.
- Method Overloading is to “add” or “extend” more to the method’s behavior.
- It is a compile-time polymorphism.
- The methods must have a different signature.
- It may or may not need inheritance in Method Overloading.

Let's take a look at the example below to understand it better.

```
1
2 class Adder {
3     Static int add(int a, int b)
4     {
5         return a+b;
6     }
7     Static double add( double a, double b)
8     {
9         return a+b;
10    }
11    public static void main(String args[])
12    {
13        System.out.println(Adder.add(11,11));
14        System.out.println(Adder.add(12.3,12.6));
15    }}
```

Method Overriding:

- In Method Overriding, the subclass has the same method with the same name and exactly the same number and type of parameters and same return type as a superclass.
- Method Overriding is to "Change" existing behavior of the method.
- It is a run time polymorphism.
- The methods must have the same signature.
- It always requires inheritance in Method Overriding.

Let's take a look at the example below to understand it better.

```
1 class Car {
2     void run() {
3         System.out.println("car is running");
4     }
5     Class Audi extends Car{
6         void run()
7         {
8             System.out.println("Audi is running safely with 100km");
9         }
10    public static void main( String args[])
```

```

10{
11Car b=new Audi();
12b.run();
13}
14}
15

```

Q110. Can you override a private or static method in Java?

You cannot override a private or static method in Java. If you create a similar method with the same return type and same method arguments in child class then it will hide the superclass method; this is known as method hiding. Similarly, you cannot override a private method in subclass because it's not accessible there. What you can do is create another private method with the same name in the child class. Let's take a look at the example below to understand it better.

```

class Base {
1 private static void display() {
2 System.out.println("Static or class method from Base");
3 }
4 public void print() {
5 System.out.println("Non-static or instance method from Base");
6 }
7 class Derived extends Base {
8 private static void display() {
9 System.out.println("Static or class method from Derived");
10 }
11 public void print() {
12 System.out.println("Non-static or instance method from Derived");
13 }
14 }
15 public class test {
16 public static void main(String args[])
17 {
18 Base obj= new Derived();
19 obj1.display();
20 obj1.print();
21 }

```

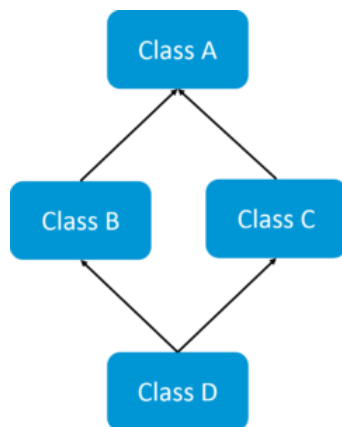
19 }

20

21

22

Q111. What is multiple inheritance? Is it supported by Java?



If a child class inherits the property from multiple classes is known as multiple inheritance. Java does not allow to extend multiple classes.

The problem with multiple inheritance is that if multiple parent classes have the same method name, then at runtime it becomes difficult for the compiler to decide which method to execute from the child class.

Therefore, Java doesn't support multiple inheritance. The problem is commonly referred to as Diamond Problem.

In case you are facing any challenges with these java interview questions, please comment on your problems in the section below.

Q112. What is encapsulation in Java?

Encapsulation is a mechanism where you bind your data(variables) and code(methods) together as a single unit. Here, the data is hidden from the outer world and can be accessed only via current class methods. This helps in protecting the data from any unnecessary modification. We can achieve encapsulation in Java by:

- Declaring the variables of a class as private.
- Providing public setter and getter methods to modify and view the values of the variables.

Q113. What is an association?

Association is a relationship where all object have their own lifecycle and there is no owner. Let's take the example of Teacher and Student. Multiple students can associate with a single teacher and a single student can associate with multiple teachers but there is no ownership between the objects and both have their own lifecycle. These relationships can be one to one, one to many, many to one and many to many.

Q114. What do you mean by aggregation?

An aggregation is a specialized form of Association where all object has their own lifecycle but there is ownership and child object can not belong to another parent object. Let's take an example of Department and teacher. A single teacher can not belong to multiple departments, but if we delete the department teacher object will not destroy.

Q115. What is composition in Java?

Composition is again a specialized form of Aggregation and we can call this as a "death" relationship. It is a strong type of Aggregation. Child object does not have their lifecycle and if parent object deletes all child object will also be deleted. Let's take again an example of a relationship between House and rooms. House can contain multiple rooms there is no independent life of room and any room can not belongs to two different houses if we delete the house room will automatically delete.

Q116. What is a marker interface?

A Marker interface can be defined as the interface having no data member and member functions. In simpler terms, an empty interface is called the Marker interface. The most common examples of Marker interface in Java are Serializable, Cloneable etc. The marker interface can be declared as follows.

```
1public interface Serializable{  
2}
```

Q117. What is object cloning in Java?

Object cloning in Java is the process of creating an exact copy of an object. It basically means the ability to create an object with a similar state as the original object. To achieve this, Java provides a method **clone()** to make use of this functionality. This method creates a new instance of the class of the current object and then initializes all its fields with the exact same contents of corresponding fields. To object clone(), the marker interface **java.lang.Cloneable** must be implemented to avoid any runtime exceptions. One thing you must note is Object clone() is a protected method, thus you need to override it.

Q118. What is a copy constructor in Java?

Copy constructor is a member function that is used to initialize an object using another object of the same class. Though there is no need for copy constructor in Java since all objects are passed by reference. Moreover, Java does not even support automatic pass-by-value.

Q119. What is a constructor overloading in Java?

In Java, constructor overloading is a technique of adding any number of constructors to a class each having a different parameter list. The compiler uses the number of parameters and their types in the list to differentiate the overloaded constructors.

```

1
2  class Demo
3  {
4      int i;
5      public Demo (int a)
6      {
7          i=k;
8      }
9      public Demo (int a, int b)
10     {
11         //body
12     }
13 }

```

In case you are facing any challenges with these java interview questions, please comment on your problems in the section below. Apart from this Java Interview Questions Blog, if you want to get trained from professionals on this technology, you can opt for a structured training from edureka!

→JAVA INTERVIEW QUESTIONS:-

→REFERENCE:- [INTERVIEW_BIT](#)

Java Basic Interview Questions

1. Why is Java a platform independent language?

Java language was developed in such a way that it does not depend on any hardware or software due to the fact that the compiler compiles the code and then converts it to platform-independent byte code which can be run on multiple systems.

- The only condition to run that byte code is for the machine to have a runtime environment (JRE) installed in it

2. Why is Java not a pure object oriented language?

Java supports primitive data types - byte, boolean, char, short, int, float, long, and double and hence it is not a pure object oriented language.

3. Difference between Heap and Stack Memory in java. And how java utilizes this.

Stack memory is the portion of memory that was assigned to every individual program. And it was fixed. On the other hand, Heap memory is the portion that was not allocated to the java program but it will be available for use by the java program when it is required, mostly during the runtime of the program.

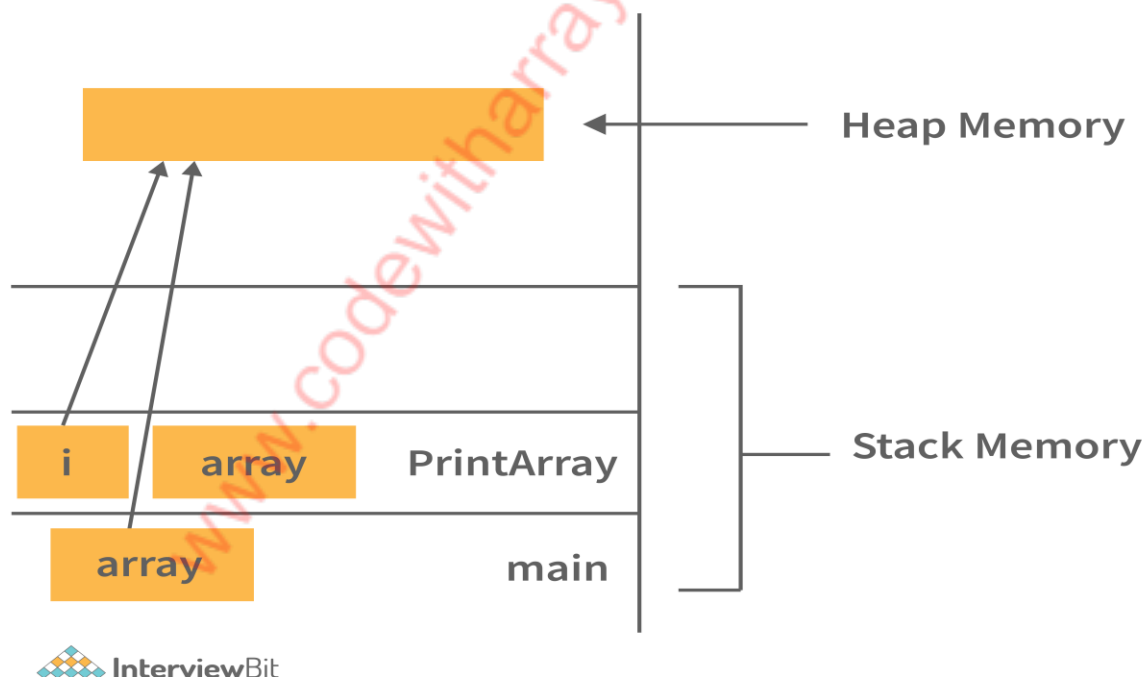
Java Utilizes this memory as -

- When we write a java program then all the variables, methods, etc are stored in the stack memory.
- And when we create any object in the java program then that object was created in the heap memory. And it was referenced from the stack memory.

Example- Consider the below java program:

```
class Main {  
    public void printArray(int[] array){  
        for(int i : array)  
            System.out.println(i);  
    }  
    public static void main(String args[]) {  
        int[] array = new int[10];  
        printArray(array);  
    }  
}
```

For this java program. The stack and heap memory occupied by java is -



Main and PrintArray is the method that will be available in the stack area and as well as the variables declared that will also be in the stack area.

And the Object (Integer Array of size 10) we have created, will be available in the Heap area because that space will be allocated to the program during runtime.

4. Can java be said to be the complete object-oriented programming language?

It is not wrong if we claim that java is the complete object-oriented programming language. Because Everything in Java is under the classes. And we can access that by creating the objects.

But also if we say that java is not a completely object-oriented programming language because it has the support of primitive data types like int, float, char, boolean, double, etc.

Now for the question: **Is java a completely object-oriented programming language?** We can say that - Java is not a pure object-oriented programming language, because it has direct access to primitive data types. And these primitive data types don't directly belong to the Integer classes.

5. How is Java different from C++?

- C++ is only a compiled language, whereas Java is compiled as well as an interpreted language.
- Java programs are machine-independent whereas a c++ program can run only in the machine in which it is compiled.
- C++ allows users to use pointers in the program. Whereas java doesn't allow it. Java internally uses pointers.
- C++ supports the concept of Multiple inheritances whereas Java doesn't support this. And it is due to avoiding the complexity of name ambiguity that causes the diamond problem.

6. Pointers are used in C/ C++. Why does Java not make use of pointers?

Pointers are quite complicated and unsafe to use by beginner programmers. Java focuses on code simplicity, and the usage of pointers can make it challenging. Pointer utilization can also cause potential errors. Moreover, security is also compromised if pointers are used because the users can directly access memory with the help of pointers.

Thus, a certain level of abstraction is furnished by not including pointers in Java. Moreover, the usage of pointers can make the procedure of garbage collection quite slow and erroneous. Java makes use of references as these cannot be manipulated, unlike pointers.

7. What do you understand by an instance variable and a local variable?

Instance variables are those variables that are accessible by all the methods in the class. They are declared outside the methods and inside the class. These variables describe the properties of an object and remain bound to it at any cost.

All the objects of the class will have their copy of the variables for utilization. If any modification is done on these variables, then only that instance will be impacted by it, and all other class instances continue to remain unaffected.

Example:

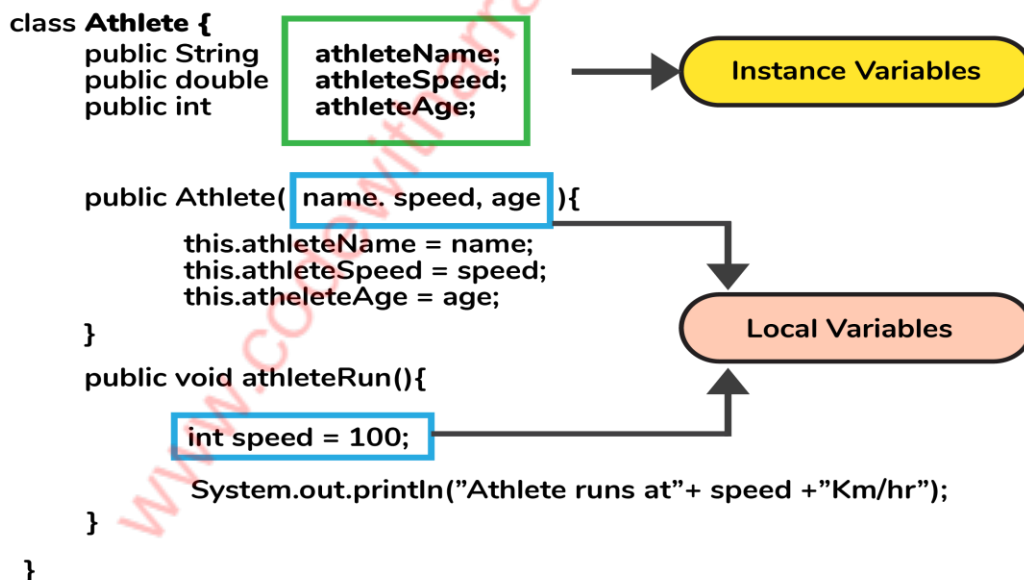
```
class Athlete {  
    public String athleteName;  
    public double athleteSpeed;  
    public int athleteAge;  
}
```

Local variables are those variables present within a block, function, or constructor and can be accessed only inside them. The utilization of the variable is restricted to the block scope. Whenever a local variable is declared inside a method, the other class methods don't have any knowledge about the local variable.

Example:

```
public void athlete() {  
    String athleteName;  
    double athleteSpeed;  
    int athleteAge;  
}
```

Instance vs Local Variables



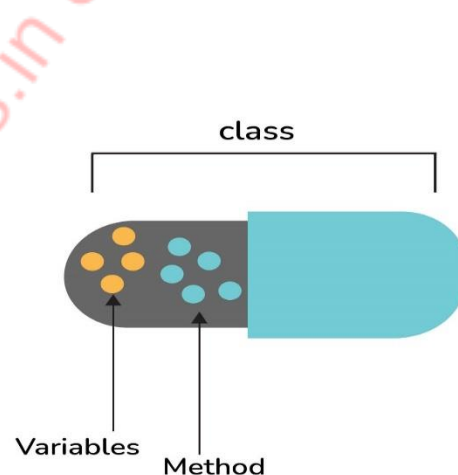
8. What are the default values assigned to variables and instances in java?

- There are no default values assigned to the variables in java. We need to initialize the value before using it. Otherwise, it will throw a compilation error of **(Variable might not be initialized)**.
- But for instance, if we create the object, then the default value will be initialized by the default constructor depending on the data type.
- If it is a reference, then it will be assigned to null.
- If it is numeric, then it will assign to 0.
- If it is a boolean, then it will be assigned to false. Etc.

9. What do you mean by data encapsulation?

- Data Encapsulation is an Object-Oriented Programming concept of hiding the data attributes and their behaviours in a single unit.
- It helps developers to follow modularity while developing software by ensuring that each object is independent of other objects by having its own methods, attributes, and functionalities.
- It is used for the security of the private properties of an object and hence serves the purpose of data hiding.

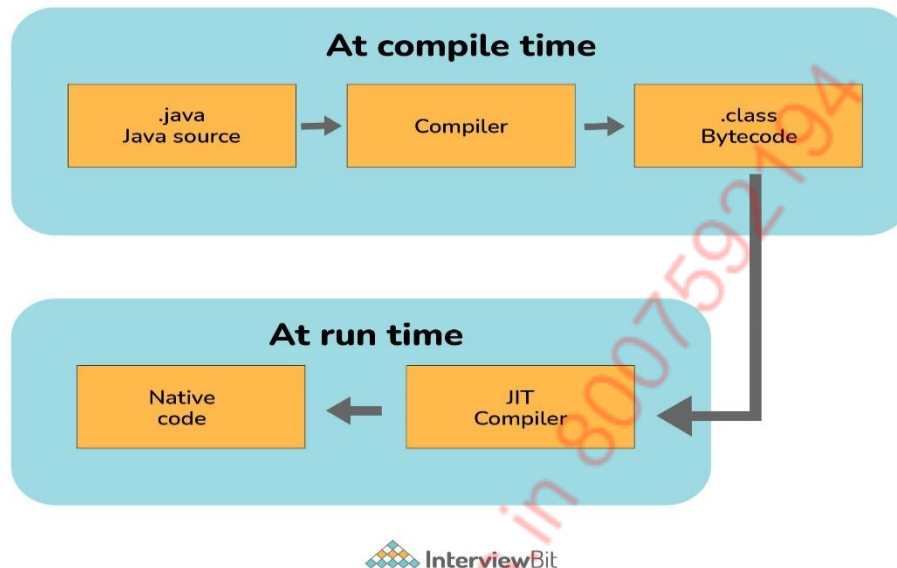
```
class
{
    data members (properties)
    +
    methods (behavior)
}
```



10. Tell us something about JIT compiler.

- JIT stands for Just-In-Time and it is used for improving the performance during run time. It does the task of compiling parts of byte code having similar functionality at the same time thereby reducing the amount of compilation time for the code to run.
- The compiler is nothing but a translator of source code to machine-executable code. But what is special about the JIT compiler? Let us see how it works:
 - First, the Java source code (.java) conversion to byte code (.class) occurs with the help of the javac compiler.
 - Then, the .class files are loaded at run time by JVM and with the help of an interpreter, these are converted to machine understandable code.

- JIT compiler is a part of JVM. When the JIT compiler is enabled, the JVM analyzes the method calls in the .class files and compiles them to get more efficient and native code. It also ensures that the prioritized method calls are optimized.
- Once the above step is done, the JVM executes the optimized code directly instead of interpreting the code again. This increases the performance and speed of the execution.



11. Can you tell the difference between equals() method and equality operator (==) in Java?

equals()

==

This is a method defined in the Object class. It is a binary operator in Java.

This method is used for checking the equality of contents between two objects as per the specified business logic.

This operator is used for comparing addresses (or references), i.e checks if both the objects are pointing to the same memory location.

Note:

- In the cases where the equals method is not overridden in a class, then the class uses the default implementation of the equals method that is closest to the parent class.
- Object class is considered as the parent class of all the java classes. The implementation of the equals method in the Object class uses the == operator to compare two objects. This default implementation can be overridden as per the business logic.

12. How is an infinite loop declared in Java?

Infinite loops are those loops that run infinitely without any breaking conditions. Some examples of consciously declaring infinite loop is:

- Using For Loop:

```
for (;;)
{
    // Business logic
    // Any break logic
}
```

- Using while loop:

```
while(true){
    // Business logic
    // Any break logic
}
```

- Using do-while loop:

```
do{
    // Business logic
    // Any break logic
}while(true);
```

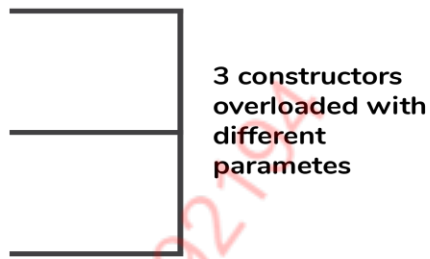
13. Briefly explain the concept of constructor overloading

Constructor overloading is the process of creating multiple constructors in the class consisting of the same name with a difference in the constructor parameters. Depending upon the number of parameters and their corresponding types, distinguishing of the different types of constructors is done by the compiler.

```
class Hospital {
int variable1, variable2;
double variable3;
public Hospital(int doctors, int nurses) {
    variable1 = doctors;
    variable2 = nurses;
}
public Hospital(int doctors) {
    variable1 = doctors;
}
public Hospital(double salaries) {
    variable3 = salaries
}
}
```

Constructor Overloading

```
class Hospital {  
    int variable1, variable2;  
    double variable3;  
  
    public Hospital(int doctors, int nurses) {  
        variable1 = doctors;  
        variable2 = nurses;  
    }  
    public Hospital(int doctors) {  
        variable1 = doctors;  
    }  
    public Hospital(double salaries) {  
        variable3 = salaries;  
    }  
}
```



3 constructors overloaded with different parameters

Three constructors are defined here but they differ on the basis of parameter type and their numbers.

14. Define Copy constructor in java.

Copy Constructor is the constructor used when we want to initialize the value to the new object from the old object of the same class.

```
class InterviewBit{  
    String department;  
    String service;  
    InterviewBit(InterviewBit ib){  
        this.department = ib.department;  
        this.service = ib.service;  
    }  
}
```

Here we are initializing the new object value from the old object value in the constructor. Although, this can also be achieved with the help of object cloning.

15. Can the main method be Overloaded?

Yes, It is possible to overload the main method. We can create as many overloaded main methods we want. However, JVM has a predefined calling method that JVM will only call the main method with the definition of -

```
public static void main(String[] args)
```

Consider the below code snippets:

```
class Main {
```

```

public static void main(String args[]) {
    System.out.println(" Main Method");
}
public static void main(int[] args){
    System.out.println("Overloaded Integer array Main Method");
}
public static void main(char[] args){
    System.out.println("Overloaded Character array Main Method");
}
public static int main(double[] args){
    System.out.println("Overloaded Double array Main Method");
}
public static void main(float args){
    System.out.println("Overloaded float Main Method");
}
}

```

16. Comment on method overloading and overriding by citing relevant examples.

In Java, **method overloading** is made possible by introducing different methods in the same class consisting of the same name. Still, all the functions differ in the number or type of parameters. It takes place inside a class and enhances program readability.

The only difference in the return type of the method does not promote method overloading. The following example will furnish you with a clear picture of it.

```

class OverloadingHelp {
    public int findarea (int l, int b) {
        int var1;
        var1 = l * b;
        return var1;
    }
    public int findarea (int l, int b, int h) {
        int var2;
        var2 = l * b * h;
        return var2;
    }
}

```

Method Overloading

```
class OverloadingHelp {
```

```
    public int findarea (int l, int b) {
```

```
        int var1;  
        var1 = l * b;  
        return var1
```

```
    }
```

```
    public int findarea (int l, int b, int h) {
```

```
        int var2;  
        var2 = l * b * h;  
        return var2;
```

```
    }
```

```
}
```

Same method
name but different
parameters

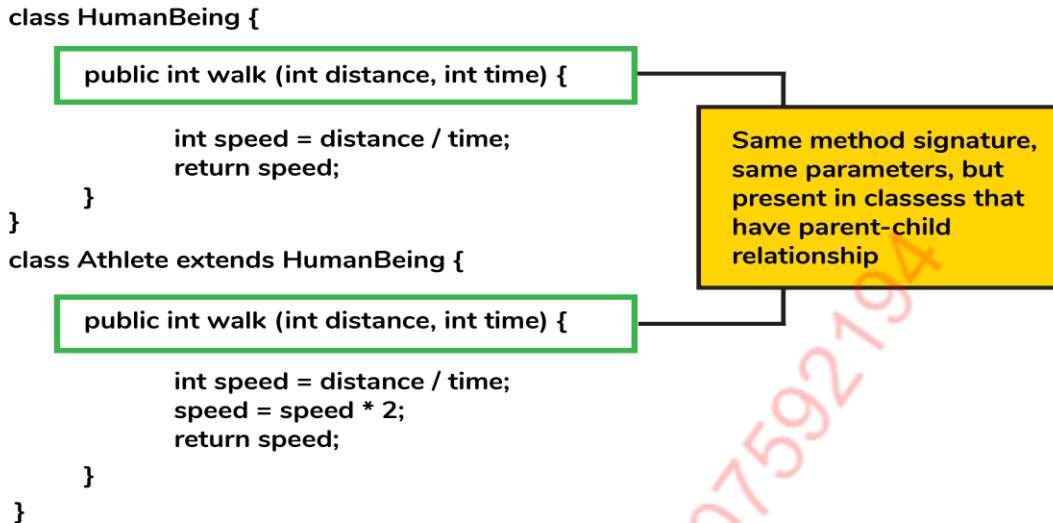
Both the functions have the same name but differ in the number of arguments. The first method calculates the area of the rectangle, whereas the second method calculates the area of a cuboid.

Method overriding is the concept in which two methods having the same method signature are present in two different classes in which an inheritance relationship is present. A particular method implementation (already present in the base class) is possible for the derived class by using method overriding.

Let's give a look at this example:

```
class HumanBeing {  
    public int walk (int distance, int time) {  
        int speed = distance / time;  
        return speed;  
    }  
}  
class Athlete extends HumanBeing {  
    public int walk(int distance, int time) {  
        int speed = distance / time;  
        speed = speed * 2;  
        return speed;  
    }  
}
```

Method Overriding



Both class methods have the name walk and the same parameters, distance, and time. If the derived class method is called, then the base class method walk gets overridden by that of the derived class.

17. A single try block and multiple catch blocks can co-exist in a Java Program. Explain.

Yes, multiple catch blocks can exist but specific approaches should come prior to the general approach because only the first catch block satisfying the catch condition is executed. The given code illustrates the same:

```
public class MultipleCatch {  
    public static void main(String args[]) {  
        try {  
            int n = 1000, x = 0;  
            int arr[] = new int[n];  
            for (int i = 0; i <= n; i++) {  
                arr[i] = i / x;  
            }  
        }  
        catch (ArrayIndexOutOfBoundsException exception) {  
            System.out.println("1st block = ArrayIndexOutOfBoundsException");  
        }  
        catch (ArithmeticException exception) {  
            System.out.println("2nd block = ArithmeticException");  
        }  
        catch (Exception exception) {  
            System.out.println("3rd block = Exception");  
        }  
    }  
}
```

Here, the second catch block will be executed because of division by 0 (i / x). In case x was greater than 0 then the first catch block will execute because for loop runs till $i = n$ and array index are till $n-1$.

18. Explain the use of final keyword in variable, method and class.

In Java, the final keyword is used as defining something as constant /final and represents the non-access modifier.

- **final variable:**
 - When a variable is declared as final in Java, the value can't be modified once it has been assigned.
 - If any value has not been assigned to that variable, then it can be assigned only by the constructor of the class.
- **final method:**
 - A method declared as final cannot be overridden by its children's classes.
 - A constructor cannot be marked as final because whenever a class is inherited, the constructors are not inherited. Hence, marking it final doesn't make sense. Java throws compilation error saying -modifier final not allowed here
- **final class:**
 - No classes can be inherited from the class declared as final. But that final class can extend other classes for its usage.

19. Do final, finally and finalize keywords have the same function?

All three keywords have their own utility while programming.

Final: If any restriction is required for classes, variables, or methods, the final keyword comes in handy. Inheritance of a final class and overriding of a final method is restricted by the use of the final keyword. The variable value becomes fixed after incorporating the final keyword. Example:

```
final int a=100;
a = 0; // error
```

The second statement will throw an error.

Finally: It is the block present in a program where all the codes written inside it get executed irrespective of handling of exceptions. Example:

```
try {
    int variable = 5;
}
catch (Exception exception) {
    System.out.println("Exception occurred");
}
finally {
    System.out.println("Execution of finally block");
}
```

Finalize: Prior to the garbage collection of an object, the finalize method is called so that the clean-up activity is implemented. Example:

```

public static void main(String[] args) {
    String example = new String("InterviewBit");
    example = null;
    System.gc(); // Garbage collector called
}
public void finalize() {
    // Finalize called
}

```

20. Is it possible that the ‘finally’ block will not be executed? If yes then list the case.

Yes. It is possible that the ‘finally’ block will not be executed. The cases are-

- Suppose we use **System.exit()** in the above statement.
- If there are fatal errors like Stack overflow, Memory access error, etc.

21. Identify the output of the java program and state the reason.

```

1. public class InterviewBit
2. {
3.     public static void main(String[] args) {
4.         final int i;
5.         i = 20;
6.         int j = i+20;
7.         i = j+30;
8.         System.out.println(i + " " + j);
9.     }
10. }

```

The above code will generate a compile-time error at Line 7 saying - **[error: variable i might already have been initialized]**. It is because variable ‘i’ is the final variable. And final variables are allowed to be initialized only once, and that was already done on line no 5.

22. When can you use super keyword?

- The super keyword is used to access hidden fields and overridden methods or attributes of the parent class.
- Following are the cases when this keyword can be used:
 - Accessing data members of parent class when the member names of the class and its child subclasses are same.
 - To call the default and parameterized constructor of the parent class inside the child class.
 - Accessing the parent class methods when the child classes have overridden them.
- The following example demonstrates all 3 cases when a super keyword is used.

```

public class Parent{
    protected int num = 1;

    Parent(){
        System.out.println("Parent class default constructor.");
    }

    Parent(String x){
        System.out.println("Parent class parameterised constructor.");
    }
}

```

```

    }

    public void foo(){
        System.out.println("Parent class foo!");
    }
}

public class Child extends Parent{
    private int num = 2;

    Child(){
        System.out.println("Child class default Constructor");
        super("Call Parent");    // to call parameterised constructor.
        super();    // to call default parent constructor
    }

    void printNum(){
        System.out.println(num);
        System.out.println(super.num); //prints the value of num of
parent class
    }

    @Override
    public void foo(){
        System.out.println("Parent class foo!");
        super.foo();    //Calls foo method of Parent class inside the
Overridden foo method of Child class.
    }
}

```

23. Can the static methods be overloaded?

Yes! There can be two or more static methods in a class with the same name but differing input parameters.

24. Why is the main method static in Java?

The main method is always static because static members are those methods that belong to the classes, not to an individual object. So if the main method will not be static then for every object, It is available. And that is not acceptable by JVM. JVM calls the main method based on the class name itself. Not by creating the object.

Because there must be only 1 main method in the java program as the execution starts from the main method. So for this reason the main method is static.

25. Can the static methods be overridden?

- No! Declaration of static methods having the same signature can be done in the subclass but run time polymorphism can not take place in such cases.
- Overriding or dynamic polymorphism occurs during the runtime, but the static methods are loaded and looked up at the compile time statically. Hence, these methods cant be overridden.

26. Difference between static methods, static variables, and static classes in java.

- **Static Methods and Static variables** are those methods and variables that belong to the class of the java program, not to the object of the class. This gets memory where the class is loaded. And these can directly be called with the help of class names.
 - For example - We have used mathematical functions in the java program like - max(), min(), sqrt(), pow(), etc. And if we notice that, then we will find that we call it directly with the class name. Like - Math.max(), Math.min(), etc. So that is a static method. And Similarly static variables we have used like (length) for the array to get the length. So that is the static method.
- **Static classes** - A class in the java program cannot be static except if it is the inner class. If it is an inner static class, then it exactly works like other static members of the class.

27. What is the main objective of garbage collection?

The main objective of this process is to free up the memory space occupied by the unnecessary and unreachable objects during the Java program execution by deleting those unreachable objects.

- This ensures that the memory resource is used efficiently, but it provides no guarantee that there would be sufficient memory for the program execution.

28. What is a ClassLoader?

- Java Classloader is the program that belongs to JRE (Java Runtime Environment). The task of ClassLoader is to load the required classes and interfaces to the JVM when required.
- **Example**- To get input from the console, we require the scanner class. And the Scanner class is loaded by the ClassLoader.

29. What part of memory - Stack or Heap - is cleaned in garbage collection process?

Heap.

30. What are shallow copy and deep copy in java?

To copy the object's data, we have several methods like deep copy and shallow copy.

Example -

```
class Rectangle{
    int length = 5;
    int breadth = 3;
}
```

Object for this Rectangle class - **Rectangle obj1 = new Rectangle();**

- **Shallow copy** - The shallow copy only creates a new reference and points to the same object. Example - For Shallow copy, we can do this by -

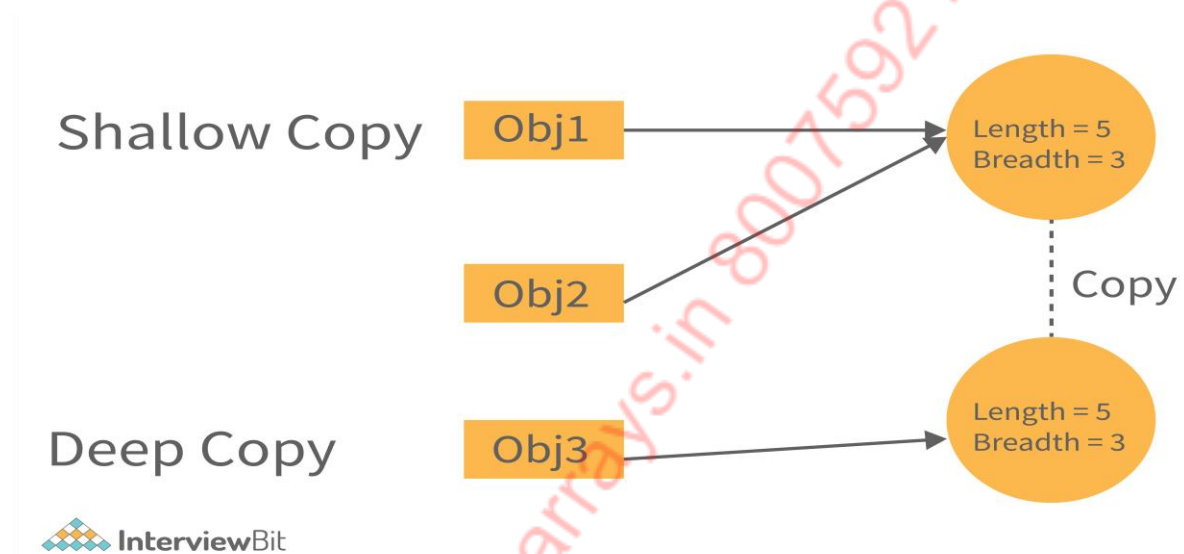
```
Rectangle obj2 = obj1;
```

Now by doing this what will happen is the new reference is created with the name obj2 and that will point to the same memory location.

- **Deep Copy** - In a deep copy, we create a new object and copy the old object value to the new object. Example -

```
Rectangle obj3 = new Rectangle();  
Obj3.length = obj1.length;  
Obj3.breadth = obj1.breadth;
```

Both these objects will point to the memory location as stated below -



Now, if we change the values in shallow copy then they affect the other reference as well. Let's see with the help of an example -

```
class Rectangle  
{  
    int length = 5;  
    int breadth = 3;  
}  
public class Main  
{  
    public static void main(String[] args) {  
        Rectangle obj1 = new Rectangle();  
        //Shallow Copy  
        Rectangle obj2 = obj1;  
  
        System.out.println(" Before Changing the value of object 1, the  
object2 will be - ");  
        System.out.println(" Object2 Length = "+obj2.length+", Object2  
Breadth = "+obj2.breadth);  
  
        //Changing the values for object1.  
        obj1.length = 10;  
        obj1.breadth = 20;
```

```

        System.out.println("\n After Changing the value of object 1, the
object2 will be - ");
        System.out.println(" Object2 Length = "+obj2.length+", Object2
Breadth = "+obj2.breadth);
    }
}

```

Output -

Before Changing the value of object 1, the object2 will be -
Object2 Length = 5, Object2 Breadth = 3

After Changing the value of object 1, the object2 will be -
Object2 Length = 10, Object2 Breadth = 20

We can see that in the above code, if we change the values of object1, then the object2 values also get changed. It is because of the reference.

Now, if we change the code to deep copy, then there will be no effect on object2 if it is of type deep copy. Consider some snippets to be added in the above code.

```

class Rectangle
{
    int length = 5;
    int breadth = 3;
}
public class Main
{
    public static void main(String[] args) {
        Rectangle obj1 = new Rectangle();
        //Shallow Copy
        Rectangle obj2 = new Rectangle();
        obj2.length = obj1.length;
        obj2.breadth = obj1.breadth;

        System.out.println(" Before Changing the value of object 1, the
object2 will be - ");
        System.out.println(" Object2 Length = "+obj2.length+", Object2
Breadth = "+obj2.breadth);

        //Changing the values for object1.
        obj1.length = 10;
        obj1.breadth = 20;

        System.out.println("\n After Changing the value of object 1, the
object2 will be - ");
        System.out.println(" Object2 Length = "+obj2.length+", Object2
Breadth = "+obj2.breadth);
    }
}

```

The above snippet will not affect the object2 values. It has its separate values. The output will be

Before Changing the value of object 1, the object2 will be -
Object2 Length = 5, Object2 Breadth = 3

After Changing the value of object 1, the object2 will be -
Object2 Length = 5, Object2 Breadth = 3

Now we see that we need to write the number of codes for this deep copy. So to reduce this, In java, there is a method called **clone()**.

The **clone()** will do this deep copy internally and return a new object. And to do this we need to write only 1 line of code. That is - **Rectangle obj2 = obj1.clone();**

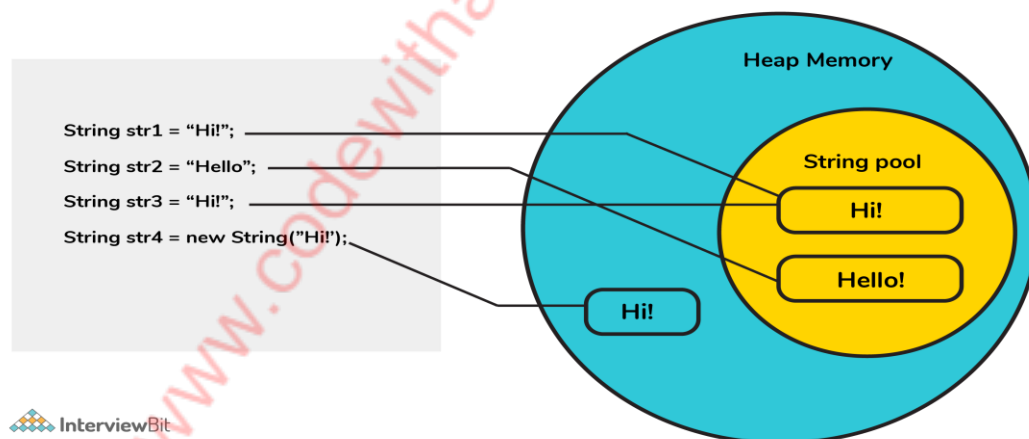
Java Intermediate Interview Questions

31. Apart from the security aspect, what are the reasons behind making strings immutable in Java?

A String is made immutable due to the following reasons:

- **String Pool:** Designers of Java were aware of the fact that String data type is going to be majorly used by the programmers and developers. Thus, they wanted optimization from the beginning. They came up with the notion of using the String pool (a storage area in Java heap) to store the String literals. They intended to decrease the temporary String object with the help of sharing. An immutable class is needed to facilitate sharing. The sharing of the mutable structures between two unknown parties is not possible. Thus, immutable Java String helps in executing the concept of String Pool.

String Pool



- **Multithreading:** The safety of threads regarding the String objects is an important aspect in Java. No external synchronization is required if the String objects are immutable. Thus, a cleaner code can be written for sharing the String objects across different threads. The complex process of concurrency is facilitated by this method.
- **Collections:** In the case of Hashtables and HashMaps, keys are String objects. If the String objects are not immutable, then it can get modified during the period when it resides in the HashMaps. Consequently, the retrieval of the desired data is not possible. Such changing states pose a lot of risks. Therefore, it is quite safe to make the string immutable.

32. What is a singleton class in Java? And How to implement a singleton class?

Singleton classes are those classes, whose objects are created only once. And with only that object the class members can be accessed.

Understand this with the help of an example-:

Consider the water jug in the office and if every employee wants that water then they will not create a new water jug for drinking water. They will use the existing one with their own reference as a glass. So programmatically it should be implemented as -

```
class WaterJug{
    private int waterQuantity = 500;
    private WaterJug(){}
    private WaterJug object = null;

    // Method to provide the service of Giving Water.
    public int getWater(int quantity){
        waterQuantity -= quantity;
        return quantity;
    }
    // Method to return the object to the user.
    public static Waterjug getInstance(){
        // Will Create a new object if the object is not already created and
        return the object.
        if(object == null){
            object = new WaterJug();
        }
        return object;
    }
}
```

In the above class, the Constructor is private so we cannot create the object of the class. But we can get the object by calling the method **getInstance()**. And the getInstance is static so it can be called without creating the object. And it returns the object. Now with that object, we can call **getWater()** to get the water.

```
Waterjug glass1 = WaterJug.getInstance();
glass1.getWater(1);
```

We can get the single object using this getInstance(). And it is static, so it is a thread-safe singleton class. Although there are many ways to create a thread-safe singleton class. So thread-safe classes can also be:

- When singletons are written with double-checked locking, they can be thread-safe.
- We can use static singletons that are initialized during class loading. Like we did in the above example.
- But the most straightforward way to create a thread-safe singleton is to use Java enums.

33. Which of the below generates a compile-time error? State the reason.

1. `int[] n1 = new int[0];`
2. `boolean[] n2 = new boolean[-200];`
3. `double[] n3 = new double[2241423798];`
4. `char[] ch = new char[20];`

We get a compile-time error in line 3. The error we will get in Line 3 is - **integer number too large**. It is because the array requires size as an integer. And Integer takes 4 Bytes in the memory. And the number (**2241423798**) is beyond the capacity of the integer. The maximum array size we can declare is - (**2147483647**).

Because the array requires the size in integer, none of the lines (1, 2, and 4) will give a compile-time error. The program will compile fine. But we get the runtime exception in line 2. The exception is - **NegativeArraySizeException**.

Here what will happen is - At the time when JVM will allocate the required memory during runtime then it will find that the size is negative. And the array size can't be negative. So the JVM will throw the exception.

34. How would you differentiate between a String, StringBuffer, and a StringBuilder?

- **Storage area:** In string, the String pool serves as the storage area. For StringBuilder and StringBuffer, heap memory is the storage area.
- **Mutability:** A String is immutable, whereas both the StringBuilder and StringBuffer are mutable.
- **Efficiency:** It is quite slow to work with a String. However, StringBuilder is the fastest in performing operations. The speed of a StringBuffer is more than a String and less than a StringBuilder. (For example appending a character is fastest in StringBuilder and very slow in String because a new memory is required for the new String with appended character.)
- **Thread-safe:** In the case of a threaded environment, StringBuilder and StringBuffer are used whereas a String is not used. However, StringBuilder is suitable for an environment with a single thread, and a StringBuffer is suitable for multiple threads.

Syntax:

```
// String
String first = "InterviewBit";
String second = new String("InterviewBit");
// StringBuffer
StringBuffer third = new StringBuffer("InterviewBit");
// StringBuilder
StringBuilder fourth = new StringBuilder("InterviewBit");
```

35. Using relevant properties highlight the differences between interfaces and abstract classes.

- **Availability of methods:** Only abstract methods are available in interfaces, whereas non-abstract methods can be present along with abstract methods in abstract classes.
- **Variable types:** Static and final variables can only be declared in the case of interfaces, whereas abstract classes can also have non-static and non-final variables.

- **Inheritance:** Multiple inheritances are facilitated by interfaces, whereas abstract classes do not promote multiple inheritances.
- **Data member accessibility:** By default, the class data members of interfaces are of the public- type. Conversely, the class members for an abstract class can be protected or private also.
- **Implementation:** With the help of an abstract class, the implementation of an interface is easily possible. However, the converse is not true;

Abstract class example:

```
public abstract class Athlete {
    public abstract void walk();
}
```

Interface example:

```
public interface Walkable {
    void walk();
}
```

36. Is this program giving a compile-time error? If Yes then state the reason and number of errors it will give. If not then state the reason.

```
abstract final class InterviewBit{
2.     public abstract void printMessage();
3. }
4. class ScalarAcademy extends InterviewBit{
5.     public void printMessage(){
6.         System.out.println("Welcome to Scalar Academy By InterviewBit");
7.     }
8. }
9. class ScalarTopics extends ScalarAcademy{
10.    public void printMessage(){
11.        System.out.println("Welcome to Scalar Topics By Scalar
Academy");
12.    }
13. }
public class Main{
    public static void main(String[] args) {
        InterviewBit ib = new ScalarTopics();
        ib.printMessage();
    }
}
```

The above program will give a compile-time error. The compiler will throw 2 errors in this.

- **[Illegal Combination of modifiers: abstract and final]** at line 1.
- **[Cannot inherit from final 'InterviewBit']** at line 4.

It is because abstract classes are incomplete classes that need to be inherited for making their concrete classes. And on the other hand, the final keywords in class are used for avoiding inheritance. So these combinations are not allowed in java.

37. What is a Comparator in java?

Consider the example where we have an ArrayList of employees like(EId, Ename, Salary), etc. Now if we want to sort this list of employees based on the names of employees. Then that is not possible to sort using the Collections.sort() method. We need to provide something to the sort() function depending on what values we have to perform sorting. Then in that case a comparator is used.

Comparator is the interface in java that contains the compare method. And by overloading the compare method, we can define that on what basis we need to compare the values.

38. In Java, static as well as private method overriding is possible. Comment on the statement.

The statement in the context is completely False. The static methods have no relevance with the objects, and these methods are of the class level. In the case of a child class, a static method with a method signature exactly like that of the parent class can exist without even throwing any compilation error.

The phenomenon mentioned here is popularly known as method hiding, and overriding is certainly not possible. Private method overriding is unimaginable because the visibility of the private method is restricted to the parent class only. As a result, only hiding can be facilitated and not overriding.

39. What makes a HashSet different from a TreeSet?

Although both HashSet and TreeSet are not synchronized and ensure that duplicates are not present, there are certain properties that distinguish a HashSet from a TreeSet.

- **Implementation:** For a HashSet, the hash table is utilized for storing the elements in an unordered manner. However, TreeSet makes use of the red-black tree to store the elements in a sorted manner.
- **Complexity/ Performance:** For adding, retrieving, and deleting elements, the time amortized complexity is $O(1)$ for a HashSet. The time complexity for performing the same operations is a bit higher for TreeSet and is equal to $O(\log n)$. Overall, the performance of HashSet is faster in comparison to TreeSet.
- **Methods:** hashCode() and equals() are the methods utilized by HashSet for making comparisons between the objects. Conversely, compareTo() and compare() methods are utilized by TreeSet to facilitate object comparisons.
- **Objects type:** Heterogeneous and null objects can be stored with the help of HashSet. In the case of a TreeSet, runtime exception occurs while inserting heterogeneous objects or null objects.

40. Why is the character array preferred over string for storing confidential information?

In Java, a string is basically immutable i.e. it cannot be modified. After its declaration, it continues to stay in the string pool as long as it is not removed in the form of garbage. In other words, a string resides in the heap section of the memory for an unregulated and unspecified time interval after string value processing is executed.

As a result, vital information can be stolen for pursuing harmful activities by hackers if a memory dump is illegally accessed by them. Such risks can be eliminated by using mutable objects or structures like character arrays for storing any variable. After the work of the character array variable is done, the variable can be configured to blank at the same instant. Consequently, it helps in saving heap memory and also gives no chance to the hackers to extract vital data.

41. What do we get in the JDK file?

- **JDK**- For making java programs, we need some tools that are provided by JDK (Java Development Kit). JDK is the package that contains various tools, Compiler, Java Runtime Environment, etc.
- **JRE** - To execute the java program we need an environment. (Java Runtime Environment) JRE contains a library of Java classes + JVM. **What are JAVA Classes?** It contains some predefined methods that help Java programs to use that feature, build and execute. **For example** - there is a system class in java that contains the print-stream method, and with the help of this, we can print something on the console.
- **JVM** - (Java Virtual Machine) JVM is a part of JRE that executes the Java program at the end. Actually, it is part of JRE, but it is software that converts bytecode into machine-executable code to execute on hardware.

JDK

Development
Tools+

JRE

Class Library +

JVM



InterviewBit

42. What are the differences between JVM, JRE and JDK in Java?

Criteria	JDK	JRE	JVM
Abbreviation	Java Development Kit	Java Runtime Environment	Java Virtual Machine
Definition	JDK is a complete software development kit for developing Java applications. It comprises JRE, JavaDoc, compiler, debuggers, etc.	JRE is a software package providing Java class libraries, JVM and all the required components to run the Java applications.	JVM is a platform-dependent, abstract machine comprising of 3 specifications - document describing the JVM implementation requirements, computer program meeting the JVM requirements and instance object for executing the Java byte code and provide the runtime environment for execution.
Main Purpose	JDK is mainly used for code development and execution.	JRE is mainly used for environment creation to execute the code.	JVM provides specifications for all the implementations to JRE.
Tools provided	JDK provides tools like compiler, debuggers, etc for code development	JRE provides libraries and classes required by JVM to run the program.	JVM does not include any tools, but instead, it provides the specification for implementation.
Summary	JDK = (JRE) + Development tools	JRE = (JVM) + Libraries to execute the application	JVM = Runtime environment to execute Java byte code.

43. What are the differences between HashMap and Hashtable in Java?

HashMap	Hashtable
HashMap is not synchronized thereby making it better for non-threaded applications.	Hashtable is synchronized and hence it is suitable for threaded applications.
Allows only one null key but any number of null in the values.	This does not allow null in both keys or values.
Supports order of insertion by making use of its subclass LinkedHashMap.	Order of insertion is not guaranteed in Hashtable.

44. What is the importance of reflection in Java?

- The term `reflection` is used for describing the inspection capability of a code on other code either of itself or of its system and modify it during runtime.
- Consider an example where we have an object of unknown type and we have a method 'fooBar()' which we need to call on the object. The static typing system of Java doesn't allow this method invocation unless the type of the object is known beforehand. This can be achieved using reflection which allows the code to scan the object and identify if it has any method called "fooBar()" and only then call the method if needed.

```
Method methodOfFoo = fooObject.getClass().getMethod("fooBar", null);
methodOfFoo.invoke(fooObject, null);
```

- Using reflection has its own cons:
 - Speed — Method invocations due to reflection are about three times slower than the direct method calls.
 - Type safety — When a method is invoked via its reference wrongly using reflection, invocation fails at runtime as it is not detected at compile/load time.
 - Traceability — Whenever a reflective method fails, it is very difficult to find the root cause of this failure due to a huge stack trace. One has to deep dive into the `invoke()` and `proxy()` method logs to identify the root cause.
- Hence, it is advisable to follow solutions that don't involve reflection and use this method as a last resort.

45. What are the different ways of threads usage?

- We can define and implement a thread in java using two ways:
 - **Extending the Thread class**

```
class InterviewBitThreadExample extends Thread{
    public void run(){
        System.out.println("Thread runs...");
    }
    public static void main(String args[]){
        InterviewBitThreadExample ib = new InterviewBitThreadExample();
        ib.start();
    }
}
```

- **Implementing the Runnable interface**

```
class InterviewBitThreadExample implements Runnable{
    public void run(){
        System.out.println("Thread runs...");
    }
    public static void main(String args[]){
        Thread ib = new Thread(new InterviewBitThreadExample());
        ib.start();
    }
}
```

- Implementing a thread using the method of Runnable interface is more preferred and advantageous as Java does not have support for multiple inheritances of classes.

- `start()` method is used for creating a separate call stack for the thread execution. Once the call stack is created, JVM calls the `run()` method for executing the thread in that call stack.

46. What are the different types of Thread Priorities in Java? And what is the default priority of a thread assigned by JVM?

There are a total of 3 different types of priority available in Java.

MIN_PRIORITY: It has an integer value assigned with 1.

MAX_PRIORITY: It has an integer value assigned with 10.

NORM_PRIORITY: It has an integer value assigned with 5.

In Java, Thread with MAX_PRIORITY gets the first chance to execute. But the default priority for any thread is NORM_PRIORITY assigned by JVM.

47. What is the difference between the program and the process?

- A program can be defined as a line of code written in order to accomplish a particular task. Whereas the process can be defined as the programs which are under execution.
- A program doesn't execute directly by the CPU. First, the resources are allocated to the program and when it is ready for execution then it is a process.

48. What is the difference between the 'throw' and 'throws' keyword in java?

- The '**throw**' keyword is used to manually throw the exception to the calling method.
- And the '**throws**' keyword is used in the function definition to inform the calling method that this method throws the exception. So if you are calling, then you have to handle the exception.

Example -

```
class Main {
    public static int testExceptionDivide(int a, int b) throws
    ArithmeticException{
        if(a == 0 || b == 0)
            throw new ArithmeticException();
        return a/b;
    }
    public static void main(String args[]) {
        try{
            testExceptionDivide(10, 0);
        }
        catch(ArithmeticException e){
            //Handle the exception
        }
    }
}
```

Here in the above snippet, the method `testExceptionDivide` throws an exception. So if the main method is calling it then it must have handled the exception. Otherwise, the main method can also throw the exception to JVM.

And the method `testExceptionDivide` 'throws' the exception based on the condition.

49. What are the differences between constructor and method of a class in Java?

Constructor	Method
Constructor is used for initializing the object state.	Method is used for exposing the object's behavior.
Constructor has no return type.	Method should have a return type. Even if it does not return anything, return type is <code>void</code> .
Constructor gets invoked implicitly.	Method has to be invoked on the object explicitly.
If the constructor is not defined, then a default constructor is provided by the java compiler.	If a method is not defined, then the compiler does not provide it.
The constructor name should be equal to the class name.	The name of the method can have any name or have a class name too.
A constructor cannot be marked as <code>final</code> because whenever a class is inherited, the constructors are not inherited. Hence, marking it <code>final</code> doesn't make sense. Java throws compilation error saying -modifier <code>final</code> not allowed here	A method can be defined as <code>final</code> but it cannot be overridden in its subclasses.
Final variable instantiations are possible inside a constructor and the scope of this applies to the whole class and its objects.	A final variable if initialised inside a method ensures that the variable cant be changed only within the scope of that method.

50. Identify the output of the below java program and Justify your answer.

```
class Main {
    public static void main(String args[]) {
        Scaler s = new Scaler(5);
    }
}
class InterviewBit{
    InterviewBit(){
        System.out.println(" Welcome to InterviewBit ");
    }
}
class Scaler extends InterviewBit{
    Scaler(){
        System.out.println(" Welcome to Scaler Academy ");
    }
    Scaler(int x){
        this();
        super();
        System.out.println(" Welcome to Scaler Academy 2");
    }
}
```

```
}  
}
```

The above code will throw the compilation error. It is because the **super()** is used to call the parent class constructor. But there is the condition that **super()** must be the first statement in the block. Now in this case, if we replace **this()** with **super()** then also it will throw the compilation error. Because **this()** also has to be the first statement in the block. So in conclusion, we can say that we cannot use **this()** and **super()** keywords in the same block.

51. Java works as “pass by value” or “pass by reference” phenomenon?

Java always works as a “pass by value”. There is nothing called a “pass by reference” in Java. However, when the object is passed in any method, the address of the value is passed due to the nature of object handling in Java. When an object is passed, a copy of the reference is created by Java and that is passed to the method. The objects point to the same memory location. 2 cases might happen inside the method:

- **Case 1:** When the object is pointed to another location: In this case, the changes made to that object do not get reflected the original object before it was passed to the method as the reference points to another location.

For example:

```
class InterviewBitTest{  
    int num;  
    InterviewBitTest(int x){  
        num = x;  
    }  
    InterviewBitTest(){  
        num = 0;  
    }  
}  
class Driver {  
    public static void main(String[] args)  
    {  
        //create a reference  
        InterviewBitTest ibTestObj = new InterviewBitTest(20);  
        //Pass the reference to updateObject Method  
        updateObject(ibTestObj);  
        //After the updateObject is executed, check for the value of num in  
        the object.  
        System.out.println(ibTestObj.num);  
    }  
    public static void updateObject(InterviewBitTest ibObj)  
    {  
        // Point the object to new reference  
        ibObj = new InterviewBitTest();  
        // Update the value  
        ibObj.num = 50;  
    }  
}
```

Output:

20

- **Case 2:** When object references are not modified: In this case, since we have the copy of reference the main object pointing to the same memory location, any changes in the content of the object get reflected in the original object.

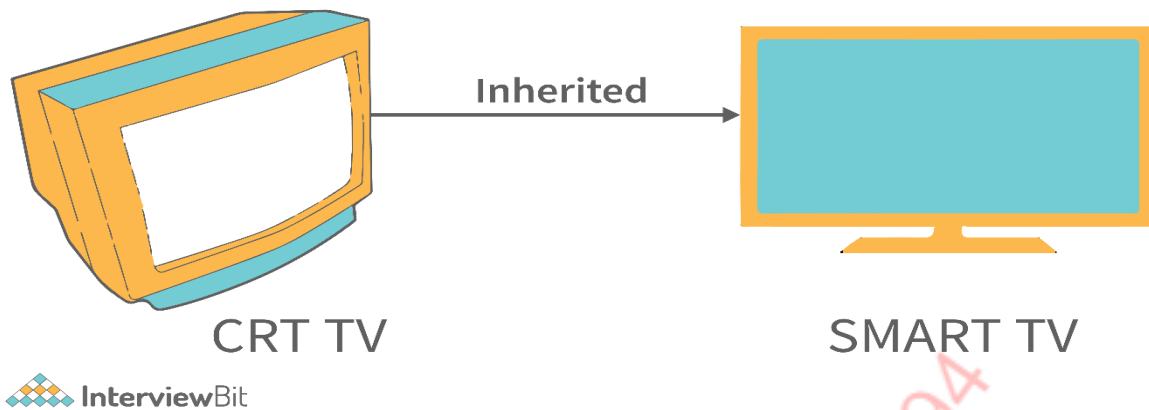
For example:

```
class InterviewBitTest{
    int num;
    InterviewBitTest(int x){
        num = x;
    }
    InterviewBitTest(){
        num = 0;
    }
}
class Driver{
    public static void main(String[] args)
    {
        //create a reference
        InterviewBitTest ibTestObj = new InterviewBitTest(20);
        //Pass the reference to updateObject Method
        updateObject(ibTestObj);
        //After the updateObject is executed, check for the value of num in
        the object.
        System.out.println(ibTestObj.num);
    }
    public static void updateObject(InterviewBitTest ibObj)
    {
        // no changes are made to point the ibObj to new location
        // Update the value of num
        ibObj.num = 50;
    }
}
Output:
50
```

52. What is the 'IS-A' relationship in OOPs java?

'IS-A' relationship is another name for inheritance. When we inherit the base class from the derived class, then it forms a relationship between the classes. So that relationship is termed an 'IS-A' Relationship.

Example - Consider a Television (Typical CRT TV). Now another Smart TV that is inherited from television class. So we can say that the Smart tv is also a TV. Because CRT TV things can also be done in the Smart TV.



So here 'IS-A' Relationship formed. [SmartTV 'IS-A' TV].

53. Which among String or String Buffer should be preferred when there are lot of updates required to be done in the data?

StringBuffer is mutable and dynamic in nature whereas String is immutable. Every updation / modification of String creates a new String thereby overloading the string pool with unnecessary objects. Hence, in the cases of a lot of updates, it is always preferred to use StringBuffer as it will reduce the overhead of the creation of multiple String objects in the string pool.

54. How to not allow serialization of attributes of a class in Java?

- In order to achieve this, the attribute can be declared along with the usage of `transient` keyword as shown below:

```
public class InterviewBitExample {  
  
    private transient String someInfo;  
    private String name;  
    private int id;  
    // :  
    // Getters setters  
    // :  
}
```

- In the above example, all the fields except `someInfo` can be serialized.

55. What happens if the static modifier is not included in the main method signature in Java?

There wouldn't be any compilation error. But then the program is run, since the JVM can't map the main method signature, the code throws "NoSuchMethodError" error at the runtime.

56. Consider the below program, identify the output, and also state the reason for that.

```
public class Main{
public static void main(String[] args) {
    System.out.println(" Hello. Main Method. ");
}
public static void main(int[] args) {
    System.out.println(" Hello. Main Method2. ");
}
}
```

The output of the above program will be **Hello. Main Method.** This is because JVM will always call the main method based on the definition it already has. Doesn't matter how many main methods we overload it will only execute one main method based on its declaration in JVM.

57. Can we make the main() thread a daemon thread?

In java multithreading, the main() threads are always non-daemon threads. And there is no way we can change the nature of the non-daemon thread to the daemon thread.

58. What happens if there are multiple main methods inside one class in Java?

The program can't compile as the compiler says that the method has been already defined inside the class.

59. What do you understand by Object Cloning and how do you achieve it in Java?

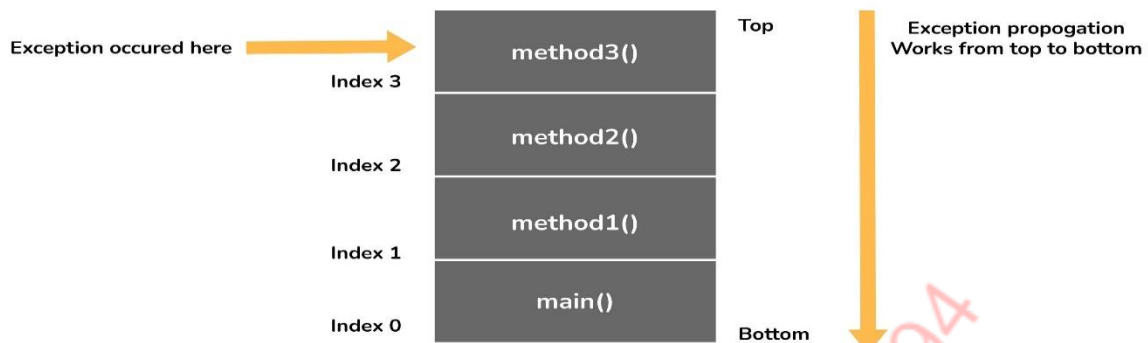
- It is the process of creating an exact copy of any object. In order to support this, a java class has to implement the Cloneable interface of java.lang package and override the clone() method provided by the Object class the syntax of which is:

```
protected Object clone() throws CloneNotSupportedException{
    return (Object)super.clone();
}
```

- In case the Cloneable interface is not implemented and just the method is overridden, it results in CloneNotSupportedException in Java.

60. How does an exception propagate in the code?

When an exception occurs, first it searches to locate the matching catch block. In case, the matching catch block is located, then that block would be executed. Else, the exception propagates through the method call stack and goes into the caller method where the process of matching the catch block is performed. This propagation happens until the matching catch block is found. If the match is not found, then the program gets terminated in the main method.



Java Stack



61. How do exceptions affect the program if it doesn't handle them?

Exceptions are runtime errors. Suppose we are making an android application with java. And it all works fine but there is an exceptional case when the application tries to get the file from storage and the file doesn't exist (This is the case of exception in java). And if this case is not handled properly then the application will crash. This will be a bad experience for users. This is the type of error that cannot be controlled by the programmer. But programmers can take some steps to avoid this so that the application won't crash. The proper action can be taken at this step.

62. Is it mandatory for a catch block to be followed after a try block?

No, it is not necessary for a catch block to be present after a try block. - A try block should be followed either by a catch block or by a finally block. If the exceptions likelihood is more, then they should be declared using the throws clause of the method.

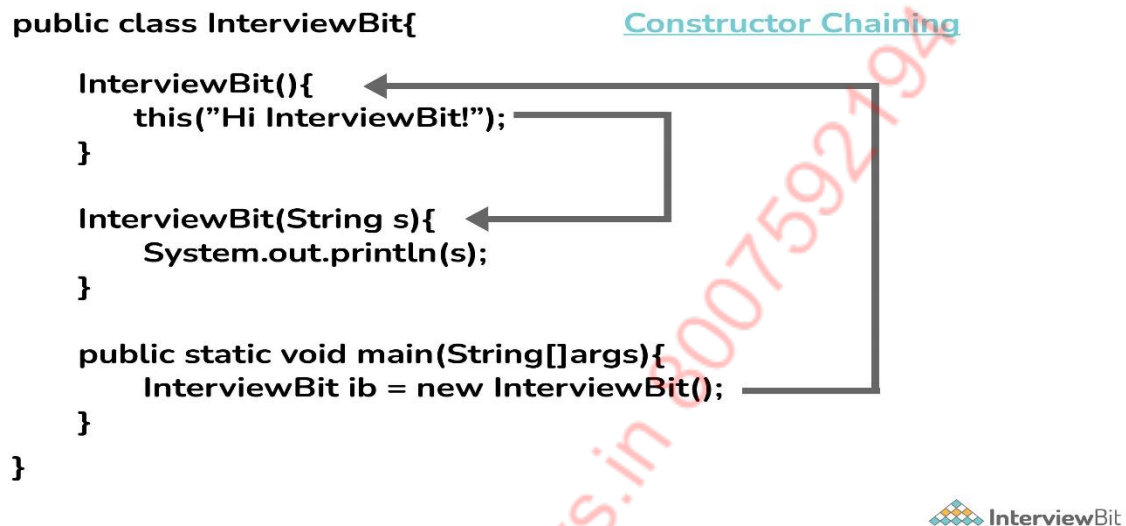
63. Will the finally block get executed when the return statement is written at the end of try block and catch block as shown below?

```
public int someMethod(int i){
    try{
        //some statement
        return 1;
    }catch(Exception e){
        //some statement
        return 999;
    }finally{
        //finally block statements
    }
}
```

finally block will be executed irrespective of the exception or not. The only case where finally block is not executed is when it encounters 'System.exit()' method anywhere in try/catch block.

64. Can you call a constructor of a class inside the another constructor?

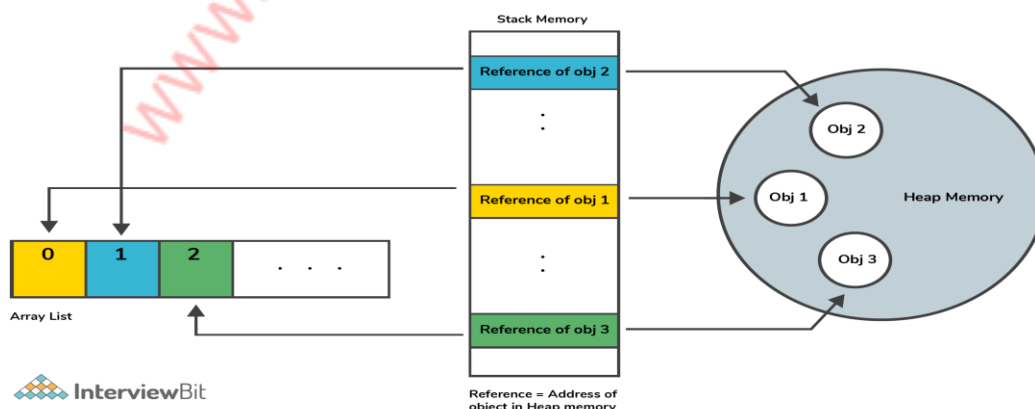
Yes, the concept can be termed as constructor chaining and can be achieved using `this()`.



65. Contiguous memory locations are usually used for storing actual values in an array but not in ArrayList. Explain.

In the case of ArrayList, data storing in the form of primitive data types (like int, float, etc.) is not possible. The data members/objects present in the ArrayList have references to the objects which are located at various sites in the memory. Thus, storing of actual objects or non-primitive data types (like Integer, Double, etc.) takes place in various memory locations.

Sorting of objects in Array List

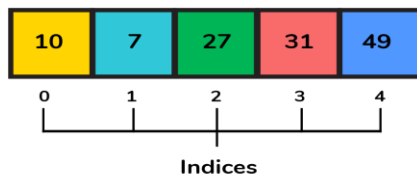


However, the same does not apply to the arrays. Object or primitive type values can be stored in arrays in contiguous memory locations, hence every element does not require any reference to the next element.

Sorting values in Array

How Array look like?

Array



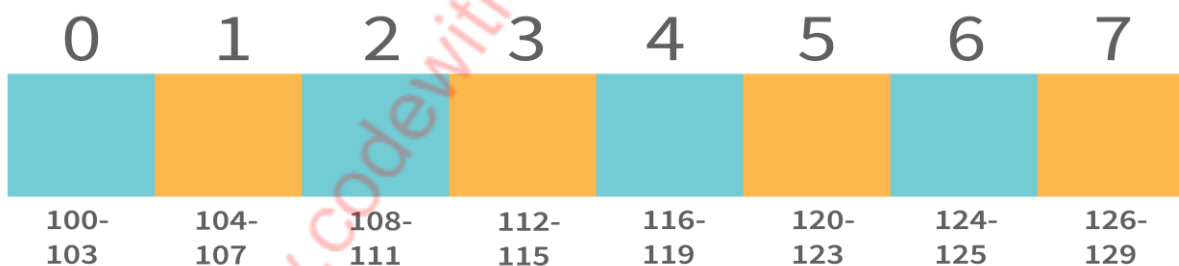
Array in memory



66. Why does the java array index start with 0?

It is because the 0 index array avoids the extra arithmetic operation to calculate the memory address.

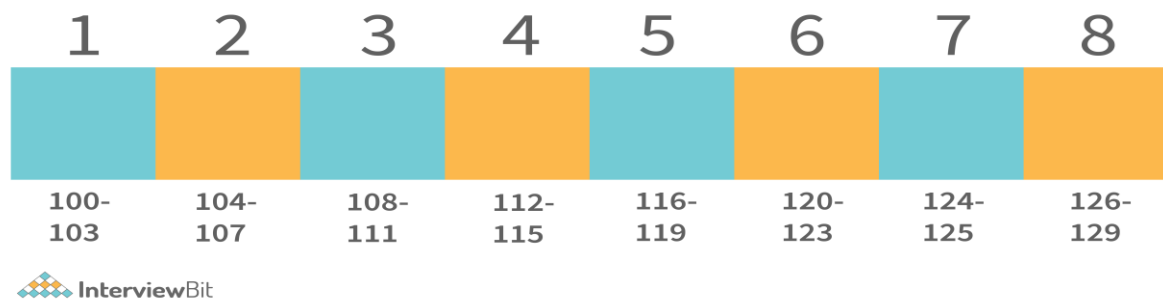
Example - Consider the array and assume each element takes 4-byte memory space. Then the address will be like this -



Now if we want to access index 4. Then internally java calculates the address using the formula-

[Base Address + (index * no_of_bytes)]. So according to this. The starting address of the index 4 will be - **[100 + (4*4)] = 116**. And exactly that's what the address is calculated.

Now consider the same with 1 index Array -



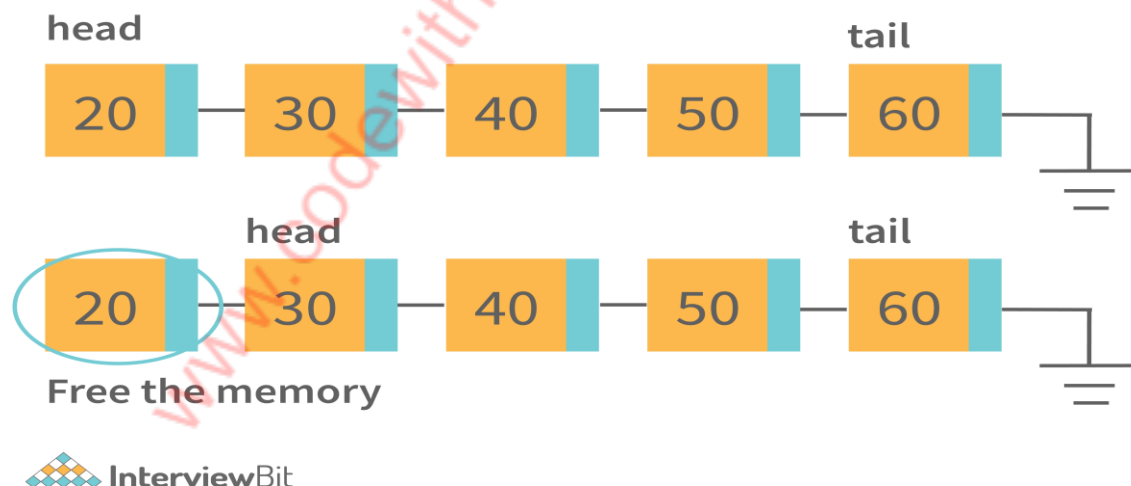
Now if we apply the same formula here. Then we get - **116** as the starting address of the 4th index. Which is wrong. Then we need to apply formula - **[Base Address + ((index-1) * no_of_bytes)]**.

And for calculating this, an extra arithmetic operation has to be performed. And consider the case where millions of addresses need to be calculated, this causes complexity. So to avoid this,) the index array is supported by java.

67. Why is the remove method faster in the linked list than in an array?

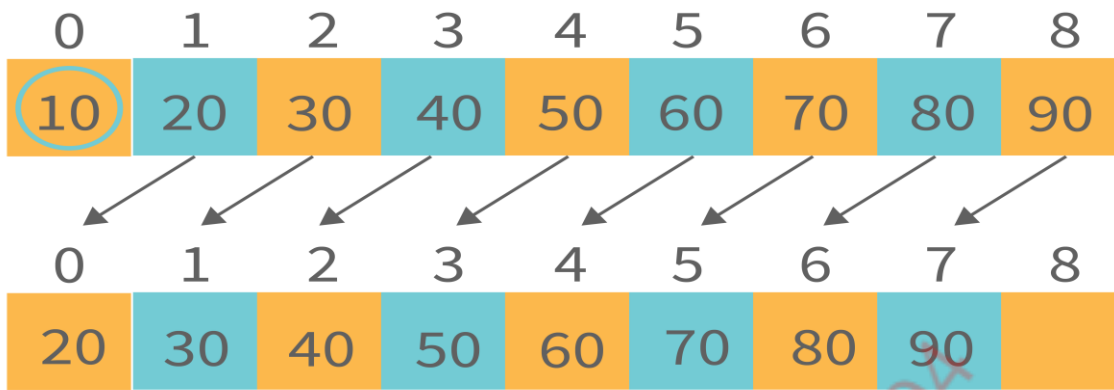
In the linked list, we only need to adjust the references when we want to delete the element from either end or the front of the linked list. But in the array, indexes are used. So to manage proper indexing, we need to adjust the values from the array So this adjustment of value is costlier than the adjustment of references.

Example - To Delete from the front of the linked list, internally the references adjustments happened like this.



The only thing that will change is that the head pointer will point to the head's next node. And delete the previous node. That is the constant time operation.

Whereas in the ArrayList, internally it should work like this-



 InterviewBit

For deletion of the first element, all the next element has to move to one place ahead. So this copying value takes time. So that is the reason why removing in ArrayList is slower than LinkedList.

68. How many overloaded add() and addAll() methods are available in the List interface? Describe the need and uses.

There are a total of 4 overloaded methods for add() and addAll() methods available in List Interface. The below table states the description of all.

Return Type	Method Description
boolean	add(Element e): This method is used for adding the element at the end of the List. The Datatype of the element is of any type it has been initially assigned with. It returns the boolean indicating successfully inserted or not.
void	add(int index, Element e): This method is the overloaded version of add() method. In this, along with the element, the index is also passed to the method for the specific index the value needs to be inserted.
boolean	addAll(Collection <extends ? Element > c): This method helps to add all elements at the end of collections from the list received in the parameter. It contains an iterator that helps to iterate the list and add the elements to the collection.
boolean	addAll(int index, Collection <extends ? Element > c): This is the overloaded method for addAll() method. In this along with the list, we can pass the specified index from which the list elements need to be added.

69. How does the size of ArrayList grow dynamically? And also state how it is implemented internally.

ArrayList is implemented in such a way that it can grow dynamically. We don't need to specify the size of ArrayList. For adding the values in it, the methodology it uses is -

1. Consider initially that there are 2 elements in the ArrayList. [2, 3].



2. If we need to add the element into this. Then internally what will happen is-

- ArrayList will allocate the new ArrayList of Size (current size + half of the current size). And add the old elements into the new. Old - [2, 3], New - [2, 3, null, null].



- Then the new value will be inserted into it. [2, 3, 4, null]. And for the next time, the extra space will be available for the value to be inserted.

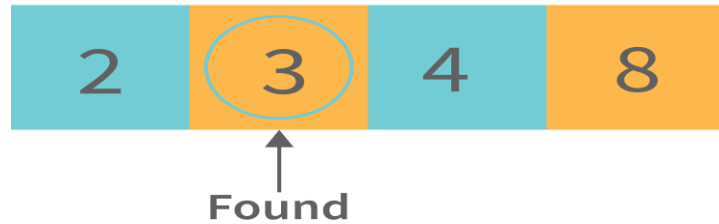


3. This process continues and the time taken to perform all of these is considered as the amortized constant time.

This is how the ArrayList grows dynamically. And when we delete any entry from the ArrayList then the following steps are performed -

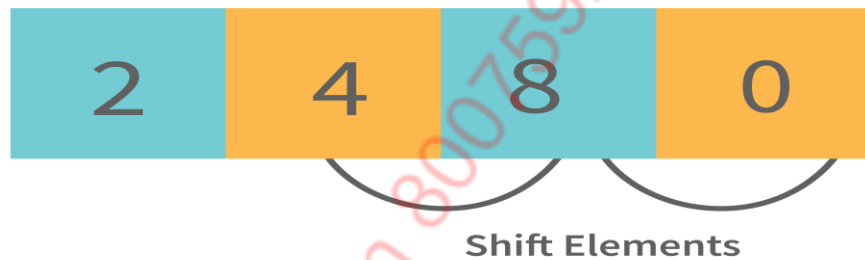
1. It searches for the element index in the array. Searching takes some time. Typically it's $O(n)$ because it needs to search for the element in the entire array.

Search 3



2. After searching the element, it needs to shift the element from the right side to fill the index.

Shifting



So this is how the elements are deleted from the ArrayList internally. Similarly, the search operations are also implemented internally as defined in removing elements from the list (searching for elements to delete).

[Java Advanced Interview Questions](#)

70. Although inheritance is a popular OOPs concept, it is less advantageous than composition. Explain.

Inheritance lags behind composition in the following scenarios:

- Multiple-inheritance is not possible in Java. Classes can only extend from one superclass. In cases where multiple functionalities are required, for example - to read and write information into the file, the pattern of composition is preferred. The writer, as well as reader functionalities, can be made use of by considering them as the private members.
- Composition assists in attaining high flexibility and prevents breaking of encapsulation.
- Unit testing is possible with composition and not inheritance. When a developer wants to test a class composing a different class, then Mock Object can be created for signifying the composed class to facilitate testing. This technique is not possible with the help of inheritance as the derived class cannot be tested without the help of the superclass in inheritance.
- The loosely coupled nature of composition is preferable over the tightly coupled nature of inheritance.

Let's take an example:

```
package comparison;
public class Top {
    public int start() {
        return 0;
    }
}
class Bottom extends Top {
    public int stop() {
        return 0;
    }
}
```

In the above example, inheritance is followed. Now, some modifications are done to the Top class like this:

```
public class Top {
    public int start() {
        return 0;
    }
    public void stop() {
    }
}
```

If the new implementation of the Top class is followed, a compile-time error is bound to occur in the Bottom class. Incompatible return type is there for the Top.stop() function. Changes have to be made to either the Top or the Bottom class to ensure compatibility. However, the composition technique can be utilized to solve the given problem:

```
class Bottom {
    Top par = new Top();
    public int stop() {
        par.start();
        par.stop();
        return 0;
    }
}
```

71. What is the difference between '>>' and '>>>' operators in java?

These 2 are the bitwise right shift operators. Although both operators look similar. But there is a minimal difference between these two right shift operators.

- **'>>' Bitwise Right Shift Operator**- This operator shifts each bit to its right position. And this maintains the signed bit.
- **'>>>' Bitwise Right Shift Operator with trailing zero**- This operator also shifts each bit to its right. But this doesn't maintain the signed bit. This operator makes the Most significant bit to 0.

Example- Num1 = 8, Num2 = -8.

So the binary form of these numbers are -

Num1 = 00000000 00000000 00000000 00001000
Num2 = 11111111 11111111 11111111 11111000

‘>>’ Operator : 8 >> 1 (Shift by one bit) :

Num1 = 00000000 00000000 00000000 00000100
Num2 = 11111111 11111111 11111111 11111100

‘>>>’ Operator : 8 >>> 1 (Shift by one bit) =

Num1 = 00000000 00000000 00000000 00000100
Num2 = 01111111 11111111 11111111 11111100

72. What are Composition and Aggregation? State the difference.

Composition, and Aggregation help to build (Has - A - Relationship) between classes and objects. But both are not the same in the end. **Let’s understand with the help of an example.**

- Consider the University as a class that has some departments in it. So the university will be the container object. And departments in it will contain objects. Now in this case, if the container object destroys then the contained objects will also get destroyed automatically. So here we can say that there is a strong association between the objects. So this Strong Association is called **Composition**.
- Now consider one more example. Suppose we have a class department and there are several professors' objects there in the department. Now if the department class is destroyed then the professor's object will become free to bind with other objects. Because container objects (Department) only hold the references of contained objects (Professor's). So here is the weak association between the objects. And this weak association is called **Aggregation**.

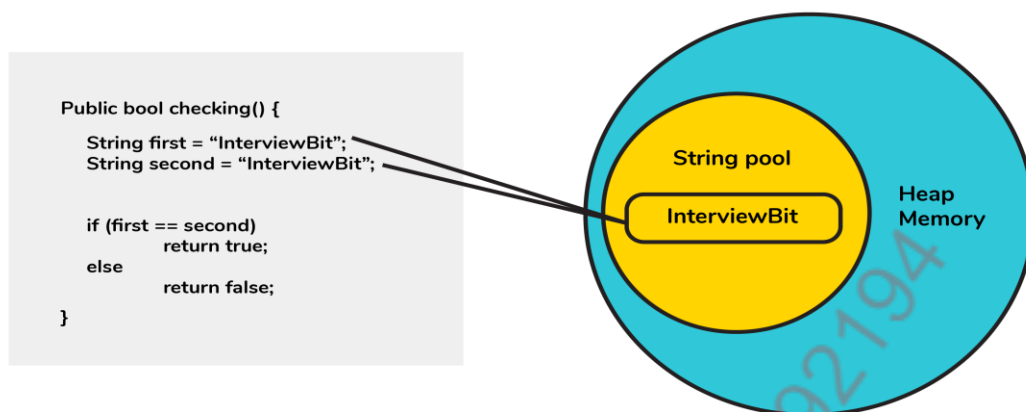
73. How is the creation of a String using new() different from that of a literal?

When a String is formed as a literal with the assistance of an assignment operator, it makes its way into the String constant pool so that String Interning can take place. This same object in the heap will be referenced by a different String if the content is the same for both of them.

```
public bool checking() {  
    String first = "InterviewBit";  
    String second = "InterviewBit";  
    if (first == second)  
        return true;  
    else  
        return false;  
}
```

The checking() function will return true as the same content is referenced by both the variables.

String Pool by means of assignment operator

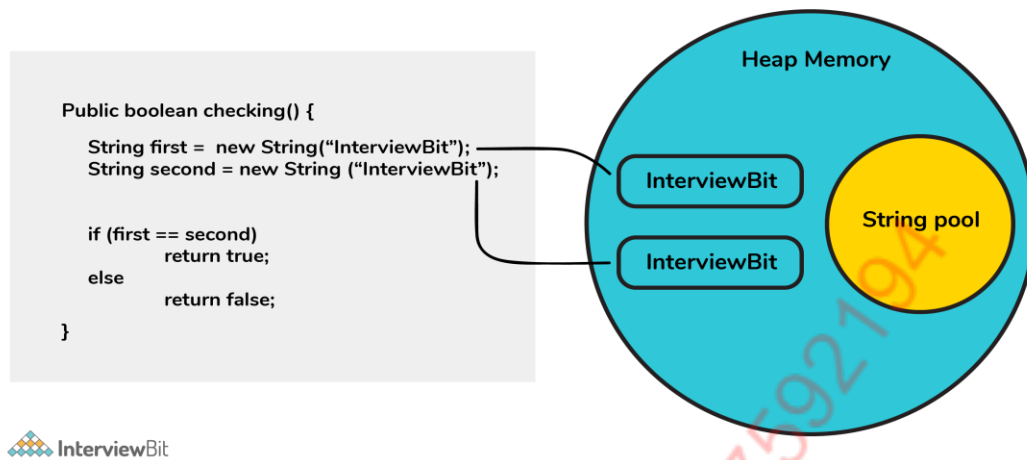


Conversely, when a String formation takes place with the help of a new() operator, interning does not take place. The object gets created in the heap memory even if the same content object is present.

```
public bool checking() {  
    String first = new String("InterviewBit");  
    String second = new String("InterviewBit");  
    if (first == second)  
        return true;  
    else  
        return false;  
}
```

The checking() function will return false as the same content is not referenced by both the variables.

String Pool by means of new operator



74. How is the 'new' operator different from the 'newInstance()' operator in java?

Both '**new**' and '**newInstance()**' operators are used to creating objects. The difference is- that when we already know the class name for which we have to create the object then we use a new operator. But suppose we don't know the class name for which we need to create the object, Or we get the class name from the command line argument, or the database, or the file. Then in that case we use the '**newInstance()**' operator.

The '**newInstance()**' keyword throws an exception that we need to handle. It is because there are chances that the class definition doesn't exist, and we get the class name from runtime. So it will throw an exception.

75. Is exceeding the memory limit possible in a program despite having a garbage collector?

Yes, it is possible for the program to go out of memory in spite of the presence of a garbage collector. Garbage collection assists in recognizing and eliminating those objects which are not required in the program anymore, in order to free up the resources used by them.

In a program, if an object is unreachable, then the execution of garbage collection takes place with respect to that object. If the amount of memory required for creating a new object is not sufficient, then memory is released for those objects which are no longer in the scope with the help of a garbage collector. The memory limit is exceeded for the program when the memory released is not enough for creating new objects.

Moreover, exhaustion of the heap memory takes place if objects are created in such a manner that they remain in the scope and consume memory. The developer should make sure to dereference the object after its work is accomplished. Although the garbage collector

endeavors its level best to reclaim memory as much as possible, memory limits can still be exceeded.

Let's take a look at the following example:

```
List<String> example = new LinkedList<String>();
while(true){
example.add(new String("Memory Limit Exceeded"));
}
```

76. Why is synchronization necessary? Explain with the help of a relevant example.

Concurrent execution of different processes is made possible by synchronization. When a particular resource is shared between many threads, situations may arise in which multiple threads require the same shared resource.

Synchronization assists in resolving the issue and the resource is shared by a single thread at a time. Let's take an example to understand it more clearly. For example, you have a URL and you have to find out the number of requests made to it. Two simultaneous requests can make the count erratic.

No synchronization:

```
package anonymous;
public class Counting {
    private int increase_counter;
    public int increase() {
        increase_counter = increase_counter + 1;
        return increase_counter;
    }
}
```

Without Synchronization

```
package anonymous;
public class Counting {
```

```
    private int increase_counter;
```

```
    public int increase() {
```

```
        increase_counter = increase_counter + 1;
```

```
        return increase_counter;
```

```
    }
```

```
}
```

01

When Thread T1 comes, increase_counter value becomes 11, if increase_counter was 10 before.



Shared Resource



02

Simultaneously, when Thread T2 comes, the value of increase_counter is viewed as 10 incorrectly instead of 11

If a thread Thread1 views the count as 10, it will be increased by 1 to 11. Simultaneously, if another thread Thread2 views the count as 10, it will be increased by 1 to 11. Thus,

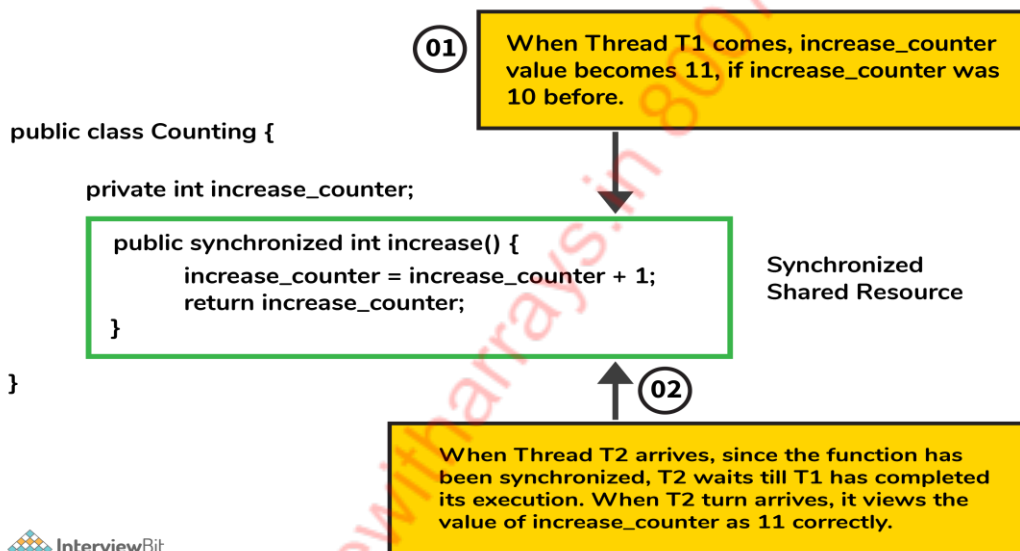
inconsistency in count values takes place because the expected final value is 12 but the actual final value we get will be 11.

Now, the function `increase()` is made synchronized so that simultaneous accessing cannot take place.

With synchronization:

```
package anonymous;
public class Counting {
    private int increase_counter;
    public synchronized int increase() {
        increase_counter = increase_counter + 1;
        return increase_counter;
    }
}
```

With Synchronization



If a thread Thread1 views the count as 10, it will be increased by 1 to 11, then the thread Thread2 will view the count as 11, it will be increased by 1 to 12. Thus, consistency in count values takes place.

77. In the given code below, what is the significance of ... ?

```
public void fooBarMethod(String... variables){
    // method code
}
```

- Ability to provide ... is a feature called varargs (variable arguments) which was introduced as part of Java 5.
- The function having ... in the above example indicates that it can receive multiple arguments of the datatype String.
- For example, the `fooBarMethod` can be called in multiple ways and we can still have one method to process the data as shown below:

```

fooBarMethod("foo", "bar");
fooBarMethod("foo", "bar", "boo");
fooBarMethod(new String[]{"foo", "var", "boo"});
public void myMethod(String... variables){
    for(String variable : variables){
        // business logic
    }
}

```

78. What will be the output of the below java program and define the steps of Execution of the java program with the help of the below code?

```

class InterviewBit{
    int i;
    static int j;
    {
        System.out.println(" Instance Block 1. Value of i = "+i);
    }
    static{
        System.out.println(" Static Block 1. Value of j = "+j);
        method_2();
    }
    {
        i = 5;
    }
    static{
        j = 10;
    }
    InterviewBit(){
        System.out.println(" Welcome to InterviewBit ");
    }
    public static void main(String[] args){
        InterviewBit ib = new InterviewBit();
    }
    public void method_1(){
        System.out.println(" Instance method. ");
    }
    static{
        System.out.println(" Static Block 2. Value of j = "+j);
    }
    {
        System.out.println(" Instance Block 2. Value of i = "+i);
        method_1();
    }
    public static void method_2(){
        System.out.println(" Static method. ");
    }
}

```

The Output we get by executing this program will be

Static Block 1. Value of j = 0

Static method.

Static Block 2. Value of j = 10

Instance Block 1. Value of i = 0

Instance Block 2. Value of i = 5

Instance method.

Welcome to InterviewBit

This is a java tricky interview question frequently asked in java interviews for the experienced. The output will be like this because, when the java program is compiled and gets executed, then there are various steps followed for execution. And the steps are -

- Identification of Static Members from top to bottom.
- Execution of Static variable assignment and a Static block from top to bottom.
- Execution of the main method.
- Identification of Instance Members from top to bottom.
- Execution of Instance variable assignment and Instance block from top to bottom.
- Execution of Constructor.

In above steps from 4 to 6, will be executed for every object creation. If we create multiple objects then for every object these steps will be performed.

Now from the above code, the execution will happen like this -

1. In the step of identification of static members. It is found that -

- static int j.
- static block.
- main method.
- static method_2.

During identification, the JVM will assign the default value in the static int j variable. Then it is currently in the state of reading and indirectly writing. Because the original value is not assigned.

2. In the next step, it will execute the static block and assign the value in static variables.

- First static block it will print and because execution from top to bottom and original value in j is not assigned. So it will print the default value of 0.
- After executing static block 1. It will execute the static method_1 because it is called from the static block 1.
- Then it will assign the original value of 5 in the j variable. And executes the remaining static block.

3. Now it will execute the main method. In which it will create an object for the class InterviewBit. And then the execution of instances will happen.

4. Identify the instance variables and blocks from top to bottom.

- int i.
- Instance block 1.
- Instance method_1.

Like a static variable, the instance variable also has been initialized with the default value 0 and will be in the state of reading and writing indirectly.

5. It will execute the instance methods and assign the original value to the instance variable.

- Prints the Instance block 1. And the current value of i is not assigned till now, so it will print 0.
- Assign the original value to i. Then print instance block 2. And after that instance method will be called and printed because it is being called in the instance block.

6. And at the last step, the constructor will be invoked and the lines will be executed in the constructor.

This is how the java program gets executed.

79. Define System.out.println().

System.out.println() is used to print the message on the console. **System** - It is a class present in **java.lang package**. Out is the static variable of type **PrintStream** class present in the **System class**. **println()** is the method present in the **PrintStream** class.

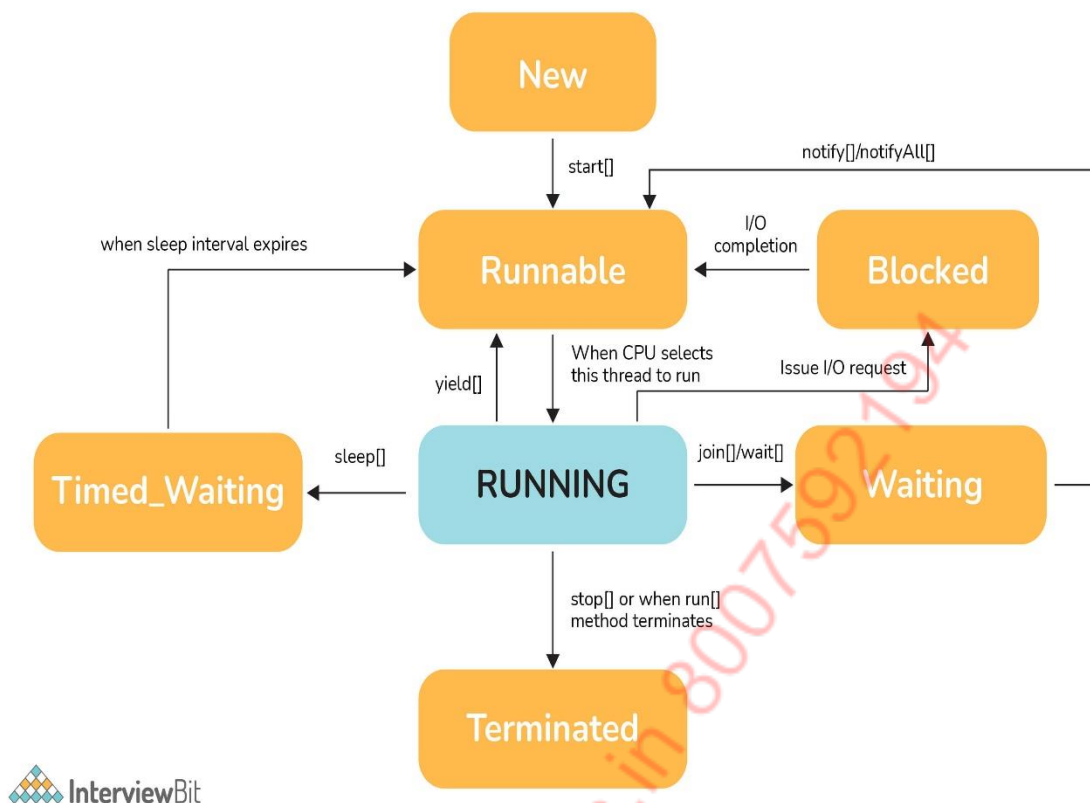
So if we justify the statement, then we can say that if we want to print anything on the console then we need to call the **println()** method that was present in **PrintStream** class. And we can call this using the output object that is present in the **System** class.

80. Can you explain the Java thread lifecycle?

Java thread life cycle is as follows:

- **New** – When the instance of the thread is created and the **start()** method has not been invoked, the thread is considered to be alive and hence in the **NEW** state.
- **Runnable** – Once the **start()** method is invoked, before the **run()** method is called by JVM, the thread is said to be in **RUNNABLE** (ready to run) state. This state can also be entered from the **Waiting** or **Sleeping** state of the thread.
- **Running** – When the **run()** method has been invoked and the thread starts its execution, the thread is said to be in a **RUNNING** state.
- **Non-Runnable (Blocked/Waiting)** – When the thread is not able to run despite the fact of its aliveness, the thread is said to be in a **NON-RUNNABLE** state. Ideally, after some time of its aliveness, the thread should go to a **runnable** state.
 - A thread is said to be in a **Blocked** state if it wants to enter **synchronized** code but it is unable to as another thread is operating in that **synchronized** block on the same object. The first thread has to wait until the other thread exits the **synchronized** block.
 - A thread is said to be in a **Waiting** state if it is waiting for the signal to execute from another thread, i.e it waits for work until the signal is received.
- **Terminated** – Once the **run()** method execution is completed, the thread is said to enter the **TERMINATED** step and is considered to not be alive.

The following flowchart clearly explains the lifecycle of the thread in Java.



81. What could be the tradeoff between the usage of an unordered array versus the usage of an ordered array?

- The main advantage of having an ordered array is the reduced search time complexity of $O(\log n)$ whereas the time complexity in an unordered array is $O(n)$.
- The main drawback of the ordered array is its increased insertion time which is $O(n)$ due to the fact that its element has to be reordered to maintain the order of array during every insertion whereas the time complexity in the unordered array is only $O(1)$.
- Considering the above 2 key points and depending on what kind of scenario a developer requires, the appropriate data structure can be used for implementation.

82. Is it possible to import the same class or package twice in Java and what happens to it during runtime?

It is possible to import a class or package more than once, however, it is redundant because the JVM internally loads the package or class only once.

83. In case a package has sub packages, will it suffice to import only the main package? e.g. Does importing of `com.myMainPackage.*` also import `com.myMainPackage.mySubPackage.*`?

This is a big NO. We need to understand that the importing of the sub-packages of a package needs to be done explicitly. Importing the parent package only results in the import of the classes within it and not the contents of its child/sub-packages.

84. Will the finally block be executed if the code `System.exit(0)` is written at the end of try block?

NO. The control of the program post `System.exit(0)` is immediately gone and the program gets terminated which is why the finally block never gets executed.

85. What do you understand by marker interfaces in Java?

Marker interfaces, also known as tagging interfaces are those interfaces that have no methods and constants defined in them. They are there for helping the compiler and JVM to get run time-related information regarding the objects.

86. Explain the term “Double Brace Initialisation” in Java?

This is a convenient means of initializing any collections in Java. Consider the below example.

```
import java.util.HashSet;
import java.util.Set;

public class IBDoubleBraceDemo{
    public static void main(String[] args){
        Set<String> stringSets = new HashSet<String>()
        {
            {
                add("set1");
                add("set2");
                add("set3");
            }
        };

        doSomething(stringSets);
    }

    private static void doSomething(Set<String> stringSets){
        System.out.println(stringSets);
    }
}
```

In the above example, we see that the `stringSets` were initialized by using double braces.

- The first brace does the task of creating an anonymous inner class that has the capability of accessing the parent class's behavior. In our example, we are creating the subclass of `HashSet` so that it can use the `add()` method of `HashSet`.
- The second braces do the task of initializing the instances.

Care should be taken while initializing through this method as the method involves the creation of anonymous inner classes which can cause problems during the garbage collection or serialization processes and may also result in memory leaks.

87. Why is it said that the length() method of String class doesn't return accurate results?

- The length method returns the number of Unicode units of the String. Let's understand what Unicode units are and what is the confusion below.
- We know that Java uses UTF-16 for String representation. With this Unicode, we need to understand the below two Unicode related terms:
 - Code Point: This represents an integer denoting a character in the code space.
 - Code Unit: This is a bit sequence used for encoding the code points. In order to do this, one or more units might be required for representing a code point.
- Under the UTF-16 scheme, the code points were divided logically into 17 planes and the first plane was called the Basic Multilingual Plane (BMP). The BMP has classic characters - U+0000 to U+FFFF. The rest of the characters- U+10000 to U+10FFFF were termed as the supplementary characters as they were contained in the remaining planes.
 - The code points from the first plane are encoded using **one** 16-bit code unit
 - The code points from the remaining planes are encoded using **two** code units.

Now if a string contained supplementary characters, the length function would count that as 2 units and the result of the length() function would not be as per what is expected.

In other words, if there is 1 supplementary character of 2 units, the length of that SINGLE character is considered to be TWO - Notice the inaccuracy here? As per the java documentation, it is expected, but as per the real logic, it is inaccurate.

88. What is the output of the below code and why?

```
public class InterviewBit{
    public static void main(String[] args)
    {
        System.out.println('b' + 'i' + 't');
    }
}
```

“bit” would have been the result printed if the letters were used in double-quotes (or the string literals). But the question has the character literals (single quotes) being used which is why concatenation wouldn't occur. The corresponding ASCII values of each character would be added and the result of that sum would be printed.

The ASCII values of 'b', 'i', 't' are:

- 'b' = 98
- 'i' = 105
- 't' = 116

$$98 + 105 + 116 = 319$$

Hence 319 would be printed.

89. What are the possible ways of making object eligible for garbage collection (GC) in Java?

First Approach: Set the object references to null once the object creation purpose is served.

```
public class IBGarbageCollect {
    public static void main (String [] args){
        String s1 = "Some String";
        // s1 referencing String object - not yet eligible for GC
        s1 = null; // now s1 is eligible for GC
    }
}
```

Second Approach: Point the reference variable to another object. Doing this, the object which the reference variable was referencing before becomes eligible for GC.

```
public class IBGarbageCollect {
    public static void main(String [] args){
        String s1 = "To Garbage Collect";
        String s2 = "Another Object";
        System.out.println(s1); // s1 is not yet eligible for GC
        s1 = s2; // Point s1 to other object pointed by s2
        /* Here, the string object having the content "To Garbage Collect" is
        not referred by any reference variable. Therefore, it is eligible for GC */
    }
}
```

Third Approach: Island of Isolation Approach: When 2 reference variables pointing to instances of the same class, and these variables refer to only each other and the objects pointed by these 2 variables don't have any other references, then it is said to have formed an "Island of Isolation" and these 2 objects are eligible for GC.

```
public class IBGarbageCollect {
    IBGarbageCollect ib;
    public static void main(String [] str){
        IBGarbageCollect ibgc1 = new IBGarbageCollect();
        IBGarbageCollect ibgc2 = new IBGarbageCollect();
        ibgc1.ib = ibgc2; //ibgc1 points to ibgc2
        ibgc2.ib = ibgc1; //ibgc2 points to ibgc1
        ibgc1 = null;
        ibgc2 = null;
        /*
        * We see that ibgc1 and ibgc2 objects refer
        * to only each other and have no valid
        * references- these 2 objects for island of isolation - eligible
        for GC
        */
    }
}
```

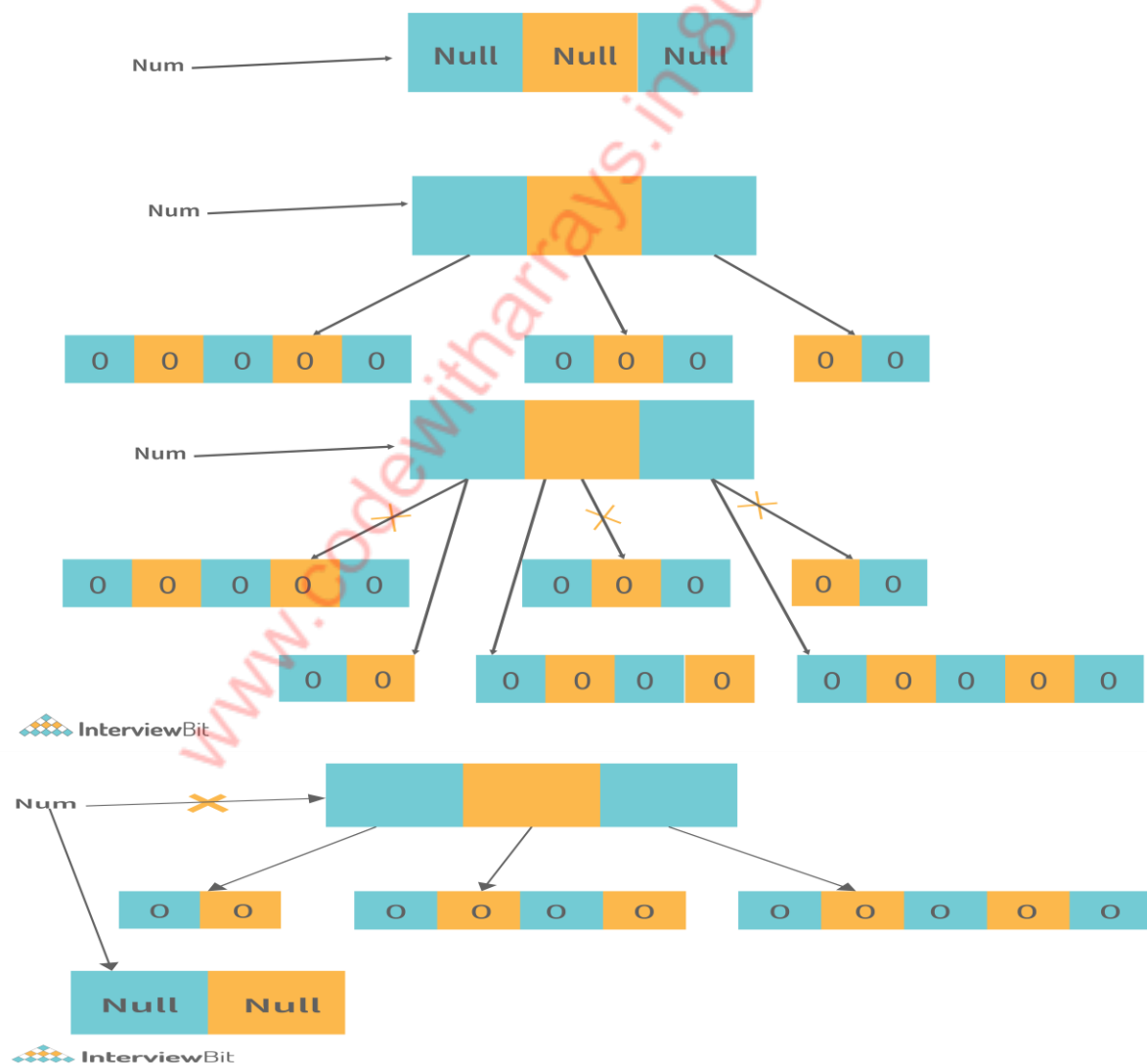
90. In the below Java Program, how many objects are eligible for garbage collection?

```
class Main{
    public static void main(String[] args){
        int[][] num = new int[3][];
        num[0] = new int[5];
        num[1] = new int[2];
        num[2] = new int[3];

        num[2] = new int[5];
        num[0] = new int[4];
        num[1] = new int[3];

        num = new int[2][];
    }
}
```

In the above program, a total of 7 objects will be eligible for garbage collection. Let's visually understand what's happening in the code.



In the above figure on line 3, we can see that on each array index we are declaring a new array so the reference will be of that new array on all the 3 indexes. So the old array will be pointed to by none. So these three are eligible for garbage collection. And on line 4, we are creating a new array object on the older reference. So that will point to a new array and older multidimensional objects will become eligible for garbage collection.

91. What is the best way to inject dependency? Also, state the reason.

There is no boundation for using a particular dependency injection. But the recommended approach is -

Setters are mostly recommended for optional dependencies injection, and constructor arguments are recommended for mandatory ones. This is because constructor injection enables the injection of values into immutable fields and enables reading them more easily.

92. How we can set the spring bean scope. And what supported scopes does it have?

A scope can be set by an annotation such as the @Scope annotation or the "scope" attribute in an XML configuration file. Spring Bean supports the following five scopes:

- Singleton
- Prototype
- Request
- Session
- Global-session

93. What are the different categories of Java Design patterns?

Java Design patterns are categorized into the following different types. And those are also further categorized as

Structural patterns:

- Adapter
- Bridge
- Filter
- Composite
- Decorator
- Facade
- Flyweight
- Proxy

Behavioral patterns:

- Interpreter
- Template method/ pattern
- Chain of responsibility
- Command pattern
- Iterator pattern

- Strategy pattern
- Visitor pattern

J2EE patterns:

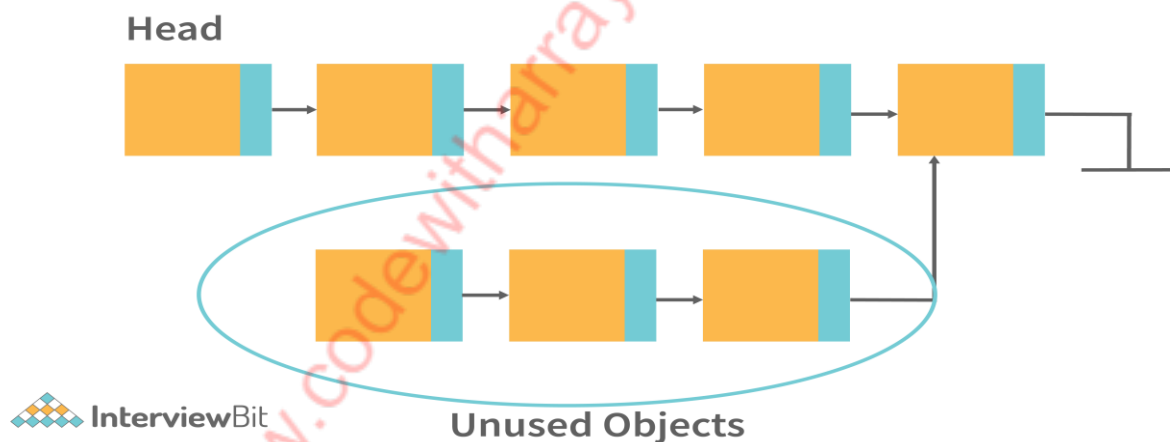
- MVC Pattern
- Data Access Object pattern
- Front controller pattern
- Intercepting filter pattern
- Transfer object pattern

Creational patterns:

- Factory method/Template
- Abstract Factory
- Builder
- Prototype
- Singleton

94. What is a Memory Leak? Discuss some common causes of it.

The Java Garbage Collector (GC) typically removes unused objects when they are no longer required, but when they are still referenced, the unused objects cannot be removed. So this causes the memory leak problem. **Example** - Consider a linked list like the structure below -



In the above image, there are unused objects that are not referenced. But then also Garbage collection will not free it. Because it is referencing some existing referenced object. So this can be the situation of memory leak.

Some common causes of Memory leaks are -

- When there are Unbounded caches.
- Excessive page swapping is done by the operating system.
- Improper written custom data structures.
- Inserting into a collection object without first deleting it.
etc.

95. Assume a thread has a lock on it, calling the sleep() method on that thread will release the lock?

A thread that has a lock won't be released even after it calls sleep(). Despite the thread sleeping for a specified period of time, the lock will not be released.

Java Programming Interview Questions

96. Check if a given string is palindrome using recursion.

```
/*
 * Java program to check if a given inputted string is palindrome or not
 * using recursion.
 */
import java.util.*;
public class InterviewBit {
    public static void main(String args[]) {
        Scanner s = new Scanner(System.in);
        String word = s.nextLine();
        System.out.println("Is "+word+" palindrome? -
"+isWordPalindrome(word));
    }

    public static boolean isWordPalindrome(String word){
        String reverseWord = getReverseWord(word);
        //if word equals its reverse, then it is a palindrome
        if(word.equals(reverseWord)){
            return true;
        }
        return false;
    }

    public static String getReverseWord(String word){
        if(word == null || word.isEmpty()){
            return word;
        }

        return word.charAt(word.length()- 1) +
getReverseWord(word.substring(0, word.length() - 1));
    }
}
```

97. Write a Java Program to print Fibonacci Series using Recursion.

```
class InterviewBit {
    public static void printFibonacci(int val_1, int val_2, int num){
        //Base Case
        if(num == 0)
            return;

        //Printing the next Fibonacci number
        System.out.print( val_1 + val_2 + " ");

        //Recursively calling for printing Fibonacci for remaining length
        printFibonacci(val_2, val_1+val_2, --num);
    }
}
```

```

public static void main(String args[]) {
    System.out.println(" *** Fibonacci Series *** ");

    //Printing the first two values
    System.out.print("0 1 ");

    //Calling Method to print the fibonacci for length 10
    printFibonacci(0, 1, 10);
}
}

```

In the above code, we are printing the base 2 Fibonacci values 0 and 1. And then based on the length of Fibonacci to be printed, we are using the helper function to print that.

98. Write a Java program to check if the two strings are anagrams.

The main idea is to validate the length of strings and then if found equal, convert the string to char array and then sort the arrays and check if both are equal.

```

import java.util.Arrays;
import java.util.Scanner;
public class InterviewBit {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        //Input from two strings
        System.out.print("First String: ");
        String string1 = s.nextLine();
        System.out.print("Second String: ");
        String string2 = s.nextLine();
        // check for the length
        if(string1.length() == string2.length()) {
            // convert strings to char array
            char[] characterArray1 = string1.toCharArray();
            char[] characterArray2 = string2.toCharArray();
            // sort the arrays
            Arrays.sort(characterArray1);
            Arrays.sort(characterArray2);
            // check for equality, if found equal then anagram, else not an
            anagram
            boolean isAnagram = Arrays.equals(characterArray1, characterArray2);
            System.out.println("Anagram: "+ isAnagram);
        }
    }
}

```

99. Write a Java Program to find the factorial of a given number.

```
public class FindFactorial {  
    public static void main(String[] args) {  
        int num = 10;  
        long factorialResult = 1l;  
        for(int i = 1; i <= num; ++i)  
        {  
            factorialResult *= i;  
        }  
        System.out.println("Factorial: "+factorialResult);  
    }  
}
```

100. Given an array of non-duplicating numbers from 1 to n where one number is missing, write an efficient java program to find that missing number.

Idea is to find the sum of n natural numbers using the formula and then finding the sum of numbers in the given array. Subtracting these two sums results in the number that is the actual missing number. This results in O(n) time complexity and O(1) space complexity.

```
public class IBMissingNumberProblem {  
    public static void main(String[] args) {  
        int[] array={4,3,8,7,5,2,6};  
        int missingNumber = findMissingNum(array);  
        System.out.println("Missing Number is "+ missingNumber);  
    }  
  
    public static int findMissingNum(int[] array) {  
        int n=array.length+1;  
        int sumOfFirstNNums=n*(n+1)/2;  
        int actualSumOfArr=0;  
        for (int i = 0; i < array.length; i++) {  
            actualSumOfArr+=array[i];  
        }  
        return sumOfFirstNNums-actualSumOfArr;  
    }  
}
```

101. Write a Java Program to check if any number is a magic number or not. A number is said to be a magic number if after doing sum of digits in each step and return doing sum of digits of that sum, the ultimate result (when there is only one digit left) is 1.

Example, consider the number:

- Step 1: 163 => 1+6+3 = 10
- Step 2: 10 => 1+0 = 1 => Hence 163 is a magic number

```
public class IBMagicNumber{

    public static void main(String[] args) {
        int num = 163;
        int sumOfDigits = 0;
        while (num > 0 || sumOfDigits > 9)
        {
            if (num == 0)
            {
                num = sumOfDigits;
                sumOfDigits = 0;
            }
            sumOfDigits += num % 10;
            num /= 10;
        }

        // If sum is 1, original number is magic number
        if(sumOfDigits == 1) {
            System.out.println("Magic number");
        }else {
            System.out.println("Not magic number");
        }
    }
}
```

102. Write a Java program to create and throw custom exceptions.

```
class InterviewBit {
    public static void main(String args[]) throws CustomException {

        // Throwing the custom exception by passing the message
        throw new CustomException(" This is my custom Exception ");
    }
}

//Creating Custom Exception Class
class CustomException extends Exception{
    //Defining Constructor to throw exception message
    public CustomException(String message){
        super(message);
    }
}
```

We have created the exception class named with CustomException and called the base exception constructor with the error message that we want to print. And to avoid handling exceptions in the main method, we have used the throws keyword in the method declaration.

103. Write a Java program to reverse a string.

```
class InterviewBit{
    public static void main(String[] args){
        //Input String
        String str = "Welcome to InterviewBit";

        //Pointers.
        int i = 0, j = str.length()-1;

        //Result character array to store the reversed string.
        char[] revString = new char[j+1];

        //Looping and reversing the string.
        while(i < j){
            revString[j] = str.charAt(i);
            revString[i] = str.charAt(j);
            i++;
            j--;
        }
        //Printing the reversed String.
        System.out.println("Reversed String = " +
String.valueOf(revString));
    }
}
```

In the above code, we are storing the last character from the string to the first and the first value to the last in the output character array. And doing the same thing in the loop for the remaining 2nd to n-1 characters. This is how the string will be reversed.

104. Write a Java program to rotate arrays 90 degree clockwise by taking matrices from user input.

```
import java.util.Scanner;
public class InterviewBit
{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int no;
        System.out.print("Enter size of Array : ");
        no = sc.nextInt();
        int[][] a = new int[no][no];
        System.out.print("Enter  "+ no*no+" Element Array : ");

        for(int i = 0; i<no; i++){
            for(int j = 0; j<no; j++){
                a[i][j] = sc.nextInt();
            }
        }
        System.out.print("\nArray Before Rotation\n\n");
        for(int i = 0; i<no; i++){
            for(int j = 0; j<no; j++){
                System.out.print(a[i][j] + " ");
            }
            System.out.println();
        }

        System.out.println("\n");
    }
}
```

```

//Rotation

//Transpose
for(int i = 0; i < no; i++){
    for(int j = i; j < no; j++){
        int temp = a[i][j];
        a[i][j] = a[j][i];
        a[j][i] = temp;
    }
}

//Reverse Each row
for(int i = 0; i < no; i++){
    int l, j;
    for(j = 0, l = no - 1; j < l; j++){
        int temp = a[i][j];
        a[i][j] = a[i][l];
        a[i][l] = temp;
        l--;
    }
}

System.out.println("Array After Rotation - \n");

for(int i = 0; i < no; i++){
    for(int j = 0; j < no; j++){
        System.out.print(a[i][j] + " ");
    }
    System.out.println();
}
}

```

In the above code, for rotating the matrix to 90 degrees we are first transposing the matrix so the row becomes the column. And after that, we are reversing each row in the matrix. So this is how the matrix got rotated.

105. Write a java program to check if any number given as input is the sum of 2 prime numbers.

Example :

Input - 18

Output -

18 = 13 + 5

18 = 11 + 7

```

public class InterviewBit
{
    // Method to Check Prime Number
    private static int check_prime(int num){
        int flag = 0;
        for(int i = 2; i<=num/2; i++){
            if(num%i == 0){
                flag = 1;
                return 1;
            }
        }
        if(flag == 0)
            return 0;
        return 1;
    }
    // Method to get print the prime sum
    private static void find(int num){
        for(int i = 2; i <= num/2; i++){
            if(check_prime(i) == 0){
                if(check_prime(num-i) == 0)
                    System.out.println(num + " = " + (num-i) + " " + i);
            }
        }
    }
    public static void main(String[] args) {
        find(18);
    }
}

```

In the above code, for any number **n**, we find all the 2 pairs of numbers that are added together resulting in **n**. And each checking number if it is prime. If it is prime then we are printing that.

106. Write a Java program for solving the Tower of Hanoi Problem.

```

public class InterviewBit
{
    //Recursive Method for Solving the Tower of hanoi.
    private static void TOH(char source, char auxiliary, char destination,
int numOfDisk){
        if (numOfDisk > 0){
            TOH(source, destination, auxiliary, numOfDisk-1);
            System.out.println("Move 1 disk from "+source+" to
"+destination+" using "+auxiliary+".");
            TOH(auxiliary, source, destination, numOfDisk-1);
        }
    }
    public static void main(String[] args) {
        TOH('A','B','C', 3);
    }
}

```

In the above code we are first moving the **n-1** disk from Tower **A** to Tower **B**, then moving that **nth** disk from Tower **A** to Tower **C**, and finally, the remaining **n-1** disk from Tower **B** to Tower **C**. And we are doing this recursively for the **n-1** disk.

107. Implement Binary Search in Java using recursion.

```
public class Main
{
    //Recursive method for binary search
    private static boolean binarySearch(int[] arr, int low, int high, int
key){

        //Calculating Mid.
        int mid = (low + high)/2;

        //Base Case.
        if(low > high)
            return false;

        //Checking if the key is found in the middle.
        if(arr[mid] == key)
            return true;

        //Searching on the left half if a key exists there.
        if(key < arr[mid])
            return binarySearch(arr, low, mid-1, key);

        //Searching on the other half otherwise.
        return binarySearch(arr, mid+1, high, key);
    }
    public static void main(String[] args) {

        int[] arr = {2, 5, 9, 13, 17, 21, 30};
        if(binarySearch(arr, 0, (arr.length-1), 30))
            System.out.println(" Element Found. ");
        else
            System.out.println(" Element not Found.");
    }
}
```

In the above code, we are finding the middle element each time and checking if the element is in the middle or not. If it is not, then we check on which side from the middle it exists. And Recursively searching on the particular subarray. So this way we are reducing the search space by 2 every time. So the search time is very low.

Conclusion

108. Conclusion

Java is one of the simple high-level languages that provides powerful tools and impressive standards required for application development. It was also one of the first languages to provide amazing threading support for tackling concurrency-based problems. The easy-to-use syntax and the built-in features of Java combined with the stability it provides to applications are the main reasons for this language to have ever-growing usage in the software community.

- [How to Become a Java Developer?](#)
- [How much does a Java Developer earn in India?](#)
- [Java Projects](#)
- [Java Programming Questions for interview](#)
- [Java 8 Interview Questions](#)
- [Java String Interview Questions](#)
- [Spring Interview Questions](#)
- [Hibernate Interview Questions](#)
- [Java Collections Interview Questions](#)
- [Array Interview Questions](#)
- [Design Patterns Interview Questions](#)
- [Multithreading Interview Questions](#)
- [Java Tutorial](#)
- [Java MCQ](#)
- [Advance Java MCQ](#)
- [Difference Between C++ and Java](#)
- [Difference Between C and Java](#)
- [Difference Between Java and Javascript](#)
- [Kotlin Vs Java](#)
- [Java Vs Python](#)
- [Features of Java 9](#)
- [Java 8 Features](#)
- [Java Frameworks](#)
- [Java Developer Skills](#)
- [Java IDE](#)
- [Java 11 Features](#)

→Java MCQ:-

REFERENCE:- INTERVIEW_BIT:-

1.What is the component used for compiling, debugging, and executing java programs?

- ☒ JDK
- ☐ JVM
- ☐ JRE
- ☐ JIT

2. What component does the task of bytecode to machine code conversion?

- ☒ JDK
- ☐ JVM
- ☐ JRE
- ☐ JIT

3. Which of the following is the functionality of the java interpreter?

- ☐ Interpreter is nothing but the JIT compiler.
- ☐ It acts as medium between JVM and JIT.
- ☐ It does the conversion of byte code to machine code.
- ☐ It reads the high level code and executes them.

4. When an object has its own lifecycle and its child object can't belong to another parent object, what is it called?

- ☒ Association
- ☐ Aggregation
- ☐ Composition
- ☐ Encapsulation

5. What is the output of the below piece of code?

```
class InterviewBit {  
    public void method1 (int num1,float num2){  
        System.out.println("int-float method");  
    }  
    public void method1(float num1,int num2){  
        System.out.println("float-int method");  
    }  
    public static void main(String[] args){  
        InterviewBit interviewBit=new InterviewBit();  
        interviewBit.method1(40,20);  
    }  
}
```

- ☒ int-float method
- ☐ float-int method
- ☐ Compilation Error
- ☐ Run Time error

6. What is the output of the following code?

```
class InterviewBit{
    int fun (int n)
    {
        int result;
        result = fun (n - 1);
        return result;
    }
}
class Driver{
    public static void main(String args[])
    {
        InterviewBit ib = new InterviewBit() ;
        System.out.print(ib.fun(12));
    }
}
```

- ☒ 0
- ☐ 1
- ☐ Compilation Error
- ☐ Run time error

7. Which of the following happens when the garbage collection process kicks off during the execution of the thread?

- ☒ Garbage collection does not happen during thread execution.
- ☐ Thread pauses while the garbage collection process runs.
- ☐ Both the process takes place simultaneously and does not interfere its execution.
- ☐ Nothing happens, the thread proceeds with execution.

8. What is the output of the below code?

```
class InterviewBit{
    public static void main(String args[])
    {
        String obj = "Hello";
        String obj1 = "InterviewBit";
        String obj2 = "Hello";
        System.out.println(obj.equals(obj1) + " " + obj.equals(obj2));
    }
}
```

- ☒ false false
- ☐ true true
- ☐ true false

☐ false true

9. What is the functionality of `Class.getInstance()` ?

- ☒ It invokes the constructor.
- ☐ It has the same functionality of new operator.
- ☐ It creates object if the class does not have constructor defined.
- ☐ None of the above.

10. What is the output of the below code?

```
class InterviewBit{
    public int num1;
    static int num2;
    void calculate(int a, int b)
    {
        num1 += a ;
        num2 += b;
    }
}
class Driver{
    public static void main(String args[])
    {
        InterviewBit obj1 = new InterviewBit();
        InterviewBit obj2 = new InterviewBit();
        obj1.num1 = 0;
        obj1.num2 = 0;
        obj1.calculate(1, 2);
        obj2.num1 = 0;
        obj2.calculate(2, 3);
        System.out.println(obj1.num1 + " " + obj2.num2);
    }
}
```

- ☒ 1 2
- ☐ 1 5
- ☐ 4 2
- ☐ 2 5

11. What is the output of the following code?

```
class InterviewBit{
    int calculate(int a, int b)
    {
        try{
            return a-b;
        }catch(Exception e){
            return a+b;
        }finally{
```

```

        return a*b;
    }
}
class Driver{
    public static void main(String args[])
    {
        InterviewBit obj1 = new InterviewBit();
        int result = obj1.calculate(2, 3);
        System.out.println("Result: " + result);
    }
}

```

- ☐ 5
- ☐ -1
- ☐ 6
- ☐ Run time Error

→1.Oracle – Java Interview Questions:-

Here is the list Java Interview Questions which are recently asked in Oracle company. These questions are included for both Freshers and Experienced professionals.

1. Can you explain the thread lifecycle in Java?

The life cycle of the thread in java is controlled by Java virtual machine. The java thread states are as follows:

- New
- Runnable
- Running
- Non-Runnable (Blocked)
- Terminated

NEW is a newly created thread that has not yet started the execution. RUNNABLE is either running or ready for execution but it's waiting for resource allocation. BLOCKED is waiting to acquire a monitor lock to enter or re-enter a synchronized block/method. The thread is in running state if the thread scheduler has selected it. A thread is in terminated or dead state when its run() method exits.

2. What do you know about Interface in Java?

The interface in Java is a mechanism to achieve abstraction. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritance.

3. How do you make a thread stop in Java?

Modern ways to suspend/stop a thread are by using a boolean flag and Thread. interrupt () method. We can define a boolean variable which is used for stopping/killing threads say 'exit' using a boolean flag. Whenever we want to stop a thread, the 'exit' variable will be set to true.

4. What purpose do the Volatile variable serve in Java?

Volatile keyword is used to modify the value of a variable by different threads and used to make classes thread safe. It means that multiple threads can use a method and instance of the classes at the same time without any problem.

5. Please compare Serialization with Deserialization in Java.

Serialization is a mechanism of converting the state of an object into a byte stream. Deserialization is the reverse process where the byte stream is used to recreate the actual Java object in memory. So, the object serialized on one platform can be deserialized on a different platform.

6. What do you understand by OutOfMemoryError in Java?

OutOfMemoryError is a runtime error which occurs when the Java Virtual Machine is unable to allocate an object due to insufficient space in the Java heap. The Java Garbage Collector cannot free up the space required for a new object.

7. Is Java platform independent?

Java is platform-independent because it does not depend on any type of platform. Programs are compiled into byte code and that byte code is platform-independent. Hence, Java is platform-independent language.

8. What all memory areas are allocated by JVM?

Memory allocation in java refers to the process where the computer programs and services are allocated dedicated to virtual memory spaces. The memory in the JVM divided into 5 different parts:

- Class Area.
- Heap.
- Stack.
- Program Counter Register.
- Native Method Stack.

9. Java vs. C ++?

C++ is derived from C and has the features of both procedural and object-oriented programming languages. C++ was designed for application and System development.

Java is built upon a virtual machine which is very secure and highly portable in nature. It is grouped with a comprehensive library to provide support for the abstraction of the existing platform.

10. Explain public static void main(String args[])

public static void main(String[] args) Java main method is the entry point of any java program. Its syntax is always public static void main(String[] args) . You can only change the name of String array argument, for example you can change args to myStringArgs.

Free PDF : Get our updated Java Course Content pdf

11. What is javac ?

The javac command reads source files that contain module, package and type declarations written in the Java programming language, and compiles them into class files that run on the Java Virtual Machine. The javac command can also process annotations in Java source files and classes.

12. What is class?

Java provides a class with name Class in java. lang package. Instances of the class Class represent classes and interfaces in a running Java application. The primitive Java types (boolean, byte, char, short, int, long, float, and double), and the keyword void are also represented as Class objects.

13. What is the base class of all classes?

lang. Object class is the root or superclass of the class hierarchy, which is present in java.

14. What is a wrapper class in Java?

A Wrapper class in Java is the type of class that provides a mechanism to convert the primitive data types into the objects and vice-versa. When a wrapper class is created, there is a creation of a new field in which we store the primitive data types.

15. Different Data types in Java.

- Primitive data types - includes byte , short , int , long , float , double , boolean and char.
- Non-primitive data types - such as String, Arrays and Classes (you will learn more about these in a later chapter)

16. What is Type casting in Java?

Type Casting is a feature in Java using which the form or type of a variable or object is cast into some other kind or Object, and the process of conversion from one type to another is called Type Casting.

17. What is an Array?

An array is a group of like-typed variables that are referred to by a common name. Arrays in Java work differently than they do in C/C++. Each item of an array is an element. All the elements in an array must be of the same type.

18. Does Java support Multiple Inheritance?

Java does not support multiple inheritance using classes. "A class can extend only one class but it can implement multiple interfaces." For example, below inheritance using multiple classes is wrong as two classes cannot be extended or inherited.

19. What is Polymorphism and what are the types of it?

The word "poly" means many and "morphs" means forms. So polymorphism means many forms. Polymorphism is a feature of OOPs that allows the object to behave differently in different conditions. There are two types of polymorphism in Java: compile-time polymorphism and runtime polymorphism.

20. What is method overriding in Java?

In any object-oriented programming language, Overriding is a feature that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes.

→ 2. Accenture – Java Interview Questions:-

Here is the list Java Interview Questions which are recently asked in Accenture company. These questions are included for both Freshers and Experienced professionals.

1. What is Polymorphism?

Polymorphism is the ability of an object to take many forms. It allows performing the same action in many different ways. Polymorphism in Java has two types: Compile time polymorphism (static binding) and Runtime polymorphism (dynamic binding).

2. What is runtime polymorphism or dynamic method dispatch?

Runtime polymorphism or Dynamic Method Dispatch is a process in which a call to an overridden method is resolved at runtime rather than compile-time. It is called as through the reference variable of a superclass.

3. What is abstraction in Java?

Data Abstraction is the property by virtue of which only the essential details are displayed to the user. We use abstract class and interface to achieve abstraction.

4. What do you mean by an interface in Java?

An interface in the Java programming language is an abstract type that is used to specify a behavior that classes must implement. It is a collection of abstract methods. A class implements an interface, thereby inheriting the abstract methods of the interface. Method bodies exist only for default methods and static methods.

5. What is the difference between abstract classes and interfaces?

An abstract class allows creating functionality that subclasses can implement or override. An interface only allows defining functionality, not implementing it and whereas a class can extend only one abstract class, it can take advantage of multiple interfaces.

6. What is inheritance in Java?

Java Inheritance is a mechanism in which one class acquires the property of another class. When an "Is-A" relationship exists between two classes, we use Inheritance. The parent class is called a super class and the inherited class is called a subclass

7. What are the different types of inheritance in Java?

- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

8. What is method overloading and method overriding?

Method Overloading

Method Overriding

It is used to increase the readability of the program

Provides a specific implementation of the method already in the parent class

It is performed within the same class

It involves multiple classes

Free PDF : Get our updated Java Course Content pdf

9. Can you override a private or static method in Java?

No, we cannot override private or static methods. Methods declared as private can never be overridden, they are in-fact bounded during compile time. Private methods in Java are not visible to any other class which limits their scope to the class in which they are declared.

10. What is multiple inheritance? Is it supported by Java?

Multiple Inheritance is a feature of object oriented concept, where a class can inherit properties of more than one parent class. The problem occurs when there exist methods with same signature in both the super classes and subclass.

11. What is encapsulation in Java?

Encapsulation is defined as the wrapping up of data under a single unit. It is the mechanism that binds together code and the data it manipulates. It is the process of hiding information details and protecting data and behavior of the object.

12. What is composition in Java?

Composition is a restricted form of Aggregation in which two entities are highly dependent on each other. It represents part-of relationship. In composition, both entities are dependent on each other. When there is a composition between two entities, the composed object cannot exist without the other entity.

13. What is a copy constructor in Java?

Copy constructor is a special type of constructor that creates an object using another object of the same Java class. It returns a duplicate copy of an existing object of the class. We can assign a value to the final field but the same cannot be done while using the clone() method.

14. What is a servlet?

A servlet is a Java programming language class that is used to extend the capabilities of servers that host applications accessed by means of a request-response programming model.

15. What is the life-cycle of a servlet?

Servlet life cycle can be defined as the entire process from its creation till the destruction. The servlet is initialized by calling the init () method. The servlet calls service () method to process a client's request.

16. How does cookies work in Servlets?

In cookies technique, we add cookie with response from the servlet. So cookie is stored in the cache of the browser. After that if request is sent by the user, cookie is added with request by default. Thus, we recognize the user as the old user.

→2(A)--<Accenture Java Interview Questions>:-

We have broken down the **Accenture java interview questions** into different sets. This is the set-1 of *Accenture Java developer interview questions* for experienced.

- Describe A Problem You Faced And How You Deal With It?
- How To Read And Write Image From A File?
- How ConcurrentHashMap Works?
- Can a Static Block Throw Exception?
- What Is the Difference Between Iterator Access And Index Access?
- What Is Java Reflection API?
- What Is The Difference Between Serializable And Externalizable Interfaces?

Accenture Java Developer Interview Questions & Answers

Question: Describe A Problem You Faced And How You Deal With It In Your Previous Work Experience?

Answer: It's a question to access your problem-solving skill and work experience. You can explain any issue you had faced during your project work and what solution you have implemented to resolve the issue.

Question: How To Read And Write Image From A File?

Answer: `javax.imageio.ImageIO` class is used to read and write image files either from local disk or from URL.

Other classes required to read and write an image from a file are,

```
import java.io.File class to read write image file.  
import java.io.IOException to handle exceptions while read write image file.  
import java.awt.image.BufferedImage object is used to store image in RAM.  
import javax.imageio.ImageIO is used to read and write the image files.
```

Question: What is ConcurrentHashMap and How ConcurrentHashMap Works?

Answer: ConcurrentHashMap allows concurrent access to the map. Part of the map called Segment (internal data structure) is only getting locked while adding or updating the map.

So Concurrenthashmap allows concurrent threads to read the value without locking at all. This data structure was introduced to improve performance.

Concurrency-Level defines the number which is an estimated number of concurrently updating threads. The implementation performs internal sizing to try to accommodate this many threads.

Load-Factor is a threshold, used to control resizing.

Initial Capacity is the implementation performs internal sizing to accommodate these many elements.

Question: Can a Static Block Throw Exception?

Answer: Static block can throw only a “RuntimeException”. If we want to catch a checked exception there should be a try-catch block.

Question: What is the Difference Between Index Access and Iterator Access?

Answer: Index Access – Index based access allow access of the element directly on the basis of index.

The cursor of the datastructure can directly goto the ‘n’ location and get the element.

It doesnot traverse through n-1 elements.

Iterator Access – In iterator based access, the cursor has to traverse through each element to get the desired element.

So to reach the ‘n’th element it need to traverse through n-1 elements.

Insertion, updation or deletion will be faster for iterator based access if the operations are performed on elements present in between the datastructure.

Insertion, updation or deletion will be faster for index based access if the operations are performed on elements present at last of the datastructure.

Traversal or search in Index based data structure is faster.

ArrayList is index access and LinkedList is iterator access.

Question: What Is Java Reflection API?

Answer: Reflection is commonly used by programs that require the ability to examine or modify the runtime behavior of applications running in the Java virtual machine.

The java.lang.Class provides many methods that can be used to get metadata, examine and change the run time behavior of a class.

The java.lang and java.lang.reflect packages provide classes for java reflection.

The Java Reflection API is mainly used in IDE (Integrated Development Environments) e.g. Eclipse, MyEclipse, NetBeans etc. Debugger, and Test Tools etc.

Question: What is the difference between Serializable and Externalizable Interfaces?

Answer: Serializable is a marker interface i.e. does not contain any method.

Externalizable interface contains two methods writeExternal() and readExternal() which implementing classes MUST override.

Serializable interface pass the responsibility of serialization to JVM and it's default algorithm.

Externalizable provides control of serialization logic to programmer to write custom logic.

Default serialization does not call any class constructor.

A public no-arg constructor is required while using Externalizable interface.

Externalizable interface is child interface of Serializable i.e. Externalizable extends Serializable.

You might be interested in

[Java Interview Guide](#)

1. What Is An Abstract Method?
2. What Value Does Read() Return When It Has Reached The End Of A File?
3. Can A Byte Object Be Cast To A Double Value?
4. What Is The Difference Between A Static And A Non-static Inner Class?
5. What Is An Object's Lock And Which Object's Have Locks?
6. What Is The % Operator?
7. When Can An Object Reference Be Cast To An Interface Reference?
8. Which Class Is Extended By All Other Classes?
9. Which Non-unicode Letter Characters May Be Used As The First Character Of An Identifier?

Accenture Java Interview Questions for Experienced

What Restrictions Are Placed On Method Overloading?

What Is Transient Variable?

What Is Collection Api?

What Is Casting?

What Is The Return Type Of A Program's Main() Method?

If A Variable Is Declared As Private, Where May The Variable Be Accessed?

What Do You Understand By Private, Protected And Public?

What Is Downcasting?

What Modifiers May Be Used With An Inner Class That Is A Member Of An Outer Class?

How Many Bits Are Used To Represent Unicode, Ascii, Utf-16, And Utf-8 Characters?

What Restrictions Are Placed On The Location Of A Package Statement Within A Source Code File?

What Is Assembly Condition Codes?

What Is Data Movement?

What Are Kinds Of Processors?

What Are Assembly Attributes?

What Are The Types Of Assemblies?

Explain An Intermediate Language?

What Is Assembly Language?

Accenture Real-Time Interview Questions

What Are The Advantages Of Assembly Language?

What Are The Basic Features Of Pc Hardware?

What Is Binary Number System?

What Is Hexadecimal Number System?

What Is Local Environment Setup?

How To Installing Nasm?

What Are The Assembly Program Sections?

What Is The Data Section?

What Is The Bss Section?

What Is The Text Section?

What Are The Assembly Language Statements?

What Is The Syntax Of Assembly Language Statements?

What Are Memory Segments?

What Are The Processor Registers?

What Linux System Calls?

What Are The Basic Modes Of Addressing?

What Is The Equ Directive?

Which Location Do You Want To Work In And Why?



<https://www.youtube.com/@codewitharrays>



<https://www.instagram.com/codewitharrays/>



<https://t.me/codewitharrays> Group Link: <https://t.me/cceesept2023>



[+91 8007592194](tel:+918007592194) [+91 9284926333](tel:+919284926333)



codewitharrays@gmail.com



<https://codewitharrays.in/project>